面向角色的服务架构

© 2026 S++ project contributors. All rights reserved. Book copyright.

English edition / 英文版

S++（SPP）的建模思想、语言机制与工程实践

本书面向熟悉面向对象开发的读者。角色说明参与者以什么身份进入业务，服务说明他们完成什么行为，场景组织协作，合约记录当事人的约定与结果。全书从“服务＝参与者×行为”出发，把角色建模、服务组合与工程实践连成一个体系。

正文用业务例子解释概念，可以顺序阅读。形式化定义、证明、反例与性能推导集中在附录。公式采用普通文本，不依赖 LaTeX 插件；代码片段链接到案例源码。

全书分为七篇、35章及六篇附录。每篇有独立目录和阅读介绍，每章有本篇目录、上一章与下一章导航。

第一篇 角色与服务的基本模型

- 第1章 从对象世界到业务活动
- 第2章 服务等于参与者乘行为
- 第3章 角色与业务身份
- 第4章 服务的分类与场景组织
- 第5章 量子行为与量子服务
- 第6章 量子服务的叠加交换与纠缠
- 第7章 查询、计算、校验与普通更新的特点
- 第8章 通过角色多态复用服务与场景

第二篇 服务协作与执行

- 第9章 服务协作与执行顺序
- 第10章 并行执行与服务调用成本
- 第11章 服务最小颗粒度与部署原则

第三篇 用 S++ 表达角色与服务

- 第12章 SPP APP PHY 的分层职责
- 第13章 Entity Reference Role Contract
- 第14章 用 take … as 声明扮演关系
- 第15章 四类原子服务
- 第16章 业务计算规则与集合参与者
- 第17章 场景中的串行并行与合约形成

第四篇 一致性与执行保障

- 第18章 业务事实参与者身份与存储归属
- 第19章 事务边界执行计划与部署能力

- 第20章 量子属性调用范围与守恒成交
- 第21章 回滚反交易冲正与补偿
- 第22章 生成链路物理资源与运行配置

第五篇 业务与技术分离

- 第23章 业务与技术分离的主要收益
- 第24章 缓存与按需数据提升性能
- 第25章 加解密指纹与安全服务
- 第26章 幂等防重与可靠执行
- 第27章 运行治理与技术演进

第六篇 完整实践 Petstore

- 第28章 从需求识别角色行为与服务
- 第29章 零开户充值与入库
- 第30章 报价下单付款与发货
- 第31章 取消退货预约与失败处置
- 第32章 技术配置部署与综合验证

第七篇 进阶案例与架构方法

- 第33章 银行存款偿还分配与冲正
- 第34章 从现有面向对象系统迁移
- 第35章 架构设计评审与能力边界

附录

- 附录 A 服务基本模型与形式化推导
- 附录 B 交换重排与并行的成立条件
- 附录 C 性能模型与实验方法
- 附录 D 量子服务与守恒机制
- 附录 E 语言与工程参考
- 附录 F 资料来源与验证证据

编辑资料

待讨论问题单独记录尚需作者讨论的定义和契约，不属于书稿阅读顺序。案例源码与验证基线见附录 F。

第一篇 角色与服务的基本模型

© 2026 S++ project contributors. All rights reserved. Book copyright.

从一个人在购买、付款和经营中的不同身份开始，认识参与者、行为、角色、服务与合约。用转账和下单解释服务的分解、最小颗粒度、组合，并通过角色多态复用计算、校验与完整场景。同时说明查询、计算、校验与普通更新的特点及协作规则。读完这一篇，应能用“谁以什么身份完成什么行为”描述一项业务。

本篇目录

- 第1章 从对象世界到业务活动
- 第2章 服务等于参与者乘行为
- 第3章 角色与业务身份
- 第4章 服务的分类与场景组织
- 第5章 量子行为与量子服务
- 第6章 量子服务的叠加交换与纠缠
- 第7章 查询、计算、校验与普通更新的特点
- 第8章 通过角色多态复用服务与场景

返回全书目录

第1章 从对象世界到业务活动

© 2026 S++ project contributors. All rights reserved. Book copyright.

面向对象与面向角色都要表达实体及其行为，侧重点在于选择什么作为建模的主体。面向对象以对象实体为中心，围绕对象组织属性与动作；面向角色以业务行为为中心，围绕完整的业务目标组织参与者。前者从对象出发描述它能做什么，后者从行为出发描述谁以什么身份参与、怎样完成这项业务。

本书关注这套建模方法在商业服务架构中的应用。面向角色作为建模方法的表达范围，以及它如何构成复杂数据结构和算法，将在第七篇第35章进一步讨论。

以对象为中心组织动作

一家商店接受订单时，系统里会出现客户、商品、库存、订单等概念。面向对象通常先建立这些对象，再把动作组织在对象之上：客户维护资料，商品调整价格，库存预留数量，订单改变状态。业务过程通过对象之间的协作完成。

在这种组织方式中，动作依附于所属对象，模型的关注点是对象具有哪些属性和能力，以及不同对象怎样相互调用。接口、继承和组合可以帮助复用能力，组织业务时仍以对象及其能力为起点。

以业务行为为中心组织参与者

面向角色先描述“购买”这一完整行为：买方选择商品，与卖方形成成交约定，获得相应的商品或交付承诺。然后识别买家、卖家、商品等参与角色，再由具体实体扮演这些角色。

例如，个人为家庭购买用品，公司为办公室采购设备，实际买方分别是个人和组织。它们都可以扮演买家，只要满足这项购买行为所要求的资格、事实和义务。购买行为围绕买家角色展开，具体由哪个实体扮演买家，可以更换。

同一个实体也可以进入不同的行为。小林购买商品时扮演买家，经营商店时扮演卖家。参与身份随着业务行为变化，不需要把小林这个自然实体理解为变成了另一个人。

因此，面向角色将业务活动本身作为明确的建模对象。购买的目标、参与关系和完成条件是模型的主线，对象以参与者的身份加入这条主线。对象仍然提供事实并承载实际变化，但业务行为的完整含义不依附于某个特定实体。

从一次购买开始

小林购买两件商品。商店确认小林具备购买资格，核对商品和数量，计算成交价格，预占库存，形成订单。客户资料与商品资料是事实基础，购买资格、成交价格、预占数量和订单约定则属于这次业务活动。

沿着购买行为阅读模型，小林以买家的身份参与，产品以可售商品的身份参与，库存资源参与履约，订单记录成交约定。每项查询、计算和更新，都需要放回这次购买中解释：它为哪个参与方准备事实，为哪个约定形成结果，怎样帮助整笔业务完成。

假设库存已经预留两件商品，订单却没有保存成功，单项操作的完成并不等于购买已经成立。行为的完整性要求我们继续说明成交约定是否形成、已有变化怎样处理。这些要求从“完成一次购买”的目标出发，再落实到各参与者的操作上。

动作与行为

动作是对象能够执行的操作，行为是具有内在逻辑和目的的活动。修改一个字段是动作；确认一笔付款是行为。前者告诉我们发生了什么局部变化，后者还要说明变化为何发生、涉及哪些参与者，以及怎样才算完成。

这种区分有助于避免按技术步骤直接划分业务服务。序列化、发请求、更新一行记录都是运行系统中的操作，但它们各自未必构成完整的业务行为。支付服务的目标也不能仅用“请求已经发出”来判断；它需要形成能够解释的成交结果。

同一个对象的不同参与身份

一个人在不同活动中可以是买家、商家、付款方或账户所有者。这些身份的资格和事实不同。买家关心会员资格与购买关系，商家关心经营属性，付款方关心所使用的资金资源。让一个 Person 类承担所有规则，容易使互不相关的业务变化汇聚到同一个模型。

S++ 用角色表达参与身份。实体提供自然事实，角色说明这些事实如何进入具体服务。实体与角色的关系将在第一篇第3章《角色与业务身份》展开；这里先抓住一个变化：读业务模型时，我们开始沿着“谁以什么身份参与什么行为”寻找意义。

用结果约束模型

业务建模需要回答完成条件。下单成功意味着成交内容和库存预占符合约定；付款成功意味着资金变化与付款状态符合约定。明确这些结果，才能判断服务分解是否合理、哪些步骤需要共同成功，以及哪些技术失败需要恢复。

面向角色的服务架构保留了对象世界中的事实，同时把业务活动、参与身份和协作关系显式表达出来。下一章将用一个简单模型连接这几个概念：服务等于参与者乘行为。

本篇目录 · 全书目录 · 下一章

第2章 服务等于参与者乘行为

© 2026 S++ project contributors. All rights reserved. Book copyright.

“转账”是一个行为名称，一个行为是如何形成业务的，需要知道谁提供资金、谁接收资金、金额是多少，以及转账满足什么约束。反过来，两个账户放在一起，也不会自行完成转账。参与者与行为共同构成有业务意义的活动。

“服务＝参与者×行为”中的“服务”，采用传统意义上的宽泛概念，例如通常所说的下单服务、支付服务：参与者共同完成一项业务，形成符合目标的结果。乘号强调双方共同成立：行为需要参与者承载，参与者需要进入行为才能产生业务结果。

SPP 进一步把这项业务中的**服务**与**场景**区分开来。服务承担明确的业务职责，场景围绕完整业务目标组织参与者与服务。本章先说明共同的业务模型，再说明它怎样映射到这两个概念。

一个简单的表达

把一次业务活动写为：

```
B = p × b
业务活动 = 参与者 × 行为
```

其中，p 表示参与者或参与者组合，b 表示具有业务目的的行为，B 表示形成目标与结果的业务活动。传统意义上的一个完整业务服务可以由多项活动组成，因而写为各项活动的组合。完整记号与适用条件见附录 A。

这里的乘法是一种建模表达。它不意味着可以把账户数值和程序方法直接相乘，也不意味着任意业务步骤都天然服从普通代数的运算规则。它让我们检查一个服务定义是否缺失参与方、行为或结果。

参与者决定行为的具体含义

同样是支付，买家向商家付款、商家向买家退款、结算账户向钱包充值，参与关系各不相同。把这些活动都写成一个名为 pay 的方法，很难看出它们的业务区别。模型需要明确谁承担哪个角色，行为改变哪些事实，以及通过什么合约表达本次约定。

参与者也不等同于接口里的一组基本参数。账户号可以定位账户，却不能完整表达付款方的资格、账户事实与身份确认状态。金额需要属于明确的付款约定，否则调用者难以判断这个数字究竟是报价、实付还是退款。所以参与者不是传统意义上的Class，而是需要具备一定的角色信息。

从完整业务到服务与场景

通常所说的“订单付款服务”，需要查询买家、商家和订单，确认付款资格，准备金额，完成资金转移，并记录订单状态与办理凭证。这些活动共同兑现“完成订单付款”的目标。

在 SPP 中，它们有两种不同的组织责任：

概念	订单付款中的含义
服务	查询参与者、检查资格、准备金额、转移资金、保存订单等明确职责
场景	组织买家、商家、订单与付款合约，组合所需服务，表达顺序、事务边界与完成要求

例如，资金转移服务说明买家钱包减少、商家资金增加；付款场景还要说明该付哪笔订单、金额从哪里取得、具备什么资格，以及资金变化怎样与订单记录共同形成付款结果。仅调用资金转移，不能代替这一整项业务。

因此，分析时先说清楚“买家与商家按照付款约定完成订单付款”，再识别其中的服务职责，并用场景组织它们。模型从业务目标出发，SPP语言把局部职责与整体协作分别表达。

本篇目录 · 全书目录 · 上一章 · 下一章

第3章 角色与业务身份

© 2026 S++ project contributors. All rights reserved. Book copyright.

产品在产品目录中有名称、规格和成本。进入销售活动之后，还会有售价、上架状态和销售文案。产品与可售商品有密切联系，却不承担完全相同的业务责任。S++ 用 Product 表达产品，用 Goods 表达销售中的商品角色。

角色使业务参与身份成为模型的一部分。它可以借用扮演者的事实，也可以拥有在该参与关系中形成的新事实。

身份与资格

同一个 Person 可以扮演 Buyer 和 Merchant。一个人的姓名属于自然事实，会员等级属于买家身份，店铺名称属于商家身份。创建一个人并不自动说明他可以经营商店；业务需要分别建立和检查相应的角色资格。

这种建模方式使变化更容易定位。会员优惠发生变化时，讨论可以集中在买家角色和定价服务。客户姓名的维护则仍归客户域。角色连接业务活动与实体事实，也帮助我们区分事实的所有权。

联合扮演

有些角色需要多个参与方共同确定。购物车明细既涉及商品，也涉及买家：同一商品在不同买家的购物车中有不同数量。因此，商品与买家共同扮演 CartLine，数量是这个组合角色自己的事实。

联合扮演提供了一种表达关系业务的方式。识别角色时，可以问它的事实离开哪些参与方就失去意义。若购物车数量脱离了买家与商品便无法解释，它的归属就应体现这种关系。

角色可以继续扮演角色

商品进入订单时形成成交明细。商品的当前售价与订单明细的成交单价可能不同，因为成交事实需要保留当时的约定。Goods 可以进一步扮演 OrderLine，明细拥有成交数量、单价和金额。

例如，小林下单时，商品叫“幼猫粮”，单价是60元。后来商店把商品改名为“幼猫配方粮”，并将售价调整为65元。商品页面可以显示新的名称和价格，旧订单仍应记录当时的“幼猫粮”和60元，说明小林那次购买的具体内容。

因此，商品扮演订单明细时，需要区分哪些信息读取商品的当前值，哪些信息要在成交时单独记下来。扮演关系说明它们之间的联系，不表示商品修改后，订单中的名称和单价也要跟着修改。

角色与对象类型的区别

对象类型说明一个实体是什么，角色说明它以什么身份参与一项业务。例如，小林是一个人，在商店购物时扮演买家，经营自己的店铺时扮演卖家。他仍然是同一个人，但两种身份承担不同的业务要求：买家确认购买内容并付款，卖家按约定提供商品。

反过来，同一种角色可以由不同类型的实体扮演。小林为家庭购买用品，公司为办公室采购设备，个人与公司都可以扮演买家。购买服务围绕买家角色约定所需事实、资格和义务，具体由谁来扮演这个角色，可以更换，只要满足这些要求。

因此，角色与对象类型不是一一对应的。一个实体可以扮演多种角色，一种角色也可以由不同实体扮演。面向角色建模时，先从业务行为中识别需要哪些参与身份，再确定由哪些实体承担这些身份。

本篇目录 · 全书目录 · 上一章 · 下一章

第4章 服务的分类与场景组织

© 2026 S++ project contributors. All rights reserved. Book copyright.

一次订单付款既包含业务活动，也需要技术支持。确认买家资格、计算金额、转移资金，是付款业务的组成部分；解析请求、转换数据、保护通信、访问存储，则帮助这些业务活动实际运行。先区分两类服务，再细分业务职责，才能看清场景中每一项服务的作用。

技术服务与业务服务

业务服务直接表达业务目标和规则。例如，判断订单是否允许付款、计算应付金额、把资金从买家转给商家，都能从付款需求中找到依据。

技术服务提供执行这些业务所需的手段。例如，接收 HTTP 请求、解析 JSON、加解密报文、建立数据库连接。改用另一种通信协议，付款方、收款方和应付金额仍应保持相同业务含义，变化发生在技术实现上。

区分时要看具体职责，不能只看服务名称。“校验请求签名”属于技术安全，“校验买家是否有购买资格”属于业务规则。业务服务声明需要什么事实和结果，技术服务负责按约定取得事实、传递信息和保障执行。

四类业务服务

业务服务按职责分为查询、校验、计算和更新四类。以付款为例，可以这样识别：

类别	回答的问题	付款中的例子
查询	参与者有哪些事实?	取得订单、买家和商家的资料
校验	是否满足业务条件?	检查订单归属及是否允许付款
计算	按规则应该得到什么结果?	根据订单准备本次付款金额
更新	哪些业务事实实际发生变化?	转移资金，记录付款状态与凭证

普通查询和普通更新是内置系统服务，开发者不能修改其实现；业务计算和校验在场景中单独调用。查询补全事实，校验判断条件，计算准备结果，更新实施变化。计算出应付100元，并不表示已经付款；只有相应变化实际完成，才能形成付款结果。

这些类别也适用于其他业务。注册时查询已有登记、校验注册资格、准备登记内容，再建立参与身份；报价时查询商品和会员信息、检查报价条件并计算价格，可以不发生成交更新。

描述性属性更新

描述性属性说明参与者是什么、具有怎样的资料或状态。例如，客户更换地址，商家修改店铺名称，商品从上架变为下架。更新服务记录这些变化。

数字也可以是描述性属性。例如，商家把商品售价从60元调整为65元，是改变报价依据，没有因此从某个账户搬走5元。判断属性类别，需要看业务含义，不能仅根据它是字符串还是数字。

这类更新仍需要防止竞争覆盖，也可能与其他变化共同完成一项业务。它的特点是修改资料或状态，具体保护方式将在一致性部分讨论。

可量化交易属性更新

另一类属性表示交易中的数量，例如钱包余额和商品库存。它们的变化需要说明增减多少、来源与去向，以及允许的范围。

买家支付100元，钱包减少100元，商家资金增加100元；预占两件商品，可售数量减少2，预留数量增加2。业务关注的是这次转移及双方结果，不能只把一个字段赋成新值就认为交易已经完整发生。

假设买家钱包原有500元，查询后又收到20元充值。此时付款100元，应从实际余额520元中扣减，得到420元；若直接把查询时算出的400元写回，就会抹去后来的充值。因此，交易属性更新需要表达实际增减，并由成交机制处理竞争、范围与相关方的一致变化。

一对量子属性之间的守恒变化，构成最小的量子服务。后面的章节将说明这种服务怎样表达来源与去向，以及怎样共同成交。

场景组织这些服务

四类服务分别承担职责，场景把它们组织成完整业务。付款场景说明先取得哪些事实、检查什么资格、怎样准备金额，再安排资金变化、订单状态和凭证记录。

一次业务可以同时涉及两类更新：付款转移资金，也改变订单状态。资金变化与状态变化各有含义，场景需要说明它们怎样共同形成付款结果；技术层则提供相应的存储与执行保障。

从服务分类走到场景组织，既保留了每项职责的清楚含义，也保留了整笔业务的完整目标。下一章从交易属性的增减出发，介绍量子行为与量子服务。

本篇目录 · 全书目录 · 上一章 · 下一章

第5章 量子行为与量子服务

© 2026 S++ project contributors. All rights reserved. Book copyright.

修改联系地址和转移100元，都表现为数据发生变化，但它们对业务系统提出的要求差别很大。联系地址描述客户的资料，资金变化则直接改变各方持有的财产。若把它们都当成一般的字段更新，就容易忽略交易在时间、可靠性和并发处理上的特殊要求。

理解这些差异，才能说明为什么要定义量子服务，以及为什么保障它高效、稳定地执行，是资产流转类业务系统的核心重点。

时效性差异

客户修改备用联系地址，只要在下一次使用之前生效，通常不会影响已经完成的业务。这样的资料维护可以按需要安排同步时间，即使次日生效，也可能完全满足使用要求。若地址直接用于正在办理的配送，则要按照该项业务的时效要求处理。

付款往往不同。买家等着付款确认后提货，商家等着收款后发货。交易迟迟不能确认，双方就无法继续后续业务。转账也需要尽快明确资金是否到达，而不是只收到“修改请求已提交”的答复。

因此，交易系统需要缩短从请求到确定成交结果的时间。这里的快包含可靠确认：更早返回一个无法说明是否到账的消息，并没有真正加快业务完成。

可靠性差异

地址保存失败，常见结果是仍保留原地址，客户可以重新修改。资金转移失败则必须进一步确定钱有没有扣、有没有到账、是否重复执行。少付、重复扣款或只完成一侧，都可能造成真实财产损失。

例如甲向乙转100元。甲少了100元，乙没有增加，是一笔没有完整完成的转账；甲扣了两次，乙只收到一次，同样是错误。转账不能只检查每个更新方法有没有返回成功，而要核对双方共同形成的结果。

一笔交易已经完成、响应却没有送达时，重试也不能再次转移100元。系统需要识别原交易并确认结果。可靠性要求由此覆盖完整成交、重复处理和故障后的结果确认。

并发性差异

一个客户通常很少修改联系地址；同一个账户却可能连续收款、付款，多个请求同时改变余额。热门商品也可能在短时间内被许多买家争购。

这种竞争会发生在同一份资源上。最后一件商品被两位买家同时选中，不能让两笔预占都成功；两个请求分别拿着旧余额计算，再覆盖回去，也可能漏掉其中一笔资金变化。

因此，交易服务需要在高频请求下保护真实增减和成交范围，同时避免把所有不相关请求都挤进一条等待队列。不同账户、不同商品可以分别处理，同一资源上的竞争则必须得到正确裁决。

更新含义与守恒的差异

地址更新说明新的资料是什么。资金交易还要说明这一笔从哪里转出、转给谁、转了多少。甲减少100元，乙增加100元，双方变化之和为零；库存从可售转到预留，也需要交代数量的去向。

所以，交易属性更新的基本表达是本次增减及其关联方。它不能简单地把事前计算的新余额覆盖到库里，因为在查询之后，可能已经有其他交易完成。

守恒还要求模型覆盖真实来源与去向。充值增加钱包余额，应同时说明对应资金来源；入库增加可售数量，也应说明供应来源。否则系统只有增加后的数字，无法完整解释资源怎样进入。

失败处理与可追踪性的差异

在没有其他修改介入的情况下，把同一个地址再保存一次，仍然得到相同资料；把同一笔入账再执行一次，却会增加第二笔金额。交易需要明确身份，让系统知道这是原请求重试还是一笔新业务。

交易结束后若需要撤销，应依据原来的实际增减处理，并保留后续已经发生的业务。把整个余额恢复成旧值，可能抹去后来收到的其他款项。原交易身份、来源、去向、金额和结果，因而都需要能够追踪。

这些要求使成交与恢复成为交易服务的一部分，而不是发生错误后临时补上的处理。

为什么定义量子服务

上述差异集中到同一个问题：怎样将交易变化表达为清楚、不可任意拆开的业务单位，并为它提供专门的执行保障。SPP 将一对量子属性组成的守恒服务定义为量子服务：一侧减少，另一侧增加，二者共同表达同一业务维度上的真实转移。这就是量子服务的最小颗粒度。

以甲向乙转100元为例，可以分别分析甲减少和乙增加，但不能把只扣款当作一笔完整转账。继续拆开之后，各个片段便无法独自兑现这次资金转移的目标。量子服务要保留的就是这一完整业务意义。

假设甲有1000元，乙有200元。转移完成后为900元与300元；转移被拒绝时仍为1000元与200元。不能把900元与200元的局部状态当作完整成交结果。实际如何保障，由相应提交和恢复机制承接，详细论证见附录 D。

核心重点是高效稳定地成交

对于支付、账务、库存等系统，资料查询、校验和计算为交易准备事实与结果，资产的实际变化则发生在成交环节。只加快页面或查询，而让成交长时间排队、重复执行或无法确认，整项业务仍然不能可靠完成。

量子服务的高效执行意味着在规定的资源条件下，尽快处理合法交易，并承受预期并发；稳定执行意味着不丢失、不重复、保持范围与守恒，遇到故障仍能确认或恢复结果。

因此，架构应把准备工作与实际成交职责分清，围绕量子服务安排资源、调度和恢复，使最关键的交易路径清楚而可验证。效率和可靠性共同决定业务能否持续运行，不能只靠减少检查获得表面上的速度。

在服务与场景中表达

对于余额和库存增减，SPP 使用 quantum update 定义变化，调用者说明允许的范围。第四篇第20章《量子属性调用范围与守恒成交》会进一步介绍属性、约束与实际成交。

付款场景仍要组织订单查询、资格校验、金额准备、资金转移和凭证记录。量子服务聚焦完整的交易变化，场景把它与其他业务职责连接起来，形成整笔付款。

本篇目录 · 全书目录 · 上一章 · 下一章

第6章 量子服务的叠加交换与纠缠

© 2026 S++ project contributors. All rights reserved. Book copyright.

量子服务与传统更新服务的一个重要区别，是怎样对待交易过程中的数值。传统更新通常先取得并锁定当前值，再计算一个确定的新值；量子服务表达本次增减，即使本次变化已经成交，成交后的属性值也不能仅凭事前查询和本次增减确定。

S++ 借用量子系统的叠加、交换和纠缠概念，说明量子服务的数值状态、执行次序和变化关系。先从余额更新理解叠加，再讨论交换与守恒。

叠加允许成交后的数值不确定

账户有1000元，本次扣除100元。传统更新的一种做法，是在事务中通过 `select for update` 锁定账户记录，读出1000，再计算并写入900。在这段受保护的更新期间，其他请求不能修改同一记录，因此本事务可以确定这次更新得到900元。

量子服务表达的是“扣除100元”这项变化。查询时看到1000元，并不意味着要把这个值锁定到本次事务结束，也不意味着最终必须得到900元。期间其他请求还可能对同一账户产生变化。

例如，本次请求查询到1000元，准备扣除100元；另一笔并发付款扣除200元，也已生效。本次扣款完成时，得到的余额便可能是700元：

```
查询时余额：1000元
本次变化：−100元
另一笔并发变化：−200元
两笔均成交后的余额：1000 − 100 − 200 = 700元
```

本次扣除的100元已经成交，但这并不确定成交后观察到的余额就是900元。其他请求的成交与执行先后也会影响这个余额。量子服务保证本次增减得到正确执行，允许成交后的属性值受到并发变化影响；要知道具体数值，需要在观测时取得相应事实。

这就是这里借用“观测前处于叠加态”的含义：本次服务成交，不等于成交后的属性值只能是一个由本次操作推算出的数值。若没有其他变化，观察到的余额可以是900元；若另一次扣款200元也已生效，则可以是700元。两种情况中，本次扣除100元都可以已经成功成交。

因此，叠加描述的是量子服务成交后、观测前的数值不确定性。成交确认说明本次变化是否成功，数值观测说明当时属性值是多少；前者不能代替后者。执行机制仍需落实范围、守恒与成交要求。附录 D 的 D12 节进一步解释成交与观测的区别。

交换表示量子服务的执行顺序无关

同一个账户有1000元，量子服务 A 扣除100元，量子服务 B 扣除200元。先执行 A 再执行 B，或者先执行 B 再执行 A，两项变化全部成交后，余额都是700元：

```
先 A 后 B: 1000 − 100 − 200 = 700
先 B 后 A: 1000 − 200 − 100 = 700
```

这就是量子服务的交换性：多个量子服务的执行顺序不影响它们共同产生的结果。每项服务表达本次确定的增减，不把执行过程中读到的余额当作后续服务必须遵循的固定值。因此，即使它们作用于同一个量子属性，也可以交换执行顺序。

交换性与叠加相互联系。执行过程中，先发生哪一项变化、某次观察读到多少，可以不同；这些中间数值不决定已经准备好的增减量。全部变化成交后，它们共同形成的结果保持一致。这为量子服务的乱序与并行执行提供了基础。附录 B 的 B0 节给出两项及多项量子变化的证明。

纠缠落到守恒关系

钱包支付 100 元，商家收到 100 元，双方变化之和为零。可售库存减少 2，预留库存增加 2，同样表达一个守恒转移。这些变化互相约束：缺少对侧，就无法完整说明钱或商品转到了哪里。守恒成交体现这种联系。

资金与库存即使都属于量子属性，也不能把数值直接相加来证明守恒。金额和数量具有不同业务维度，不同商品也需要按各自身份分别检查。它们可以在一个场景里协作，守恒组则要依据真实关系确定。

多个量子服务可以合并提交

交换性还带来一个直接的执行优势：多个量子服务可以合并成一个大的量子请求，一起提交。各项服务已经说明参与方和确定的增减量，执行机制可以收集这些变化，统一交给量子执行端处理。

例如甲向乙转100元，甲向丙转200元。逐项提交时，需要分别发送两次转账请求、接收两次结果；合并提交时，可以用一个请求同时携带这两项变化：

```
量子服务 A：甲 −100，乙 +100
量子服务 B：甲 −200，丙 +200

合并请求：{A，B}
共同变化：甲 −300，乙 +100，丙 +200
```

由于两项量子服务可以交换执行顺序，提交方不必为了固定先后次序，等待第一项完成后才发送第二项。执行端可以统一安排这些变化，按共同的提交边界处理。业务模型仍然可以用两项清楚的服务表达职责，实际通信则合并为一次请求。

这样减少了逐项调用产生的网络往返、请求封装和协议处理开销。服务数量越多、单项变化越小，这些固定通信开销就越值得合并。降低通信损耗的依据，正是量子服务的交换性；附录 D 的 D6 节进一步说明合并提交与增量汇总的关系。

本篇目录 · 全书目录 · 上一章 · 下一章

第7章 查询、计算、校验与普通更新的特点

© 2026 S++ project contributors. All rights reserved. Book copyright.

量子服务处理余额、库存等可量化属性的变化。完整业务还需要取得事实、计算金额、判断资格，以及保存资料和办理状态。这些服务各有特点，场景应根据它们的职责安排协作，应用则据此选择合适的执行方式。

以付款为例，系统查询买家和商家的资料，校验订单能否付款，准备付款金额，再执行资金转移并保存订单与凭证。查询、计算、校验和普通更新都为这项业务提供明确职责。

大部分查询天然适合并发

普通查询和普通更新均由系统内置提供，开发者只能在场景中调用，不能修改其服务实现。业务规则由场景中单独的计算与校验服务表达。

查询的主要任务是取得已有事实。查买家资料不会扣除商品库存，查商品价格也不会改变买家资料。当两项查询的输入已经具备，且不需要对方的结果时，通常可以同时发起。

Petstore 下单场景同时查询买家与商品，付款场景同时查询买家与商家。查询完成后，场景再使用已经补全的参与者事实。这样可以把两段先后等待重叠起来，尤其适合需要访问数据库或外部业务域的查询。

“大部分”保留了必要的依赖。例如先找到订单，才能知道要查询哪些商品，商品查询就要等待订单查询。查询还会把结果补入内存中的参与者；如果两个查询同时覆盖同一个参与者的同一属性，也需要明确回填关系。因此，判断并发时看的是输入依赖和事实补全范围。

查询成功还需要确认找到了相应参与者。查询没有找到买家与访问客户系统失败，是两种不同结果。场景不能把未确认的资料当作完整输入继续计算。

计算可以复用其他计算

计算服务使用已经准备好的事实，按公式形成结果。例如明细金额等于单价乘数量，订单小计等于各明细金额之和，再根据会员等级计算折扣。

这些公式可以分别定义并复用。Petstore 的 `quoteOrder` 在处理订单明细时调用 `priceLine`，使用算出的明细金额汇总订单小计。计算结果直接写入参与者，后续计算使用这些共同事实。

计算服务允许调用其他计算服务，但不能在计算过程中查询数据库、调用校验、实施更新或启动场景。如果报价缺少商品价格，应由场景先查询；如果数量是否合法需要判断，应由场景先校验。这样，读者看到计算服务时，就能沿已有事实理解结果是怎样算出来的。

计算服务之间的调用关系不能成环：A 直接调用 A，或者 A 调用 B、B 再调用 A，都属于不允许的递归调用。编译器检查服务调用图，发现环就报错，即使递归写有退出条件，也会被拒绝。这个限制不禁止在计算体中使用 for 或 while 逐步迭代。例如从第一条明细开始累计订单小计，可以用循环完成，也可以在每轮调用明细计算服务，只要服务之间的调用关系没有环。检查调用图不会证明循环一定终止，循环仍需有明确的结束条件。

原子服务的业务协作由场景组织

原子服务承担一项明确职责，普通原子服务不允许调用其他服务或场景。查询、校验、计算与保存分别完成自己的任务，业务协作在场景中显式组织。普通查询与普通更新均为内置系统服务：查询按参与者补全事实，更新保存已准备完整的事实，开发者不修改这两类服务的实现。

计算之间的纯计算复用是明确允许的规则：计算可以调用计算，整个调用仍然用于形成计算结果。它不能借此扩展为查询、校验和更新的混合流程。

例如付款场景显式安排“查询订单、校验付款资格、准备付款金额、转移资金、保存付款凭证”。读者能够沿场景看清什么时候发生资金变化，哪些事实在什么时候保存。原子服务的调用限制使每项职责及其组合关系保持可见。

内置查询与更新所需的仓储访问由系统按照 APP/PHY 配置执行。业务开发者在场景中选择参与者和调用位置，数据库、映射和协议由相应配置承接。

校验决定场景怎样继续

校验服务判断已有事实是否满足业务条件，不修改参与者。订单能否付款、当前办理状态能否发货、退货原因是否填写，都可以由校验表达。

校验结果有直接的流程意义。普通校验成功后继续执行，失败则结束所在的执行范围。例如付款资格不满足，场景就不继续准备付款和扣款。若只想控制局部动作，可以让校验包围这一范围：允许时执行，不允许时跳过，然后继续办理后面的业务。

场景还可以明确要求校验失败时撤销当前事务。这与结束一个执行范围有不同作用：前者要求处理已经发生的事务变化，后者只是决定后续语句是否执行。因此，校验失败是否撤销，由场景明确表达。第三篇会介绍对应的语言写法。

计算与校验也应分清：计算回答“金额是多少”，校验回答“是否允许继续”。计算服务不能把拒绝业务藏在公式内部；多个校验之间也不互相调用，它们在场景中按需要排列。

计算和校验适合在本地执行

当参与者事实已经准备好，计算和校验通常只需要访问内存中的数据。算出明细金额、判断订单状态是否允许付款，都可以直接在场景所在进程完成，无需为每条公式或每个条件访问远程服务。

这两类服务均可本地部署。在常见业务中，按依赖顺序直接执行即可，不需要为了每项计算或校验再安排并发任务。例如先计算明细，再汇总小计，最后判断付款条件，流程清楚，也省去了通信、任务调度和结果汇合的开销。

不同业务请求仍可以同时运行。某个请求内部的计算与校验按顺序执行，不意味着整个系统一次只能处理一个请求。若以后遇到大量明细或耗时计算，可以依据实际工作量另行设计执行方式；第二篇会讨论这些优化的判断方法。

规则所需的事实也应先准备好。例如判断会员折扣需要会员等级，查询负责取得等级，计算负责使用它。这样本地计算和校验能够保持职责清楚，不会因隐藏的外部访问而突然变慢。

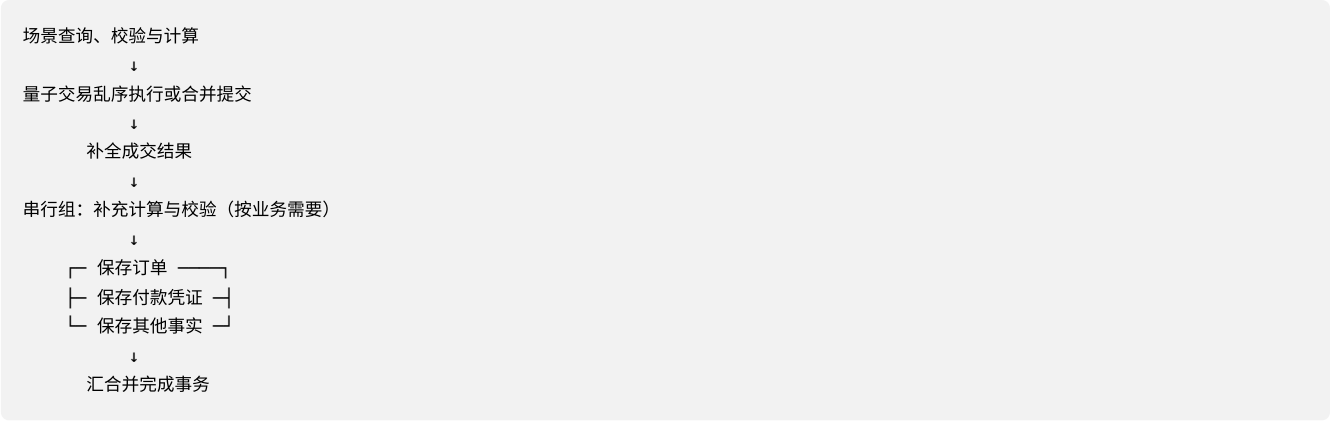
普通更新在量子服务之后并发执行

普通更新是系统内置服务，开发者不编写或修改它的更新实现。服务内部不做业务查询、计算或校验，也不调用其他业务服务。它需要的全部信息，由场景中的查询、计算和量子服务提供：查询取得已有事实，计算形成待保存事实，量子服务完成真实增减并补全成交结果。普通更新只负责保存已经准备完整的信息。

例如付款时，订单、付款金额和凭证内容由场景准备，资金转移由量子服务完成。进入普通更新阶段后，保存订单不需要等待保存凭证提供信息，保存凭证也不需要再向订单服务查询。两项服务使用已经完整的参与者事实，各自完成保存。

因此，在同一个事务边界内，SPP 编译器自动把量子交易提前到所有普通更新交易之前。普通更新只保存已补全的事实，不会改变量子交易的输入、增减量或成交结果，量子交易不需要等待这些保存动作。量子交易自身又符合交换律，可以乱序执行，也可以合并请求一起提交。

量子阶段完成后，若还需要准备待保存事实，场景先完成相应计算和校验；信息完整后，边界内的普通更新可以并发执行。交换它们的执行次序，不会改变已经准备好的业务事实，也不会改变这些事实全部保存后的结果。这是普通更新服务符合交换律的原因。开发者遵从服务规范、表达完整业务，编译器据此生成量子优先的执行计划。



需要开发者规划的是场景中保存之前的计算与校验。例如量子付款已经完成，还要根据成交结果计算凭证中的结余、生成凭证内容，再校验凭证是否完整。开发者应使用串行组明确安排“取得量子结果、计算凭证、校验凭证、保存凭证”的依赖；计算或校验没有完成，就不能启动依赖它们的保存。

串行组确定各阶段的先后；其中所有输入已经准备完整的普通更新，仍可用并行组一起执行。这里的顺序由场景表达，内置更新始终只保存当前完整事实。附录 B 的 B12 节给出普通更新的交换关系，B13 节说明量子优先与场景准备阶段的关系。

同一事务内部的更新并发，与不同事务竞争修改同一份资料，需要分别理解。两个独立请求修改同一客户资料，仍可能发生旧资料覆盖新资料；应用层需按业务要求提供版本检查或冲突处理。同一事务内更新顺序可交换，并不取消不同事务之间的并发保护。

各类服务因此形成清楚的安排：查询取得事实，计算准备结果，校验控制流程，量子服务完成真实增减，普通更新并发保存完整事实。场景组织业务依赖，执行机制落实相应的调度与事务保障。

这种职责划分使并发执行成为面向角色的服务架构的天然优势。服务之间的业务依赖由场景显式表达，普通查询与更新由系统内置提供，量子交易具有交换性；执行机制因而可以识别独立查询、集中量子请求和并发保存，减少人为串行等待。业务职责可以划分得细，而不必为每项职责都增加一次远程调用。传统微服务为可扩展性和故障隔离细化服务、采用分布式部署，也会增加远程调用与串行等待。第二篇进一步说明 SPP 如何利用原子服务并发，减少服务颗粒度变小带来的响应时间增长。

本篇目录 · 全书目录 · 上一章 · 下一章

第8章 通过角色多态复用服务与场景

© 2026 S++ project contributors. All rights reserved. Book copyright.

个人购买商品与企业采购商品，参与的实体不同，却都可能需要查询商品、计算报价、校验购买资格和下单。如果这些服务围绕具体实体类型编写，新增一种购买主体，就容易再写一套相似的业务逻辑。

面向角色的服务架构让服务围绕业务角色展开。只要不同实体能够扮演好所需角色，已经开发的计算服务、校验服务和场景就可以复用。变化主要发生在实体怎样进入角色、怎样提供角色事实，已有业务流程继续使用同一套角色与规则。

角色多态让参与者可以替换

熟悉面向对象的开发者，通常通过共同接口接纳不同对象。角色多态也让不同实体进入同一项业务，但它围绕的是“谁以什么身份参与这个行为”。个人与企业可以扮演买家，同一个个人又可以在另一笔业务中扮演卖家。

以一个允许个人和企业购买的商店为例，购买场景需要 Buyer 角色提供身份、收货信息和购买资格。个人的姓名与企业的名称怎样映射，收货信息怎样取得，由各自的扮演关系和查询补全机制承接。购买场景使用已经准备好的 Buyer，不需要分别写“个人购买场景”和“企业购买场景”。

“扮演好”意味着角色需要的事实和业务含义都已经满足。它包括参与身份、属性映射、角色自有事实，以及相应规则所需的输入。把企业名称放进买家名称字段，只完成了其中一项映射；其他购买事实也准备完整，企业才能作为 Buyer 参与已有场景。

计算服务按角色事实复用

明细计价需要成交单价和数量，计算规则为单价乘数量。扮演商品的实体可以是宠物食品、玩具或日用品，只要能提供相同含义的商品事实，已有明细计价就可以继续使用。

例如幼猫粮单价60元、买2件，明细金额为120元；玩具单价30元、买3件，明细金额为90元。参与的商品实体不同，计价服务的公式相同。服务依赖的是角色中的单价和数量，无需为每种商品重新实现乘法和舍入规则。

Petstore 的 `priceLine` 按 OrderLine 的成交单价和数量计算金额，`quoteOrder` 再复用它汇总订单报价。这种复用既发生在计算之间，也发生在不同实体进入相同业务角色之后。增加一种能够扮演 Goods 的产品，已有的明细计算与报价规则便可以继续使用。

买家也一样。折扣计算使用 Buyer 的会员等级与订单金额。实体只要通过角色提供相应事实，服务便可以按同一规则计算优惠，而不必判断买家的自然实体类型。

校验服务按角色条件复用

校验表达的是业务资格。例如购买数量必须大于零、订单必须处于未付款状态、付款方必须是该订单的买家。这些条件围绕角色、合约与办理状态定义。

个人购买2件幼猫粮，企业采购20件幼猫粮，都可以使用同一个数量校验。校验读取购买数量，不需要知道这一数字由个人还是企业填写。订单付款资格也可以围绕相同的 Buyer 与订单关系判断。

因此，新增参与实体时，可以继续使用已经开发好的校验服务。场景仍在相同位置调用校验，并按照通过或拒绝的结果控制流程。角色多态让规则复用，场景组织让规则的作用位置也保持一致。

场景可以整体复用

计算与校验可以复用，组织它们的场景也可以复用。一个购买场景说明的是买家与商品怎样共同完成购买：

```
确认买家与商品
        ↓
校验购买条件
        ↓
计算明细与订单金额
        ↓
完成量子交易
        ↓
按需要补充计算与校验
        ↓
保存完整业务事实
```

个人、企业提供不同的实体事实；食品、玩具提供不同的产品事实。它们分别扮演好 Buyer 和 Goods 后，同一个场景就可以使用已有查询、计算、校验和成交职责完成购买。复用覆盖完整业务行为，而不只是某一个方法或公式。

例如商店已经为个人购买实现了报价与下单。增加企业采购时，只要该业务沿用相同的报价、购买条件和成交约定，主要工作就是让企业进入 Buyer 角色并补全所需事实。场景中的数量校验、明细计算、订单汇总及保存安排都可以继续使用。

角色因此成为业务复用的连接点：实体满足角色，服务使用角色，场景组织服务。新增实体通过已有角色进入业务，就能接入已经开发的服务与流程。

把参与者变化与规则变化分清

复用时先看变化发生在哪里。由个人改为企业购买，如果购买规则相同，已有角色服务和场景可以继续使用。如果企业采购新增审批、赊账或专属计价，变化已经涉及业务规则，就应显式补充相应角色、校验、计算或场景步骤。原有且仍适用的服务继续复用。

这使业务扩展有清楚的起点：先寻找已经存在的角色、计算、校验和场景，再说明新实体怎样扮演角色，最后补充确实新增的业务约定。通过角色多态，已有业务能力可以不断接纳新的参与者，而不必随着实体种类增加反复编写相同逻辑。

角色多态让细颗粒度服务和场景能够复用。下一篇将讨论服务细化与分布式部署带来的调用成本，以及怎样通过原子服务并发和请求合并减少场景响应时间。

本篇目录 · 全书目录 · 上一章 · 下一章

第二篇 服务协作与执行

© 2026 S++ project contributors. All rights reserved. Book copyright.

把下单流程中的等待逐项解释清楚，再判断哪些查询和计算能够重叠执行。三章分别讨论服务协作、调用成本和部署边界；形式化证明与性能推导放在附录 B、C。

本篇目录

- 第9章 服务协作与执行顺序
- 第10章 并行执行与服务调用成本
- 第11章 服务最小颗粒度与部署原则

返回全书目录

第9章 服务协作与执行顺序

© 2026 S++ project contributors. All rights reserved. Book copyright.

下单时，商店需要取得买家资料与商品价格，再计算成交金额。读者很容易把这句话写成三个连续步骤：查买家、查商品、算价格。但其中只有“算价格要等待查询完成”是计算所需的顺序。两项查询可以同时开始。

组织服务时，先说明每一步为什么等待，比直接选择线程或网络协议更有用。

三种等待

第一种来自业务约定。商店规定核实退回的货物之后才能退款，核实结果就是退款资格。改变这条顺序，需要改变退款政策。

第二种来自数据。订单总额尚未算出，付款就没有确定金额。若计算折扣要用订单小计，折扣也必须等待小计。

第三种来自代码排列。买家查询使用会员凭证，商品查询使用商品凭证，二者不需要对方的结果。写成先后调用只是选择了一种执行方式。

可以并行的是第三类，以及其他已经准备好输入、彼此不干扰的活动。前两类需要保留依赖，或先改变业务与计算设计。

沿一个例子判断并行

假设买家查询补全会员等级，商品查询补全售价。两项查询完成后，报价才能根据等级和售价计算优惠。它们同时执行不会漏掉报价需要的事实。

再看两项计算：A 计算每条明细金额，B 根据这些金额计算订单小计。B 要读取 A 的结果，不能同时启动并使用尚未完成的明细。若有多条明细，可以分别算好各自金额，再汇总，但不能让多个线程随意覆盖同一个小计。

查询也会写入内存。两个查询都向同一个 Buyer 实例写姓名，结果取决于谁最后回填，因而还要检查它们实际补全哪些字段。

看清汇合点

下面是简化下单的阅读图，箭头表示等待关系：



汇合意味着两项查询都已完成且结果可用。一项查询没有找到身份，或发生技术故障，后面的报价就不能按正常路径继续。并行安排还要说明失败时哪些分支可以停止、哪些已经产生影响。

更新需要另外检查

两个请求都查到钱包有 100 元，各想扣 80 元。它们的准备工作可以重叠，实际成交却必须检查余额下限，最多允许一笔成功。事前查询和线程调度都不能代替成交保护。

同样，钱包扣款与商家入账需要共同形成付款结果。把它们同时发出，还要有能够保证双方结果的提交机制。

普通更新有另一套清楚的安排。遵从第一篇第7章的服务规范，普通更新所需信息由场景中的查询、计算和量子服务补全。同一事务边界内，量子服务之后的普通更新可以并发保存这些完整事实，再汇合完成事务。保存订单与保存凭证无需为了互相取得业务信息而串行等待。

本章给出阅读流程的判断方法：找输入，找输出，说明等待的原因，再检查共享写入和成交。完整的交换与并发条件、证明和反例见附录 B。

本篇目录 · 全书目录 · 上一章 · 下一章

第10章 并行执行与服务调用成本

© 2026 S++ project contributors. All rights reserved. Book copyright.

传统微服务为了提高系统的可扩展性、隔离故障，并支持各部分独立部署和扩容，倾向于把服务划分得更细，再分布到不同进程或节点运行。这让每个服务的职责更集中，却也使完整业务场景需要经过更多远程调用。

服务颗粒度越小，单项业务处理可能越短，通信、编解码、排队和协调的占比却越高。如果这些调用串行执行，等待时间会逐项累积，场景的响应时间便可能大幅增加。同时，完整业务依赖的远程服务越多，某个必经服务故障导致整项业务无法完成的机会也越多，场景层面的可用性随之承压。

SPP 通过原子服务之间的并发执行，有效减少颗粒度变小带来的响应时间增长。它保留细颗粒度的业务表达，同时把没有业务依赖的等待重叠起来。本章重点讨论响应时间；并发并未改善上述远程依赖带来的可用性问题。

细颗粒度怎样拉长响应时间

下单要查询买家资料、查询商品资料，再计算报价。假设买家查询耗时40毫秒，商品查询耗时70毫秒，计算耗时10毫秒；顺序执行需要120毫秒。这些是假设数字，用于说明调用结构。

其中只有报价计算需要等待两项查询。买家查询不需要商品查询的结果，商品查询也不需要买家查询的结果。把它们写成串行步骤，就把原本可以重叠的等待加在了一起。

若每项服务还分别承担请求编码、网络往返和服务排队，拆分后的串行等待会进一步增长。业务计算本身可能很短，完整场景的大部分时间却花在一次次调用和等待上。

原子服务并发怎样缩短等待

SPP 用场景显式表达参与者和服务依赖。两项查询输入已经具备，彼此不依赖，就可以同时执行；两项都完成后，再计算报价：

```
串行: 查买家40 — 查商品70 — 计算10 = 120毫秒

并发: 查买家40 ┐
              ├— 计算10 ≈ 80毫秒＋调度与汇合开销
      查商品70 ┘
```

两项查询的工作量没有减少，等待却发生了重叠。场景等待较慢的70毫秒查询完成，再用10毫秒计算报价。它缓解的是细颗粒度服务串行调用造成的响应时间累积。

这种并发能力来自前面介绍的服务特点：普通查询补全事实，计算和校验在场景中单独执行，量子交易具有交换性；普通更新在所需事实准备完整后可以并发保存。依赖关系清楚，执行机制就能保留必要等待，并重叠独立动作。

减少串行等待和重复通信

查询并发重叠远程等待，量子服务的交换性支持乱序执行和请求合并，完整输入下的普通更新可以并发保存。计算与校验通常在本地按依赖顺序完成，无需为每条公式、每个条件增加一次远程调用。

因此，业务职责可以细分，完整场景却不必随着服务数量增加而逐项等待。并发减少串行等待，合并请求减少重复通信，本地计算与校验减少细小服务的调用开销。这些安排共同缩短业务执行路径。

保留必要依赖并核对实际收益

并发不会消除真实业务依赖。订单小计需要明细金额，就必须等明细计算完成；量子成交后若还要计算和校验凭证内容，开发者应使用串行组明确安排，随后再保存。

并发也需要相应资源。若数据库只能同时处理一个请求，两项查询仍可能在内部排队；若连接或线程不足，增加并发也可能增加争用。比较时应保持业务规则和提交保障相同，同时测量响应时间、吞吐与资源占用。

面向角色的服务架构天然支持按职责和依赖组织并发，使系统能够在细颗粒度建模的同时减少响应时间增长。详细耗时模型、资源条件和实验方法见附录 C；服务颗粒度与部署颗粒度的关系在下一章讨论。

本篇目录 · 全书目录 · 上一章 · 下一章

第11章 服务最小颗粒度与部署原则

© 2026 S++ project contributors. All rights reserved. Book copyright.

讨论服务应该拆多细，需要先明确拆分的下限。SPP 的最小服务颗粒度有明确的业务含义：量子服务是一对量子属性组成的守恒服务，其他服务的最小颗粒度是针对一个角色的操作。在这个业务模型中，不再继续拆成更小的服务。

这一边界决定业务职责怎样划分。部署则决定这些职责在哪里运行、怎样使用资源。明确最小服务颗粒度之后，还需要按依赖、通信成本和扩容需要选择部署方式。

量子服务以一对属性的守恒变化为最小单位

甲向乙转100元，甲的余额减少100元，乙的余额增加100元。这一对量子属性及其相互对应的变化，组成一个量子服务：

```
甲.balance: -100元
乙.balance: +100元
变化之和: 0
```

只保留扣款或只保留入账，就破坏了资金转移的完整意义。因此，最小量子服务是这一对属性之间的守恒变化，不能把其中某个属性的增减独立拆成更小的量子服务。

库存预占也是如此：同一商品的可售数量减少2件，预留数量增加2件。两个属性可以属于同一个角色，也可以分属不同参与者，关键是它们在同一业务维度上形成真实的来源与去向。

多个量子服务可以组合，也可以合并成一次较大的量子请求提交。请求包含更多属性，不改变量子服务的最小定义；其中每项基本变化仍是一对量子属性组成的守恒服务。

其他服务以角色操作为最小单位

普通查询补全一个角色的事实，普通更新保存一个角色的事实；计算与校验则围绕角色完成明确的计算或判断。例如查询 Buyer、保存 Buyer、计算 OrderLine 的金额，都是以角色为业务单位的操作。

角色可能有多个属性，一项操作也可能只使用其中几个属性。这不改变它的颗粒度。例如校验买家资格只读取买家状态，服务仍是在判断 Buyer 能否参与业务；明细计价只使用单价与数量，服务仍是在计算 OrderLine 的金额。

复杂计算可以复用已有计算，完整场景可以组织多个角色操作。但继续把角色中的字段读取、一次乘法或一个条件比较拆成独立服务，就超出了 SPP 定义的最小业务服务颗粒度。它们是角色操作内部的表达。

与面向对象中的属性级操作比较

在面向对象的代码中，可以为姓名提供 `getName` 和 `setName`，为地址提供另一组访问方法。一个属性的读取或赋值，就可以成为一个独立方法；从代码操作看，颗粒度可以细到单个属性。

SPP 用角色操作定义业务服务。查询买家补全 Buyer 所需事实，保存买家落实已经准备好的 Buyer 事实。姓名和地址仍然是属性，但各自的访问方法不再被当成更小的业务服务。普通查询与更新由系统内置提供，开发者在场景中安排调用，计算与校验负责表达相应业务规则。

因此，SPP 的服务细化有明确终点：一对量子属性的守恒服务，或者针对一个角色的操作。服务划分依据业务单位，而不是不断缩小方法内部处理的数据。

按使用范围查询角色属性

以角色为最小操作单位，不意味着每次查询都要递归取得整个扮演链条中的所有属性。角色服务的业务颗粒度与实际读取的数据量，可以分别安排。

编译时可以收集该角色在场景、计算、校验等业务表达中被使用的属性，形成查询所需的属性集合。生成的默认查询方法只补全这个集合需要的事实，沿扮演关系取得相应来源，不必把每一层扮演者的全部资料都读下来。

例如 Buyer 由客户实体扮演，报价只使用会员等级，购买校验使用客户状态，下单使用收货地址。编译时收集这些用途后，默认查询可以取得这些所需属性及必要身份信息，而不读取没有被业务使用的其他客户资料。查询仍然针对 Buyer，角色的最小服务颗粒度没有改变，数据库读取、远程传输与映射的数据量却可以减少。

用多级缓存减少重复读取

角色查询还可以使用请求内缓存、进程内共享缓存等多级本地缓存。一次场景多次使用同一实体的事实，可以复用已读取的数据；不同请求反复读取相对稳定的事实，也可以按相应策略共享缓存。

对于来自远程业务域的实体，本地缓存尤其有价值。例如商品名称和分类不需要每次使用时都重新访问产品域，可以缓存远程实体的所需事实，再按扮演关系补全本地角色。缓存命中时，减少的是重复的远程访问和等待。

这些都是业务系统常用的优化手段，可以由内置查询结合 APP/PHY 配置承接，开发者仍在场景中调用角色查询。缓存应按业务要求刷新或失效；资金和库存的实际成交仍须使用量子执行机制认可的事实。

属性按需查询减少单次读取的数据量，多级缓存减少重复读取的次数。二者都能缓解角色包含较多属性、扮演链较长带来的性能成本，无需把角色服务拆成一个个属性级服务。分析与缓存的具体条件见附录 C 的 C13 节。

计算和校验优先本地部署

明细计价、折扣计算和资格校验可以保留各自的服务定义，并在场景所在进程执行。参与者事实已经准备好时，本地计算与校验直接使用这些事实，避免为短小公式和条件增加网络往返。

角色操作和量子服务可以独立定义、复用与组合，一个部署单元却可以承载多个服务。业务模型保持最小职责，运行结构按实际成本组合它们。

查询、量子交易和保存按依赖安排

普通查询根据数据所在位置选择本地或远程执行。需要同时取得的独立角色事实，可以并发查询。若产品目录需要单独扩容，或者客户域由另一团队运营，可以通过相应域的远程服务取得事实。

同一事务边界内，编译器把量子交易提前到普通更新之前，量子服务可以乱序执行或合并请求。量子结果补全后，场景按需要用串行组安排补充计算与校验，再并发执行输入完整的普通更新。部署应支持这些业务依赖，并控制通信与协调成本。

业务颗粒度与部署颗粒度不必一一对应。按角色划分的客户、产品和商店职责，可以部署在同一个进程中，也可以根据容量和维护责任分开部署。“共置”就是把多个服务放在同一个进程里运行，例如客户、产品和商店服务都在同一个 Java 程序中运行，可以直接本地调用。职责归属继续保持，变化的是实际调用路径与资源。

部署必须兑现事务边界

Petstore develop 把四个域的服务放在同一个进程中运行，并使用 LOCAL；testing 拆成客户、产品和商店三个应用。两种部署使用相同业务模型，却有不同的通信与提交条件。

多个服务在同一个进程中运行并使用共同资源时，可以在同一本地事务中处理量子变化与普通更新。拆成不同进程后，需要实际支持的协调机制，或明确的分阶段成交与恢复安排。只把部署名称改成“分布式”，不能自动获得共同提交能力。

先按明确的最小颗粒度建模，再按业务依赖、通信成本、扩容需要和事务能力部署，才能既保留细致的角色服务，又减少完整场景的响应时间成本。

本篇目录 · 全书目录 · 上一章 · 下一章

第三篇 用 S++ 表达角色与服务

© 2026 S++ project contributors. All rights reserved. Book copyright.

通过 SPP、APP 和 PHY 分离业务与技术、逻辑与物理，明确需求人员、技术人员、运维人员以及 AI 的工作范围。再将角色与服务模型写成相应源码。沿商品、购物车和充值例子阅读扮演、原子服务与场景，理解业务声明怎样成为可检查的源码。

本篇目录

- 第12章 SPP APP PHY 的分层职责
- 第13章 Entity Reference Role Contract
- 第14章 用 take … as 声明扮演关系
- 第15章 四类原子服务
- 第16章 业务计算规则与集合参与者
- 第17章 场景中的串行并行与合约形成

返回全书目录

第12章 SPP APP PHY 的分层职责

© 2026 S++ project contributors. All rights reserved. Book copyright.

S++ 分层的目的，是做到业务与技术分离、逻辑与物理分离。前一种分离让业务开发与技术适配由不同技能的人员完成；后一种分离让技术开发与部署运维各自承担明确责任。

例如一笔付款，业务人员关心谁向谁付款、付多少、什么时候允许付款、哪些结果必须共同成立。技术人员关心业务模型怎样接入存储与外部系统，运维人员关心实际使用哪个数据库、服务运行在哪里、资源怎样配置。把这些内容写进同一段程序，会让不同人员都不得不理解并修改一份混合逻辑。

SPP、APP、PHY 分别承接业务表达、应用技术约定和实际物理配置，使三种工作能够围绕明确的约定协作。

业务与技术分离，让懂业务的人表达业务

SPP 说明参与者、角色、合约、计算、校验和场景。付款金额、会员折扣、订单能否发货，以及资金变化的守恒关系，都在这一层表达。业务还要明确哪些变化应处在同一个事务边界内。

业务开发不需要同时编写数据库查询、接口报文转换、网络调用或事务框架代码。普通查询与普通更新由系统内置提供，计算和校验使用业务表达，场景组织完整行为。业务人员可以沿着角色与流程检查需求，而不用先理解大量技术实现。

例如“会员享九折优惠”，需求人员可以直接检查适用的会员角色、折扣公式和报价流程。Buyer 与客户实体的业务扮演关系在 SPP 中表达；至于客户域怎样接入现有客户系统、用什么协议读取资料，是技术适配的工作。

这种分离让业务开发与技术适配可以由不同技能的人员承担。业务开发甚至可以直接交给熟悉需求的人员：掌握 SPP 的角色、服务与场景表达后，就可以把业务规则写成可编译的模型。技术人员则专注于让这些业务表达接入实际系统。

角色和合约也成为双方的共同接口。需求人员说明付款方、收款方与金额的含义，技术人员据此提供相应事实和执行能力。沟通围绕明确业务约定展开。

逻辑与物理分离，让技术开发与运维分工

技术逻辑说明应用需要什么能力，物理配置说明这些能力在某个环境中怎样提供。APP 组织仓储映射、服务绑定、接口、防重、编号分配、逻辑资源和环境部署关系；PHY 将相关能力落实到实际数据库、协议、数值表示与运行资源。

例如应用声明需要访问客户域、保存订单，并使用相应事务资源。这些是技术开发要组织的应用约定。开发、测试和生产环境分别使用哪些服务地址、数据库连接和资源参数，则由物理配置承接。

技术人员因此可以开发与适配应用能力，运维人员根据环境配置和管理实际资源。把测试环境的客户服务地址换成生产地址，不需要重新编写付款场景；迁移数据库，也不应要求改写会员折扣公式。

某些物理能力需要技术人员先提供适配器，例如一种新的数值表示或事务协议。适配器准备好之后，运维再按环境选择和配置它。逻辑与物理分离让技术能力的开发和资源的运行管理各有明确位置。

三层怎样共同完成一笔付款

层次	需要表达的内容	主要负责人员
SPP	买家、商家、付款约定、资格、金额与事务边界	业务开发者或掌握业务表达的需求人员
APP	模型与存储的映射、服务绑定、接口及应用技术约定	技术开发与适配人员
PHY	实际数据库、端点、资源参数及物理表示选择	运维人员，并由技术人员提供所需适配能力

SPP 表达买家减少100元、商家增加100元，以及订单和凭证怎样形成付款结果。APP 组织这些事实的存储和服务接入，选择能兑现业务边界的执行策略。PHY 提供实际资源，使相关配置能够运行。

订单凭证号有直接业务用途，可以属于 SPP；只为定位存储记录而使用的内部主键由编译器和系统提供，不需要业务模型再定义一个 ID。付款金额怎样舍入是业务规则，金额采用怎样的机器表示和存储容量则由物理实现承接。区分这些内容，才能让不同人员维护自己的职责而保持业务含义一致。

AI 负责较少的源码，工具负责大量确定性输出

在传统开发中，AI 为一项付款需求生成的代码，可能同时包含业务判断、数据对象、字段转换、仓储访问、远程调用、事务控制和部署配置。代码量越大，彼此之间需要保持一致的关系就越多；需求变化后，AI 还要在这份混合逻辑中识别哪些位置应当同步修改。

S++ 为 AI 提供了更明确的表达工具。业务变化集中在 SPP，技术接入变化集中在 APP，实际环境变化集中在 PHY。AI 可以围绕当前任务修改对应的源码，而无需每次重新生成和维护所有层次的实现细节。

大量重复代码由编译器、生成器和运行时按照既定规则提供。例如角色模型、属性映射、默认查询与保存的接入、服务调用、接口与存储结构，都可以从相应声明和配置生成。相同源码、配置与工具版本，按相同规则产生相应代码。

于是，需要 AI 自由生成和维护的代码量可以大幅下降。原本由 AI 反复编写、再人工核对的一致性关系，转为由工具按规则生成和检查。AI 需要同时作出的选择更少，引入遗漏、重复实现和跨层不一致的机会也随之减少。

例如调整会员折扣时，AI 修改计算规则；模型、接口和查询保存框架继续由工具维护。接入另一种通信协议时，AI 或技术人员处理适配约定，付款业务继续使用原有角色与场景。代码生成承担可确定的转换，AI 更集中于需求理解、业务建模和必要的适配。

工具仍按输入表达执行。付款规则是否符合需求，适配能力能否兑现约定，需要各自负责人员审查与验证。分层把这些问题变得可定位，也减少了 AI 必须维护业务、技术、部署和运维混合逻辑的负担。

人类专注业务，更容易把控 AI 项目的质量

SPP 层只有业务模型、规则和流程。人类审查 AI 的业务输出时，可以直接核对谁参与、以什么角色参与、怎样计算、满足什么条件才能继续，以及成交和失败时应该形成什么结果。技术接入和物理部署另有明确位置，业务审查不需要同时追踪背后的实现细节。

例如 AI 修改会员优惠，需求人员可以沿着 Buyer 的会员等级、折扣公式和报价场景，检查优惠适用于谁、比例是否正确、运费是否按约定计算。审查对象是可读的业务表达，能够与需求逐项对应。

这让人类更专注于业务本身，也使人与 AI 的配合更高效。人提出规则、解释例外、确认业务结果，AI 将这些要求表达为角色、服务和场景；编译器与生成器再按既定规则输出大量实现代码。需求人员可以直接发现业务遗漏，而不用等待技术人员从混合代码中还原业务含义。

项目质量也更容易把控。每次业务变更需要重点审查的是相应 SPP 表达与业务结果；技术适配和运维配置分别由具有对应技能的人员核对。大量确定性生成的重复代码由工具链的规则、检查和验证保障，减少了过去对每次输出都事无巨细地逐行 review 的负担。工具链或适配能力发生变化时，再集中检查相应实现与生成行为。

代码量减少的价值由此进一步体现为审查范围的收敛：人能够看清 AI 改了哪项业务、为什么这样改、结果是否符合需求。业务表达、确定性生成和实际业务验证共同形成质量控制的依据。

用变化判断应该修改哪一层

会员折扣变化，修改 SPP 中的业务计算；接入新的客户系统，处理 APP 中的服务与事实适配；生产服务地址或数据库连接变化，调整 PHY 中的实际资源配置。

这种分工让需求人员、技术人员、运维人员以及 AI 都有清楚的工作范围。各层通过共同约定连接，业务规则保持可读，技术适配保持明确，实际部署有可核对的资源配置。接下来的章节将逐步介绍这些表达怎样写成源码。

第五篇第23至27章进一步讨论这种分离的收益，以及性能、安全、可靠执行和运行治理方面的技术服务。

本篇目录 · 全书目录 · 上一章 · 下一章

第13章 Entity Reference Role Contract

> © 2026 S++ project contributors. All rights reserved. Book copyright.

业务模型中的类型需要回答不同问题。Entity 表示当前域拥有的自然实体，Reference 表示对外域概念的引用，Role 表示参与身份，Contract 表示场景形成的约定。它们帮助我们同时理解业务含义与生命周期归属。

自然实体与外域引用

客户域拥有 Person，产品域拥有 Product。商店使用客户与产品事实，相关生命周期仍归原域维护。Reference 明确引用来源，接入方式由应用和物理配置决定。

Reference 可以引用提供方的实体、角色或合约。例如商店引用产品域的 Product，付款引用库存域或其他域提供的业务概念。引用说明事实来自哪里，维护生命周期仍由提供方负责。

角色与场景合约

Buyer 表达买家身份，Goods 表达商品的销售身份。ShopOrder 表达下单约定，PaymentAgreement 表达付款约定。订单可以被后续付款、发货和退货场景引用，但引用订单不等于重新执行下单。

下面摘录 Petstore 的一个简短合约：

```
Contract WalletFundingAgreement {
    Buyer buyer;
    SettlementAccount source;
    BigDecimal(19, 2) amount;
}
```

参与者与金额共同说明一次钱包充值。合约记录约定，资金增减通过相应服务完成。

组成事实与独立参与者

订单明细保留商品、数量与成交单价。订单总额属于订单约定中的组成事实，未必需要为它创建一个独立金额角色。是否形成独立角色，要看是否具有真实参与关系与生命周期。

合约也不能仅因为聚合明细，就被解释为反过来扮演每条明细。扮演关系和组成关系需要分别表达。

内部主键与有业务含义的 ID

传统开发者习惯先给实体加一个唯一的 `id`，再用它标识对象和数据库记录。在 S++ 中，不必为了技术定位给业务实体添加这样的字段：编译器会为实体建立业务不可见的内部主键，系统据此维护存储身份。技术上需要唯一定位，不意味着业务模型也必须出现一个 ID。

习惯性添加的 `id`、`customerId` 等字段，容易让读者误以为它们是业务需要的凭证。编译器会检查这类疑似习惯性 ID，并给出警告，提示开发者确认它是否有真实业务含义。

如果这个 ID 确实来自业务需求，可以用 `business` 声明其用途，告诉编译器它是业务事实，从而消除相应警告。例如一个回执号供客户查询办理结果：

```
Entity Receipt {
    business("业务办理回执号，供客户查询和核对办理结果") String receiptId;
}
```

这里的 `receiptId` 有客户实际使用的含义。若只是为了给数据库记录分配一个唯一值，系统内部主键已经承接这项责任，无需把它加入 SPP。`business` 用来说明为什么这个看起来像技术 ID 的字段应当留在业务模型中。

所以，`business` 的重点是消除习惯性 ID 带来的歧义。模型评审仍需核对说明是否符合真实需求：谁使用这个编号，在哪项业务中使用，如何凭它办理或核对结果。“内部唯一标识”只说明技术用途，不能据此把技术字段解释成业务凭证。

模型已按会员、商品、购物车、订单等业务功能拆分，完整阅读入口见 Petstore 源码导读。

本篇目录 · 全书目录 · 上一章 · 下一章

第14章 用 take ... as 声明扮演关系

© 2026 S++ project contributors. All rights reserved. Book copyright.

定义 Goods 只说明销售角色需要哪些事实，模型还要说明由谁扮演它。S++ 用 `take ... as` 声明这一关系，并表达属性映射、借用和角色自己的事实。

`take` 后面写扮演者，`as` 后面写所扮演的角色。例如 `take Product as Goods` 表示 Product 扮演 Goods。

单方扮演与映射

Petstore 将产品映射为商品：

```
take Product as Goods {
    mapsame;
```

```
    String sku = productCode;
}
```

`mapsame` 显式启用同名属性映射。例如 Product 与 Goods 都有 name，类型等条件满足时，商品名称可以从产品名称取得。名称不同的属性则显式映射：上例把 Product.productCode 映射为 Goods.sku。

`mapsame` 不是默认规则。若只写下面的映射，就只明确了商品编号的来源，同名的 name 不会因此自动加入映射：

```
take Product as Goods {
    String sku = productCode;
}
```

无映射块的 `take ... as` 简写也只声明扮演关系，不自动启用同名映射。`takeall` 是另一种明确的写法，用于让角色继承扮演者的属性，并为相应属性生成映射；它与仅声明扮演关系的简写不同。

映射还决定事实归属。直接借用的事实由原扮演者拥有，角色新增或明确重定义的事实由角色拥有。查询需要补全两类事实，保存则不能任意跨越所有权边界。

查询扮演链与属性回填

角色查询会沿扮演关系查询来源事实。例如查询 Goods，系统取得商品自己的事实，并查询相应 Product。取得产品之后，再按照已声明的映射把相应属性补入 Goods。

因此，“查询到了 Product.name”和“把 Product.name 用作 Goods.name”是两个步骤。前者取得来源数据，后者由映射决定。上例显式使用 mapsame 时可以补全同名名称；仅映射 sku 时，查询产品不会自动增加 name 的映射。这也解释了为什么递归查询扮演链，并不使 mapsame 成为可省略的默认行为。

联合扮演

购物车明细由商品与买家共同扮演：

```
take Goods, Buyer as CartLine {
    Goods goods = Goods;
    Buyer buyer = Buyer;
    int quantity;
}
```

第一行声明两个扮演者：Goods 和 Buyer 共同扮演 CartLine。映射块中的两行分别把这两个参与者作为整体放到购物车明细中。以 `Goods goods = Goods;` 为例：

- 左边的 `Goods` 是属性类型，说明这个属性表示一个商品角色。
- 左边的 `goods` 是 CartLine 中的属性名。
- 右边的 `Goods` 指向第一行声明的商品扮演者，表示把这个参与者整体映射给 `goods`。

`Buyer buyer = Buyer;` 同理。右边写的是扮演者名称，不是在创建一个新买家，也不是读取买家的某个同名属性。生成的映射把参与者对象交给相应属性；若角色尚未声明该属性，编译器会补上类型相符的属性。

因此，购物车明细的业务结构可以这样理解：

```
CartLine（购物车明细）
├─ goods: Goods（本条明细的商品）
├─ buyer: Buyer（选购该商品的买家）
└─ quantity（本条明细的选购数量）
```

这里的 quantity 是 CartLine 自己的事实，需要在角色定义中声明。使用商品名称时访问 `cartLine.goods.name`，使用买家名称时访问 `cartLine.buyer.name`。即使商品和买家都有 name，也能明确区分。上述结构表达业务上的参与关系，具体怎样存储仍由 APP 配置决定。

小林和小王都把同一商品加入购物车，分别选了 2 件和 3 件。两条明细的商品相同，买家不同，数量也不同。CartLine 因而要保留“哪位买家选了哪个商品”的关系；数量属于这段关系，APP 按双方身份定位各自明细。

整体参与者映射与 takeall 的区别

当前语言中，批量引入扮演者属性的关键字是 `takeall`，并没有 `mapall`。它与上面的整体参与者映射有不同的结构。

例如，Goods 有 sku、name 两个属性：

```
take Goods as DisplayGoods {
    takeall;
}
```

`takeall` 把 Goods 的属性引入 DisplayGoods；尚未声明的属性会被补入，并生成相应映射。因此可以直接访问 `displayGoods.sku`、`displayGoods.name`，无需先经过一个 goods 属性。

写法	角色中的结构	商品名称的访问方式
`Goods goods = Goods;`	有一个 Goods 类型的 goods 属性，保留完整商品参与者	`cartLine.goods.name`
`takeall;`	把扮演者的属性引入角色本身	`displayGoods.name`

购物车明细需要同时表达“商品”和“买家”，整体参与者映射使两者各有明确的位置。如果对 Goods、Buyer 一起使用 `takeall`，它也不会自动产生 goods、buyer 两个分组；双方的同名属性还需要明确区分来源，不能仅靠批量引入表达“商品名称”和“买家名称”。

`mapsame` 又有所不同：它为角色已有的兼容同名属性生成映射；`takeall` 还会把扮演者中角色尚未声明的属性引入角色。选择写法时，先确定业务需要的是完整参与者，还是参与者的某些属性，再声明对应的映射。

角色自己的事实

角色可以借用扮演者的资料，也可以拥有这个业务身份需要的事实。例如，小林扮演商家时，商家借用小林的个人资料，同时拥有店铺名称“林家宠物店”与经营资金。小林修改个人资料，商家查询按映射取得相应资料；修改店铺名称，则是在维护商家自己的事实，不是在修改小林的个人姓名。

因此，声明扮演关系时需要分清：哪些属性来自扮演者，哪些属性属于角色自己的业务身份。合约保留成交时的信息是另一项要求，见第四篇第18章的说明。

扮演关系的合法性与持久化

`take ... as` 是需要编译器检查的业务声明。编译器检查扮演者是否为已定义的合法参与者类型、`as` 后面的目标是否为角色，以及属性类型是否符合扮演关系的要求。当前实现的严格业务模型检查允许 Entity、Reference 或 Role 扮演角色，拒绝把 Contract 当作扮演者；这一检查由 `strictRoleOnlyBusinessModel` 配置启用。

合约仍可以作为服务或场景的上下文传入，供计算、校验和后续处理使用，不必为了读取合约就声明扮演关系。例如，退款场景可以读取原订单合约，同时由买家、商家等角色参与退款。合约包含订单明细，也不等于合约扮演明细：包含关系与扮演关系需要分别表达。

例如，扮演者的 name 是字符串，角色中的同名属性也必须保持相同类型，不能改成数值类型；整体参与者映射也必须指向声明中的扮演者，并与目标属性的类型一致。这样的错误应在编译时发现，不应等到查询或保存时才暴露。

声明合法之后，编译器才能据此生成类型、映射与查询元数据。但声明一个扮演关系，并不等于创建一个实际参与者。例如，声明 Person 可以扮演 Merchant，不会自动为小林建立商家记录；何时开店、何时保存商家自己的资料，仍由场景安排。

持久化也需要遵守事实归属。普通角色更新自己的事实，客户或产品的资料则通过相应的内置服务维护。编译时确认谁可以扮演谁，运行时根据具体参与者完成查询与保存，两者共同支撑扮演关系。

本篇目录 · 全书目录 · 上一章 · 下一章

第15章 四类原子服务

© 2026 S++ project contributors. All rights reserved. Book copyright.

S++ 将普通业务原子服务分为 query、check、calculate 和 update。分类表达的是服务职责：获得事实、判断条件、形成候选结果，以及实施变化。完整业务场景通过组合这些职责完成。

查询补全事实

普通 query 是内置系统服务，开发者不编写或修改其实现。场景直接调用 `findBuyer` 等服务，系统按 APP 和 PHY 配置取得事实并补全入口参与者，不需要重复声明标准查询服务。服务没有业务返回值，不写成 `buyer = findBuyer()`。查询是否确认了身份也是执行状态的一部分，不能只根据字段非空推断。补全角色所需事实不要求读取整个扮演链的全部属性；可以通过编译期属性使用分析和缓存优化内置查询，具体方法见第二篇第11章。

入口可以省略实例名，默认采用类型名；同类型多个参与者需要显式别名。调用也可以省略参数，但必须按完整类型和集合维度唯一匹配。歧义时应显式选择。

校验判断资格

check 读取事实，使用 require 表达业务要求。它不修改参与者。普通 check 失败结束对应词法范围，并不自动保证事务回滚；需要撤销时必须采用受支持的显式控制约定。

把校验放在场景可见的位置，有助于理解业务准入。成交范围则需要在实际量子执行中检查，不能用事前余额读取代替。

计算准备结果

calculate 按业务公式修改候选事实。它可以调用其他纯计算，不能调用查询、校验、更新或场景来隐藏技术与业务副作用。

例如单行金额计算：

```
atomic calculate priceLine(OrderLine) {
    OrderLine.amount = roundHalfUp(
        OrderLine.unitPrice * OrderLine.quantity, 2);
}
```

这是 定价源码 中的定义，排版做了换行。

更新实施变化

普通 update 同样是内置系统服务，开发者不编写或修改其实现。场景准备完整事实后，直接调用 `insert`、`update`、`delete` 加参与者类型名的标准服务。保存服务接收当前参与者，技术基线由框架保留，不需要业务额外传入 before 快照。计算和校验在场景中单独调用，内置更新内部不执行这些业务逻辑。量子更新进一步描述特殊属性的增减与成交要求。

分类让依赖更容易检查。准备事实、判断资格与候选计算通常可以在成交边界前完成，事务内部聚焦受保护的更新。但具体流程仍需考虑业务与并发约束，不能机械地把四类服务各执行一次就当作完整业务。

本篇目录 · 全书目录 · 上一章 · 下一章

第16章 业务计算规则与集合参与者

© 2026 S++ project contributors. All rights reserved. Book copyright.

业务计算需要表达金额、数量、资格与政策。S++ 保留开发者熟悉的条件与循环写法，同时将运算含义与物理表示分开。阅读公式时，读者首先看到业务关系，再由 PHY 承接表示能力。

把舍入写进业务

明细金额采用单价乘数量，并按约定保留两位小数。`roundHalfUp` 表达明确的舍入规则。若业务没有要求舍入，物理层不能因为数据库容量或默认精度而静默改值。

BigDecimal 是一种可选物理表示，无法精确完成某些运算时仍需要明确的业务舍入或其他表示。使用精确十进制不等于所有数学结果都能有限表示。

规则按确定顺序执行

Petstore 报价先计算明细与小计，再根据会员等级计算折扣，之后判断运费，最后形成总额。这些步骤具有事实依赖，规则按执行到当前位置的参与者事实生效。

if、switch、while 与 for 提供确定的控制流程。它们不引入自动反复触发的规则议程，也不允许任意 Java 方法调用。业务公式需要在语言支持的表达和调用契约内完成。

集合与内部明细

参与者可以是一维集合，例如 Goods[]。场景查询补全集合，计算遍历商品与购物车明细，形成订单内部的新明细。标量入口实例不能被替换，集合入口保留容器并更新内容。

语法允许声明多维数组，例如 `Goods[][]`，但当前实现尚不支持多维参与者集合的查询、更新与计算操作。本书中的集合示例均采用一维集合；多维支持将在未来出现实际需求时再扩展。

下面是说明汇总职责的示意片段，省略域与其他定义：

```
atomic calculate sumOrderLines(ShopOrder) {
    ShopOrder.subtotal = 0;
    for (OrderLine line : ShopOrder.lines) {
        ShopOrder.subtotal += line.amount;
    }
}
```

这段循环按顺序执行。若要并行处理明细，应另外明确分支结果、汇总关系与执行实现。

候选事实与成交事实

计算可以把候选订单标记为已付款，但这一内存状态必须通过受保护的更新才能成为实际业务结果。技术失败时也不能把候选状态直接当成已成交状态对外发布。

把计算与更新分开，能让公式独立验证并缩小事务范围。与此同时，需要可靠的版本或成交机制，防止准备事实之后发生的竞争使旧结果覆盖新结果。

本篇目录 · 全书目录 · 上一章 · 下一章

第17章 场景中的串行并行与合约形成

© 2026 S++ project contributors. All rights reserved. Book copyright.

原子服务解释局部职责，Scenario 表达一次明确的业务活动。读者应当能够从场景看到参与者、资格、依赖和提交边界，而无需逐层追踪隐藏调用。

从充值场景阅读结构

以下片段摘自 钱包充值源码:

```
Scenario serial fundShopWallet(
    Buyer, SettlementAccount, WalletFundingAgreement) {
    checkBuyerCredential; checkFundingAmount; prepareWalletFunding;
    parallel { findBuyer; findSettlementAccount; }
    identify(Buyer); identify(SettlementAccount);
    transaction {
        transferWalletFunds bounds {
            SettlementAccount.balance: [null, null];
            Buyer.wallet: [0, null];
        };
    }
}
```

入口给出买家、来源账户和充值约定。首先检查凭证与金额，准备来源和合约关系，再并行补全双方事实。身份确认之后，资金转移在显式事务内执行，成交范围在调用处声明。

场景中的合约

WalletFundingAgreement 表达金额与参与方；ShopOrder 表达订单；ReturnRequestAgreement 表达退货申请。一个合约对应明确活动，不能只为了包装列表或返回结果随意创建合约类型。

后续活动可以引用已有合约。付款使用订单，退货申请使用原订单与申请理由。这样的引用保持业务关系，不需要再次调用原场景。

显式编排与自动绑定

无参数调用减少重复写入口名称，但没有删除参与者。编译器按类型寻找唯一绑定，同类型多实例时需要显式参数。自动绑定服务于可读性，不能用猜测掩盖歧义。

串行组表达顺序，parallel 组表达重叠执行。业务必须先确认输入和影响关系；事务计划还会检查部署是否支持所需操作与恢复能力。

形成可审查的业务入口

一个场景应当对应可命名的业务动作。把许多动作塞进通用命令字符串，会把资格和参与者范围隐藏在分支中。独立付款、发货和退货入口更容易说明各自需要什么事实。

场景使服务组合成为正文可读的业务叙述。下一篇将讨论这些声明如何通过身份、存储、事务与资源约定得到保障。

本篇目录 · 全书目录 · 上一章 · 下一章

第四篇 一致性与执行保障

© 2026 S++ project contributors. All rights reserved. Book copyright.

一笔付款不仅需要公式正确，还要定位正确参与者、保护竞争更新并解释失败。沿事实归属、事务、量子成交、恢复和生成链路，说明这些要求如何得到实际保障。

本篇目录

- 第18章 业务事实参与者身份与存储归属
- 第19章 事务边界执行计划与部署能力
- 第20章 量子属性调用范围与守恒成交
- 第21章 回滚反交易冲正与补偿
- 第22章 生成链路物理资源与运行配置

返回全书目录

第18章 业务事实参与者身份与存储归属

© 2026 S++ project contributors. All rights reserved. Book copyright.

一个买家角色可能同时包含客户姓名、会员等级和钱包余额。这些事实出现在同一个业务参与者中，却未必属于同一个存储所有者。角色查询需要把它们补全，角色保存则必须遵守各自归属。

借用与自有事实

客户姓名通过扮演关系借用，仍由客户域维护。会员等级属于买家角色，钱包余额由量子存储成交。角色普通 insert、update、delete 只写自己的表，不自动级联维护扮演者。

这种约定防止商品更新意外改写产品资料，也防止购物车保存隐式修改买家。需要维护原实体时，场景通过原域服务明确办理。

合约保留成交事实

合约需要记录成交时约定的内容。例如，顾客下单时商品叫“幼猫粮 1kg”，一个月后商家把商品改名为“幼猫配方粮 1kg”。商品页面应显示新名称，旧订单中的成交名称则仍应显示“幼猫粮 1kg”，便于顾客核对当时买了什么。

商品的当前名称与合约中的成交名称承担不同职责。查询旧订单时，不能用当前商品名称覆盖合约保存的成交名称。这类要求需要通过 APP 层的存储与映射配置落实，不能仅凭一般角色的扮演关系推导出来。

例如，某银行的失败处置记录需要保存当时的存入或冲正合约，其中包括系统给出的原因、服务、失败阶段和恢复状态。若只保存关联后查询当前事实，后来的余额或合约变化可能使这份记录失去原含义。

当前 APP 的 `snapshot` 可以将显式声明的关联参与者及集合保存为历史快照。查询这类快照时，不沿扮演或关联链继续补全当前事实，也不以当前量子余额覆盖快照中的历史余额；普通实时关联仍按原规则补全。两种用途由 APP 配置区分，详见某银行案例的快照验证。

身份不等于全部事实

买家的钱包和姓名会变化，业务身份仍可能保持。APP 根据主键、业务凭证或组合关系定位参与者；完整事实比较则用于另一类问题，例如确认候选内容是否变化。

编译器为实体建立业务不可见的内部主键，系统通过内部元数据维护相应存储身份，不需要业务开发者习惯性地添加 id 字段。订单号、会员凭证等真正供业务使用的编号，则按其实际用途进入业务模型。平台版本、防重键和仓储定位规则各自承担技术责任。

查询补全与基线

普通查询与普通更新均由系统内置提供，开发者不能修改其实现。以下补全、基线记录与版本比较由系统根据 APP/PHY 配置承接，场景负责安排调用和业务计算、校验。

查询除了补全当前事实，还可以记录后续保存需要的内部基线。采用版本保护时，更新应比较查询时获得的版本，而不能临时读取新版本再覆盖竞争者的结果。

候选计算在查询后完成，如果另一个请求已修改记录，保存应发现冲突。这样既能缩小持锁范围，也要求缺失基线或身份变化时明确拒绝写入。

Master-Detail

Master-Detail 表示主记录与明细记录的关系。例如，订单是 Master，每条订单明细是 Detail：订单记录整笔交易，明细记录各商品的数量、成交单价等具体内容。购物车与购物车明细也采用这样的结构。

样例保存明细成员身份，并通过查询元数据补全明细及相关参与者。保存 Master 不自动替代 Detail 的新增、修改与删除；这些操作仍需要在场景中明确安排。

下单场景显式删除原购物车明细、插入成交明细、保存订单和购物车。阅读这些操作，可以看出每项变化的归属，避免把整个聚合快照当成任意覆盖的存储单位。

零事实的生命周期

新角色建立零量子事实，与非零资金入账不同。例如，开户时建立余额为零的账户，只是在准备后续交易所需的存储；充值则需要执行守恒的量子交易。

角色与量子事实存放在同一个库时，建立过程由本地共同事务完成。分库时，当前内置运行时先把角色记录和技术恢复待办一起提交到本地库，确认提交后，再建立远端零记录。本地事务回滚，两者一起撤销；远端不可用或响应丢失，待办保留，供后续重试。重试只建立缺失的零记录，不覆盖已有成交值，也不产生量子交易流水。

这些步骤由内置服务承接，业务场景仍只调用角色插入服务，不需要自行拆分开户或编写业务补偿。远端零记录尚未建立时，开户还不能视为完整成功。当前机制支持内置 JDBC 普通仓储与 JDBC 量子库，详见 分库零记录技术恢复。

存储设计的目标是让每项事实都有明确所有者、定位方式和维护入口。完整约定见 事务与存储规范。

本篇目录 · 全书目录 · 上一章 · 下一章

第19章 事务边界执行计划与部署能力

© 2026 S++ project contributors. All rights reserved. Book copyright.

“这些变化必须共同成立”是业务要求，“使用什么资源和协议完成”是实现问题。SPP 用 transaction 表达边界，APP 选择策略，PHY 提供真实资源。生成的执行计划把三者连接起来。

普通更新默认使用乐观锁

SPP 的普通 update 默认使用乐观锁。场景查询参与者时，系统保留查询基线；场景完成计算与校验后，内置更新服务依据基线进行条件更新。如果相关普通事实已被另一个请求修改，本次更新就不能直接覆盖对方的结果，而应报告并发冲突。冲突后的重试或技术恢复策略在 APP 层定义。

例如，两个请求都读到同一份客户资料。第一个请求修改地址并成功保存，第二个请求仍拿着原资料修改联系电话。第二次保存需要发现查询基线已经变化，不能把整份旧资料写回，导致地址又变成旧值。乐观锁让查询和业务计算不必一直占有记录锁，同时在实际更新时检查是否发生竞争。

普通更新默认的乐观锁，与事务内查询的自动锁定可以同时存在。查询放在事务外时，不提前锁定记录，更新依靠查询基线作条件提交；查询放在事务边界内时，系统按参与者的属性和本事务的修改范围决定是否锁定读取。

查询对象	事务内的读取方式
普通参与者	自动锁定读取，当前 JDBC 实现使用 `SELECT ... FOR UPDATE`
量子事实	不添加普通记录读锁，由量子实现保障并发成交
同时含普通与量子属性的参与者	本事务也修改普通属性时锁定普通事实；只有量子变化时不添加普通读锁

SPP 业务源码不需要显式写 `select for update`，但这不意味着运行时没有锁定读取能力。例如，把普通客户资料查询放在事务外，系统采用查询基线保护后续保存；放在事务内，内置查询会锁定相应普通记录。锁定普通记录也不能理解为锁定钱包余额等量子事实，其增减仍由量子服务处理。具体规则见 事务内查询与量子事实。

本地共同提交

Petstore develop 使用 LOCAL。相关域同 JVM，并共用满足约定的数据库、schema 和连接资源，量子变化与普通更新进入同一个本地事务。失败处理在这一资源范围内进行。

相同数据库名称不足以证明共用同一事务连接，跨进程也不能因为访问同一数据库便自动获得 LOCAL。编译和启动检查需要核对实际资源身份。

分阶段执行

非 LOCAL 配置下，集中量子组可以先成交，普通更新按模式分组执行，不同已提交阶段通过 Saga 恢复。已经提交的量子变化不能再依赖后续普通数据库的回滚撤销。

执行计划因此需要明确恢复能力。缺少恢复约定的组合应被拒绝，不能由框架猜测业务补偿。跨阶段共同成功的含义也应与单资源原子提交区分。

嵌套边界

REQUIRED 加入当前边界，内层结束不额外提交。付款场景中的嵌套 REQUIRED 用于表达逻辑范围，不能把它读成先付款、再独立提交订单。

REQUIRES_NEW 形成独立边界，其结果不会自动归入外层恢复链。例如，付款中的内层 REQUIRED 与外层一起完成；独立开户若采用 REQUIRES_NEW，则外层失败时该开户可能已经完成，业务需要另行处理。示例后端的传播支持范围见附录 E。

编译器自动形成量子优先的执行计划

普通更新由系统内置提供，开发者不编写或修改其实现。更新内部不做业务查询、计算或校验；所需事实由场景的查询、计算和量子服务补全，内置更新使用完整输入保存结果。

普通更新不会影响量子交易的结果，量子交易自身可以交换执行次序。因此，在同一个事务边界内，SPP 编译器自动将量子交易提前到所有普通更新交易之前；量子交易可以乱序执行或合并提交。若成交后还要计算凭证内容、校验办理结果，开发者用串行组安排这些场景步骤，待保存输入完整后再并

发执行普通更新。执行计划由服务职责与交换律形成，无需通过普通更新互调来维持先后顺序。运行层还需提供事务上下文、资源调度、汇合与失败处理，使并发保存兑现该边界的提交与恢复约定。这里的“量子服务之后”表示执行阶段的先后；LOCAL 下仍可在共同本地事务中最终提交。

为什么 SPP 不使用异常处理语法

SPP 不提供 `try/catch/throw`，是为了让场景表达业务规则与办理过程，技术失败的处理策略在 APP 层定义，由运行时与 PHY 提供的资源能力执行。这遵循业务与技术分离的原则：业务人员需要说明什么条件下可以办理、哪些变化应当共同成立，而不必了解数据库连接、网络超时或底层异常类型。设计文档将错误拦截与处理从业务流程中独立出来，参与者执行约定进一步明确技术异常不能成为 SPP 的业务分支条件。

首先，业务条件不满足是一种需要明确表达的业务结果。例如，订单已经发货，不符合“取消未发货订单”的条件，场景通过校验服务拒绝办理。普通校验失败结束当前块，guard 校验决定是否执行它包围的步骤；需要撤销当前事务时，用 `breakTransction` 明确声明。这样，阅读场景就能看出办理条件和失败影响，不必把业务规则藏在某个异常类或 catch 分支中。

其次，技术故障不能被当作业务事实。例如，查询订单时数据库连接失败，并不说明订单不存在；付款请求超时，也不说明付款没有成交。若场景捕获这些故障后直接走“订单不存在”或“重新付款”的分支，就可能作出错误的业务决定。APP 层应定义结果确认、重试和技术恢复策略，运行时根据实际执行与提交证据落实这些策略，不能把尚未确认的结果伪装成确定的业务条件。

这种分工也使场景不依赖具体技术实现。同一付款业务，无论使用本地数据库还是远端量子服务，业务目标与规则都应保持一致。如果场景要分别捕获数据库异常、通信异常和各提供方的错误类型，更换存储或部署方式就会迫使业务代码一起修改。在 APP 层定义这些技术处理策略，业务人员才能集中检查角色、校验、计算和成交要求；生成的 Java 与运行时仍可使用异常处理来落实这些约定。

但业务与技术分离并不意味着业务人员不必考虑失败。例如，申请已经受理，后续办理无法完成，是保留申请等待重办，还是取消申请并退还费用？这些选择会改变业务结果，必须由业务人员明确。

失败补偿

SPP 层明确处理业务层面的失败。失败补偿回答的是：业务条件已经明确不满足，已经产生的业务结果应当怎样处理？这需要业务开发者明确规定，因为相同的业务失败在不同业务中可能有不同的处置方式。

例如，库存不足导致订单无法履约，顾客可能接受等待补货，也可能要求取消并退款。框架能确认库存占用和付款的实际结果，却不能替顾客与商家选择如何继续。`transaction { ... } fail { ... }` 用于表达这类失败后的业务操作。系统先完成事务执行与技术恢复，再把失败事实交给相邻的 fail 块；fail 中需要保存业务处置时，另开显式事务。

补偿操作首先取决于失败发生时，哪些业务事实已经成立：

失败情况	业务开发者需要明确的操作
准入校验失败，尚未发生业务变化	拒绝办理并说明原因；没有实际扣款就不应退款
库存已经占用，后续业务校验拒绝订单	是否取消本次申请；取消时释放本次实际占用的库存
付款已经成交，商家确认无法履约	是保留付款等待补办，还是取消订单、退款并释放相关占用
申请独立受理成功，后续审批未通过	是保留申请等待重办，还是将申请改为已拒绝或已取消；已收费用如何处理
商品已经交付，后续业务无法完成	按约定安排退货、退款或其他补救；改回订单状态不能代替实际退货

这里列出的是需要作出的业务决定，不能机械地把每次失败都变成“逆序撤销所有步骤”。共同事务中尚未提交的变化可以由技术回滚撤销；量子服务已提供全自动反交易，业务开发者无需另写对应的反交易或补偿交易。业务仍需明确取消订单、释放业务占用或继续履约等处置。业务完成后另行申请的部分退款、扣费等，则应按其自身规则办理，不能把原付款记录当作从未发生。

补偿操作也需要遵守这一分层。例如，退款申请不符合业务规则，属于 SPP 需要明确处理的业务失败；执行退款更新时超时，则属于 APP 需要处理的技术失败。SPP 规定退款的资格、金额及业务结果，APP 规定更新结果的确认、重试和恢复，不能把通信超时直接变成“退款被业务拒绝”。

实例：预约已受理，库存不足时记录拒绝

Petstore 的预约场景先保存受理申请，再尝试预占库存。假设可用库存为 10 件，顾客申请 11 件：业务已经受理，但预占不能完成。这个场景选择保留原预约号，并把申请记为“已拒绝、库存不足”。下面摘自 真实预约场景，省略前面的准入查询和计算：

```
transaction { insertPetReservationAgreement; }
prepareReservationInventory; findStockLine; identify(StockLine[]);
transaction {
    reserveRequestedInventory bounds {
        StockLine[].available: [0, null]; StockLine[].reserved: [0, null]; };
    confirmPetReservation; updatePetReservationAgreement;
} fail {
    if (ResultInfo.failure.recoveryStatus == "ROLLED_BACK" || ResultInfo.failure.recoveryStatus == "REVERSED") {
        findStockLine; identify(StockLine[]);
        rejectPetReservation;
        transaction { updatePetReservationAgreement; }
    }
}
```

第一个 transaction 独立保存受理申请，后面的预占事务失败不会自动删除它。预占成功时，场景将申请改为 RESERVED；预占失败时，系统先回滚未提交变化，或对已成交量子阶段自动反交易，再进入 fail。

`ResultInfo` 是场景内置的系统参与者，不需要在入口中声明。样例只在 recoveryStatus 为 ROLLED_BACK 或 REVERSED 时办理拒绝：前者表示相应事务已回滚，后者表示已提交阶段已恢复。它随后重新查询库存，调用 `rejectPetReservation` 判断业务原因，再用新的 transaction 保存申请状态。这些查询和保存属于失败后的处置，不属于已经失败的预占事务。

拒绝计算服务将申请标记为 REJECTED，并根据恢复后的库存与申请数量区分 INSUFFICIENT_STOCK 和 RESERVATION_NOT_COMPLETED。因此，库存 10 件、申请 11 件的结果是保留原预约号、记录库存不足，库存仍为可用 10 件、预占 0 件。如果预占本可成功，只是后续普通保存失败，恢复后则记录“预约未完成”，不能误记为库存不足。

fail 没有手写“把库存加回来”：这由系统回滚或自动反交易完成。它表达的是业务选择——保留受理记录，并记录申请的最终拒绝状态与原因。

结果不明时，系统会提供 RECOVERY_REQUIRED，样例不会把申请判为已经安全拒绝。fail 可以记录待核对的业务信息，但技术失败仍交给 APP 处理，不能因执行过 fail 就视为恢复完成。处置完成时，系统结果仍为 FAILED，handled 为 true，表示原事务失败已处置，而非原事务成功。当前支持串行场景中的外层事务 fail；多个入口合约时用 `fail(目标合约)` 明确关联。完整协议与限制见 失败处理与系统结果。

技术失败在 APP 层定义

更新服务超时导致结果不明，是典型的技术失败：更新可能尚未提交，也可能已经提交而确认响应未到达。APP 层需要定义如何依据原请求身份查询执行结果、什么条件下允许重试，以及无法自动确认时如何保留技术恢复待办并转交运维处理。运行时和 PHY 的存储、通信能力负责执行这些策略。

例如，订单状态更新超时，SPP 不需要增加“捕获超时后取消订单”的业务分支。APP 应先确认原更新是否完成；已完成就恢复相应执行结果，未完成则按约定重试，仍不明确就保留待确认状态。超时本身不能作为触发业务退款或取消的依据。

同样，业务补偿中的退款或库存释放如果遇到技术故障，其进度记录、防重、结果确认和技术恢复也在 APP 层定义。只有业务上的退款或释放确实完成，才可以报告补偿完成；技术重试不能造成重复退款或重复释放。

对外判断实际结果

请求失败可能意味着未提交，也可能意味着部分阶段已提交，或结果暂时未知。场景的实际状态应根据成交与恢复证据判断，不能直接从一次 HTTP 错误推断所有资源已经回滚。

明确事务语义与部署能力，才能让业务边界成为可靠承诺。详细支持范围见附录 D 与 F。

本篇目录 · 全书目录 · 上一章 · 下一章

第20章 量子属性调用范围与守恒成交

© 2026 S++ project contributors. All rights reserved. Book copyright.

钱包余额、商家资金与库存数量经常面临并发变化。它们的更新不能仅依赖“查询时足够”这一判断，还需要实际成交时的范围控制和一致性保障。量子属性与量子服务将这种需求显式表达出来。

属性与增减

Petstore 声明钱包为量子属性，充值服务描述双边变化：

```
quantum update transferWalletFunds(
    Buyer, SettlementAccount, WalletFundingAgreement) {
    SettlementAccount.balance -= WalletFundingAgreement.amount;
    Buyer.wallet += WalletFundingAgreement.amount;
}
```

服务说明变化关系，调用方通过 bounds 说明本次范围。钱包的上下限通常定义在钱包角色的属性中，场景需要先查询钱包，取得这些限制，再将它们用于本次量子服务调用。服务因此可以保持增减职责，同一个服务能够处理具有不同限制的钱包。

场景先查询钱包限制

例如，普通钱包的余额下限为 0 元、上限为 10,000 元；允许透支的钱包，余额下限可以是 −500 元。这些限制属于具体钱包的业务事实，不宜在转账服务中统一写死为“余额不得低于零”。

付款场景先查询付款钱包，取得余额下限；充值场景先查询收款钱包，取得余额上限。转账同时涉及来源和去向时，场景取得双方的相应限制，再通过 bounds 传给量子服务。例如，向余额为 9,800 元、上限为 10,000 元的钱包转入 300 元，就不能满足其成交范围。

查询限制与查询余额承担不同职责：前者取得本次成交必须遵守的规则，后者可以用于展示或事前判断。场景把查询到的限制用于调用，量子执行则针对实际成交时的余额检查这些限制；不能只凭查询时的余额判断本次交易一定能够成功。

范围在成交时生效

两笔请求都查到钱包 100 元，各自要付 80 元。事前校验可以都通过，实际成交却最多允许其中一笔在不透支规则下成功。量子实现读取并处理当前值，在提交前检查约束。

上下限在调用前求值，null 表示关闭对应方向检查。它不表示未知金额，也不替代支付来源的业务依据。资格条件仍由场景显式校验。

按真实关系守恒

付款的来源减少与去向增加应保持相同金额。库存预占则把数量从 available 转向 reserved，发货再从 reserved 转向 sold。不同商品分别守恒，不能用另一商品的增加掩盖本商品的损失。

资金与库存具有不同维度，不属于同一个数值守恒等式。场景可以要求它们协作，实施时需要各自守恒组和整体事务或恢复计划。

成交结果与参与者回填

成交之后，执行框架把本次结果回填参与者。该结果说明这次成交产生的事实，不能理解为跨多个来源同一时刻的完整快照。后续交易还可能继续修改余额。

普通输入在准备完成后应保持约定稳定，动态量子事实由量子执行机制处理。业务层不能绕过量子服务直接保存非零余额变化。

全自动反交易

量子服务支持全自动反交易。需要撤销原量子成交时，系统依据原成交记录，对原参与者执行相反的增减量，无需业务开发者另写反交易服务或对应的补偿交易。

例如，原交易从甲钱包转出 100 元、向乙钱包转入 100 元，反交易就从乙钱包转出原来的 100 元、向甲钱包转入 100 元。系统恢复的是这笔交易的影响，而不是把双方余额覆盖成交易前的快照，因此可以保留其他交易产生的变化。原交易定位、反向执行、防重与技术失败恢复由系统承接。

业务场景中其他失败所需的业务处置仍要明确，例如取消订单、退回实物或保留申请等待重办。这些处置不要求业务人员重新实现量子服务的反交易。下一章进一步区分回滚、反交易与业务补偿。

本篇目录 · 全书目录 · 上一章 · 下一章

第21章 回滚反交易冲正与补偿

© 2026 S++ project contributors. All rights reserved. Book copyright.

失败恢复容易被一个“撤销”概念包办，但未提交事务、已成交资金、业务冲正和外部补偿承担不同责任。区分它们，才能正确解释失败结果和恢复范围。

回滚撤销未完成的事务

技术回滚撤销当前事务的影响，保留审计证据，并且不能覆盖其他独立事务已经成功产生的结果。能否做到，取决于资源与协议的保证范围。

Saga 中已经提交的节点不再处于原本的本地事务中。失败节点可以回滚自己的未提交变化，之前的成功节点需要另外的恢复动作。

普通更新也可以自动冲正

SPP 的普通更新是系统内置服务，内部不包含业务计算，也不查询其他服务来补全信息。计算与校验已经在场景中完成，更新只把准备好的事实写入存储。因此，系统可以记录本次更新前的事实和更新后产生的版本，在需要恢复时自动撤销这次写入，业务开发者无需另写一个重新计算旧值的反向服务。

例如，客户地址原为“甲地址”，版本为 v8。本次更新把它改为“乙地址”，产生版本 v9。若后续步骤失败，需要撤销这次更新，系统检查当前记录仍为本次写入的 v9，就可以把地址恢复为“甲地址”，同时产生新的版本。恢复的是旧的业务事实，版本标记不能退回 v8，否则原来已经失效的更新请求可能再次被错误接受。

如果另一个请求已经把地址改为“丙地址”，产生 v10，自动恢复就不能覆盖它。即使地址碰巧又变回“乙地址”，只要已经产生新版本，也不能当作没有后续更新。版本检查与恢复写入必须是一个原子条件操作，不能先检查版本，再无条件写回旧事实。

这与量子服务的自动反交易具有不同条件：量子反交易用原增量的相反数恢复该笔影响，可以保留后续增减；普通更新恢复原事实，要求当前版本仍符合本次恢复条件，没有其他更新产生的新版本。条件不满足时，系统报告恢复冲突，由 APP 层定义确认与技术处置策略，不能直接覆盖后续结果。

跨域混合事务怎样回滚与反交易

一个 SPP 事务边界可以包含多个域的更新，但这些域可能分布在不同进程、使用不同数据库。边界表达的是这些业务变化需要共同成立；实际怎样撤销，由 APP 选择的执行计划与 PHY 提供的协调能力决定。跨域、跨进程不能仅凭写在同一个 transaction 中，就获得一个共同的本地数据库事务。

以购买商品为例，同一个边界中有四项更新：

更新	业务变化	所属域
量子更新 Q1	买家钱包减少 100 元，商家资金增加 100 元	付款涉及的角色域
量子更新 Q2	商品可售库存减少 1 件，订单占用增加 1 件	库存域
普通更新 U1	保存订单及其成交明细	订单域
普通更新 U2	保存本次办理凭证	凭证所在域

Q1 与 Q2 分别保持资金和数量守恒。角色可以属于不同业务域，量子变化则由配置的统一量子系统处理。普通更新可能还要跨进程调用各域的仓储服务。当前非 LOCAL 的混合执行计划先完成量子组，再执行普通更新阶段，并记录哪些阶段已经提交；同一边界中的多个量子请求可合并成交，普通更新按配置的事务策略协调。

量子组没有成交，普通更新不继续执行。 例如 Q1 的余额下限或 Q2 的库存下限不满足，量子组无法整体成交。尚未提交的量子变化由其资源撤销，U1、U2 不继续执行。不能只扣款、不占库存，也不能为没有成交的购买保存成功凭证。若量子组的提交结果不明，则先进入技术结果确认，不能把它当作已明确拒绝。

量子组已成交，普通更新组未能提交。 假设 Q1、Q2 已整体成交，U1 已执行，但 U2 发生明确更新冲突。若 U1、U2 加入同一个受支持的普通事务协调范围，例如配置了相应 XA 或 Seata AT 能力，普通阶段先通过该协调机制撤销自己的写入；随后混合计划对已提交的量子组全自动反交易，返还原扣款并释放原库存占用。U1 执行过，并不意味着 U1 已独立提交；必须按所属普通阶段判断它的状态。

恢复过程可以理解为：

```
量子组 Q1+Q2 已成交
    → 普通组执行 U1、U2
    → U2 明确失败
    → 撤销普通组尚未完成的事务
    → 对已成交量子组自动反交易
    → 确认各恢复步骤的结果
```

普通更新已独立提交，就恢复这个已提交阶段。 如果配置采用 Saga，U1、U2 可能分别提交。此时 U1 成功提交、U2 明确失败，U2 所在未提交事务先撤销；执行计划随后按已提交阶段记录逆序恢复 U1，再对量子组执行自动反交易。已经提交的订单不能靠 U2 数据库的 rollback 消失，但可以由具备版本保护的自动冲正机制恢复：系统根据 U1 的原事实和成交版本，确认没有后续新版本后撤销其写入。业务开发者无需为这次普通保存再编写一套补偿计算。若已经出现后续版本，则保留恢复冲突，按 APP 策略处理。

任一阶段结果不明，先确认结果，再决定恢复。 例如 U2 的远程更新超时，它可能已提交凭证，也可能没有执行。当前混合计划发现结果不明时，不立即对前面的已提交阶段执行反交易，而是保留待恢复状态。APP 层需要依据原请求身份确认 U2 的结果，确认后再按实际提交记录完成或恢复交易。否则，凭证和订单可能已经成功保存，资金与库存却被错误地撤回。

恢复过程中再次失败，整笔事务仍未恢复完成。 例如 U1 的恢复失败，或者量子反交易响应丢失，运行时保留原失败和恢复失败证据，记录恢复待处理状态。当前实现会记录单个阶段的恢复失败，并继续尝试其他已提交阶段，因此可能出现部分恢复，必须核对每项结果。它不会把“已发起恢复”报告为“已全部撤销”。APP 层定义后续确认、重试和技术处置；量子反交易利用原成交记录和防重机制，不能因重试重复返款。恢复期间哪些业务可以继续，仍应遵守明确的业务约定。

这里的“回滚”用于尚未提交的当前阶段；已提交的量子变化通过自动反交易恢复，已提交的普通更新可以在版本条件满足时自动冲正。业务完成后另行申请的银行冲正，则是后文所述的新业务，不能与事务执行失败时的自动恢复混为一谈。

上述自动恢复需要后端提供原事实、版本、执行记录和相应恢复协议，跨进程普通更新是否能加入同一协调范围，也取决于实际部署能力。不同域共用数据库并不意味着它们共用事务连接；配置了 XA、Seata AT 或 Saga 名称，也不等于任意部署都已具备完整恢复能力。相关配置及边界见 环境部署与事务存储约定。

某银行案例中的本地回滚与分库恢复

第七篇第33章的某银行存款案例提供一个实际对照：现金冲正已经执行资金反向变化，随后保存反向分录失败。

部署与失败位置	资金与普通事实的实际结果	系统结果
共用本地事务资源，反向分录保存失败	资金、原交易标记、冲正合约共同回滚；原交易仍未冲正	`ROLLED_BACK`
量子库与普通库分开，部分普通阶段已提交后分录保存失败	量子反交易恢复资金；原交易标记、冲正合约等可能已有普通残留	`RECOVERY_REQUIRED`，`handled=false`
失败处置记录也保存失败	保留原交易失败、实际恢复状态与处置二次故障	不能因进入 `fail` 就改成处置成功

当前案例的分库仓储尚未完成已提交普通阶段的自动恢复。这不改变普通更新可以按版本条件自动冲正的原理，但说明相应后端协议需要真正实现并验证。记录待核对事实是失败处置，恢复资金与普通记录才是事务恢复。

相应测试还通过修改已查询的原合约版本，验证冲正不能覆盖后来发生的变更。分库资金虽已恢复，先前普通阶段仍可能残留，系统同样保留待恢复结果，不能让调用方直接重试来掩盖它。案例验证记录给出了实际持久化结果。

全自动反交易保留后续结果

余额最初为 100，第一笔增加 50，第二笔又增加 20，当前为 170。撤销第一笔时减去原来的 50，结果为 120。若直接恢复旧快照 100，就会抹去第二笔业务。

量子服务依据原成交记录全自动执行反交易，业务开发者无需另写反交易服务或对应的补偿交易。原成交分量、参与身份、防重和结果确认由系统维护；反交易遇到技术故障时，其确认与恢复策略在 APP 层定义。执行反向请求报错，也不能直接说明反向变化未成交。

冲正是一笔新业务

银行因业务原因撤销原交易时，需要查原交易、判断资格，并形成新的办理结果。原偿还本金和利息应按原成交分量恢复，不能用当前政策重新计算后假定等价。

冲正有自己的事务和失败处理。跨日规则、后续依赖、已发放资金和外部效果都可能影响资格。原业务完成后仍可提出冲正，这与原事务提交前的技术回滚不同。

补偿需要业务决定

若货物已经使用，或某个外部效果无法撤回，业务可能采取其他方式处理。补偿内容、触发条件和完成标准需要明确约定，系统只能执行、记录和重试已有规则。

量子服务的反交易由系统自动提供，普通更新也可以在版本条件满足时自动冲正。这里需要业务决定的是无法仅靠恢复存储事实完成的处置，例如退回实物或安排替代履约。外部效果不会因为数据库记录恢复而自动消失；普通更新一旦产生后续版本，也不能仅凭旧值强行恢复。

APP 将明确的技术失败转换为业务失败

技术失败可以由 APP 层转换为业务失败，前提是执行结果已经确定，并且业务约定赋予它明确的含义。例如，更新因唯一性冲突被明确拒绝，APP 可以按照业务规则转换为“该编号已被占用”；乐观锁冲突可以转换为“资料已变化，请重新确认”。SPP 随后处理这些业务失败，无需了解数据库错误类型。转换规则需要在 APP 层明确，不能把任意技术错误都当作同一种业务拒绝。

超时、连接中断或提交确认丢失则不同：它们可能发生在提交之后，只能说明调用方没有收到确定结果，不能据此认定业务没有成功。APP 应先确认原执行结果，再决定返回成功还是明确的业务失败；暂时无法确认，就保留结果不明并进入技术恢复，不能直接触发 SPP 的业务失败补偿。

因此，业务与技术分离并不禁止失败信息跨层传递。APP 可以将结果确定、业务含义明确的技术失败转换为业务失败，交给 SPP 处理；结果不明的技术失败则需要先由 APP 确认与恢复。

未知结果的处理

网络超时可能发生在提交之前，也可能发生在提交之后、响应之前。应通过稳定请求身份和结果查询确认原执行，不应直接发起新身份的重复交易。

当前仓库已有量子反向执行与多类恢复证据，完整持久恢复协调与业务补偿仍有需要分别说明的范围。附录 F 将已有案例验证与未决事项分开列出。

本篇目录 · 全书目录 · 上一章 · 下一章

第22章 生成链路物理资源与运行配置

© 2026 S++ project contributors. All rights reserved. Book copyright.

声明式业务模型最终需要变成可执行代码、接口、仓储与资源访问。生成链路承担这项转换，同时检查模型和实现契约是否匹配。阅读生成结果，是理解声明实际作用的重要方法。

从模型到运行产物

编译器解析业务模型、服务调用、扮演关系与事务边界，再结合 APP 和 PHY 生成 Java 模型、服务、映射、OpenAPI、DDL 和运行目录等产物。具体输出依模块与配置而定。

这条生成链路也承接第三篇第12章介绍的 AI 协作方式：AI 主要维护 SPP、APP 和 PHY 源码，大量模型、映射、接口和框架接入代码由工具按规则生成。减少 AI 自由输出的实现代码，也减少需要由 AI 逐处保持一致的关系。

业务变更的人工审查因此可以集中在 SPP 的角色、规则与场景，并通过业务结果验证它们。大量生成代码的重复结构由工具链维护；工具链或适配能力改变时，再针对相应生成行为检查实现。这样的分工减少逐次审查全部混合代码的负担，也使人更容易核对 AI 输出是否符合业务需求。

业务源码保留在源目录，生成 Java 位于 target。修改生成文件只能影响一次构建产物，下一次生成可能覆盖它；业务规则应在 SPP 或相应源配置中维护。

生成时与启动时的检查

生成时可以发现未定义事实、歧义绑定、非法服务调用、不支持事务传播或矛盾资源配置。启动时还要根据实际覆盖参数检查环境、资源身份、提供方契约与就绪状态。

静态检查不能证明远程服务此刻能够完成所有交易，启动检查也不能代替并发与故障验证。不同层次的检查对应不同证据。

构建与本地演示

以下命令在仓库根目录执行，要求仓库说明中的 JDK 17、Maven 与依赖条件：

```
mvn -pl :petstore-demo -am package -DskipTests -Dspp.environment=develop
SPP_PETSTORE_JDBC_URL='jdbc:h2:file:/tmp/spp-book-petstore' \
  ./demos/petstore/run-demo.sh web
```

演示网页默认使用 8086 端口。这里指定独立实验数据库；重复使用同一路径会保留先前数据。完整环境变量和分进程命令见附录 E。

数值与资源

PHY 选择数字表示和存储容量，APP 组织技术映射。业务表达中的舍入仍由 SPP 明确，物理转换无法精确兑现时应失败。

相同的业务场景在 develop 与 testing 下可以使用不同部署关系和提交计划。比较运行结果时，应同时查看所选环境、生成目录与实际启动参数，避免用一套产物证明另一套配置。

验证声明的兑现

构建成功说明源码和生成链路达到相应检查层级，实际业务仍需要调用场景并核对结果。下一篇沿 Petstore 展开，从业务需求到资金与库存流转，观察角色与服务架构如何形成一个完整系统。

本篇目录 · 全书目录 · 上一章 · 下一章

第五篇 业务与技术分离

© 2026 S++ project contributors. All rights reserved. Book copyright.

业务场景已经说明谁参与、怎样计算、何时成交以及失败后怎样处理。接下来讨论技术能力怎样围绕这些业务表达工作：让系统更快、更安全、更可靠，并且能够观察、治理和持续演进。

本篇从分离的收益出发，用同一笔订单的不同技术需求说明缓存、按需数据、安全保护、幂等、防重和运行治理。APP 定义技术策略和服务拦截范围，PHY 绑定 Java 技术服务及实际资源与协议。技术人员可以实现新的 Java 技术服务，通过拦截器接入业务调用，扩展系统能力，而不向 SPP 加入技术调用。第23章介绍接入结构，第27章给出自定义实现示例；各项能力的接入方式仍需要符合其实际接口。

本篇目录

- 第23章 业务与技术分离的主要收益
- 第24章 缓存与按需数据提升性能
- 第25章 加解密指纹与安全服务
- 第26章 幂等防重与可靠执行
- 第27章 运行治理与技术演进

返回全书目录

第23章 业务与技术分离的主要收益

© 2026 S++ project contributors. All rights reserved. Book copyright.

一笔付款的业务要求是：符合资格的付款方向收款方支付约定金额，形成可以核对的办理结果。加密通信、查询缓存、重复请求识别和数据库接入，都在帮助这项业务可靠执行。S++ 把业务要求写在 SPP，把技术策略放在 APP，把实际资源与协议落实到 PHY，使三类工作各自有清楚的维护位置。

让业务人员能够直接检查业务

例如，会员折扣从九折改为八五折，业务人员需要核对适用身份、优惠比例和报价结果。若规则混在数据库访问、接口转换与重试代码中，就必须先理解大量技术细节，才能判断改动是否完整。

在 SPP 中，折扣由计算服务表达，适用条件由校验服务表达，场景说明何时调用。需求人员可以沿着这些表达审查业务，也可以与 AI 配合完成建模。默认查询、保存和接口接入由工具提供，业务审查集中于业务事实与规则。第三篇第12章介绍了这种分工怎样降低 AI 必须自由生成和维护的代码量。

技术工作因此也更集中。缓存命中率低，应检查数据访问与缓存策略；重复请求造成重复办理，应检查请求识别与执行协议。这些问题有自己的位置，不必先修改会员折扣或付款公式。

同一业务可以适配不同环境

客户资料可以来自本地数据库，也可以来自远程客户域；通信可以采用不同协议，敏感字段可以使用不同的保护方案。业务场景仍然使用买家角色，并按相同业务要求计算与成交。

例如，把客户域从同一个进程迁移到独立应用，需要调整 APP 的服务绑定和 PHY 的端点、资源配置，重新核对事务与恢复能力。只要提供方仍兑现原来的业务约定，付款场景不必增加 HTTP 报文转换或连接管理代码。

技术替换需要保持的不只是类型，还包括事实含义、更新效果和失败结果。旧接口返回“资料不存在”，新接口不能把网络超时也当成不存在。分离让适配责任明确，也使这种差异能够在接入层被发现和验证。

技术能力能够集中复用

订单、预约、充值都可能需要识别重复请求。如果每个场景分别实现登记、加锁、保存结果和超时恢复，细小差异就可能导致不同的重复执行行为。由技术层提供统一协议，各场景按需要配置，可以减少重复实现，并集中改进其可靠性。

加解密、数据压缩、日志追踪和请求限流也有类似价值。技术人员维护共同能力，业务开发者声明业务需要的角色、事实和办理结果。能力升级后，可以通过适配与配置作用于多个场景。

通过 APP 拦截接入 Java 技术服务

技术服务不必都由 S++ 编译器预先内置。技术人员可以用 Java 实现新能力，再由 APP 将它接到场景或原子服务的调用过程。SPP 仍只表达业务，APP 说明作用于哪些服务、何时执行，PHY 指定实际 Java 类与配置。

例如，要求“商店的所有服务都记录调用耗时”，不用逐个修改付款、下单和查询场景。Petstore 的 技术服务 APP 已经这样配置：

```
technical services { invocationTrace; }
intercept services {
    before: invocationTrace;
    after: invocationTrace;
    failure: invocationTrace;
}
```

`invocationTrace` 是技术能力的名称，`intercept services` 匹配当前业务域全部生成服务，包含场景、普通查询与更新以及量子服务。若只观察场景，可以用 `intercept scenarios`；若只观察指定服务，可以用 `intercept findBuyer, reserveShopPet`。这是技术绑定，不是让业务场景调用一个日志原子服务。

技术服务 PHY 为这个名称指定实现：

```
phy {
    technical service invocationTrace {
        implementation: "free.servpp.service.technical.InvocationTraceService";
        config logger = "spp.petstore.invocations";
    }
}
```

接入新能力时，Java 类实现框架的 `TechnicalService` 接口，放入应用运行时类路径；APP 声明名称与拦截范围，PHY 绑定类名与参数。第27章将沿这个接口实现一个慢调用观察服务。

三个阶段有明确含义：

阶段	执行时机	典型用途
before	业务服务调用前	技术访问控制、调用开始记录
after	业务调用正常返回后	耗时、校验结果和调用完成记录
failure	技术错误向外传播时	故障诊断和告警记录

`after` 表示方法正常返回，不保证业务通过校验，也不代表外层事务已经提交。预约失败经过 `fail` 处置后正常返回，场景也会进入 `after`。技术服务需要按这些语义记录结果，不能只看到 `after` 就认定付款成功。

当前接口提供服务名称、类别、可信调用头、耗时、校验结果及技术错误等只读元信息，不提供可改写的业务参与者。它直接适合调用观察和基于调用身份的技术控制；修改数据、替代查询或改变返回内容，需要相应适配接口。分离使 Java 技术能力能够持续扩展，同时保持业务事实与规则的维护位置明确。完整协议见 APP 技术服务与服务拦截。

技术优化必须兑现业务约定

技术策略可以改变如何执行，业务目标和规则仍需要保持。例如，商品名称展示可以接受一定时间的缓存，付款成交则不能根据缓存余额直接覆盖账户。订单列表可以只返回摘要，办理退货仍需要取得判断资格所需的事实。

因此，分离需要双方协作：业务明确事实用途与结果要求，技术选择相应策略。技术层不能为了提速省略资格校验，也不能为了提高成功率悄悄放宽量子成交下限。缓存时效、失败恢复和接口输出都要符合相应业务约定。

本篇下面几章将沿性能、安全、可靠执行和运行治理，说明技术服务可以怎样提升系统能力。这些服务可以由运行时、接入设施或适配器提供，不要求每项技术能力都成为一个远程微服务。

本篇目录 · 全书目录 · 上一章 · 下一章

第24章 缓存与按需数据提升性能

> © 2026 S++ project contributors. All rights reserved. Book copyright.

提升性能不仅是增加并发，还包括减少重复访问、减少无用数据，以及让已经取得的事实得到合理复用。SPP 继续按角色组织业务，APP 可以围绕内置查询、数据传输和接口输出选择优化策略，PHY 提供缓存与存储资源。

先用拦截器观察性能，再选择优化位置

可以为查询或场景配置耗时技术服务，通过 APP 的 `after` 和 `failure` 收集服务名称、调用耗时与错误。这样能发现订单耗时主要来自商品查询还是普通保存，无需在 SPP 中增加计时代码。技术人员还可以用 Java 实现新的指标采集器，并通过 PHY 接入现有监控系统。

当前服务拦截器只读取调用元信息，不取得可修改的参与者，也不提供“返回缓存后跳过原查询”的协议。因而耗时观察可以直接接入，查询缓存、按需字段读取和返回数据裁剪则要在查询、仓储或输出适配中实现。它们同属技术层能力，各自使用适合的扩展位置。

缓存减少重复访问

例如，购物车有十条商品明细，多条明细使用同一类商品资料。请求内缓存可以复用已经取得的事实；进程内共享缓存可以服务不同请求；多个应用实例还可以通过共享缓存减少对来源域的重复访问。

对于远程产品域，本地缓存商品名称、分类等相对稳定的资料，可以减少网络等待。商品展示仍调用角色查询，缓存命中后由技术层完成事实补全，场景无需手写“先查缓存，再查数据库”。

缓存必须区分参与者身份、字段范围和版本。取得过商品名称，不等于取得过它的价格与销售状态。缓存刷新和失效策略也要符合用途：展示名称可以采用较宽松的时效，实际成交必须由相应执行机制检查权威事实。缓存余额只能辅助展示或准备，不能替代量子成交。

按需查询减少读取的数据集

角色可能沿扮演链包含大量属性，一次报价却只用到会员等级、商品单价和数量。可以在编译时收集角色被使用的属性，形成默认查询所需的数据集合，再由查询实现按来源取得这些事实。

例如，Buyer 的资料包含姓名、头像、地址与会员等级。优惠计算只需要会员等级，就没有必要为它读取头像和整份地址；下单还需要地址时，再把相应事实纳入查询范围。服务颗粒度仍是对 Buyer 的查询，优化的是实际加载的数据量。

查询还要保留必要的身份和内部基线，并标明哪些属性已经加载。没有查询到的地址不能被当成空地址保存，否则一次性能优化就会变成数据丢失。部分加载与普通保存之间需要明确协议，不能仅靠生成一条更短的 SQL。

按需返回减少传输的数据集

场景执行需要的事实与页面展示需要的事实并不相同。报价可能使用会员等级、地址、商品完整定价信息，页面却只展示商品名称、数量、单价和总额。

APP 可以定义不同入口的输出字段集合：订单列表返回订单号、时间、金额和状态；订单详情返回明细；付款结果返回办理凭证与系统结果。业务角色和合约仍保持原有含义，输出适配只选择入口允许且需要展示的数据。

按需查询决定系统取什么，按需返回决定接口给什么。即使场景内部需要某项敏感事实，页面也未必应当收到它。输出选择还要与权限、脱敏及幂等结果回放保持一致。

当前生成网页接口主要返回执行后的入口对象及结果信息，通用的按需输出方案需要进一步扩展；按角色属性使用集合裁剪默认查询及通用多级缓存，也应按各自实现范围核对。本节说明优化方法，不把这些方案一概视为当前已完成的能力。查询、缓存与部分保存的详细条件见附录 C 的 C13 节。

批处理与执行路径优化

同一业务需要取得多个独立商品时，可以批量查询或并发查询，减少串行等待。多个量子服务可以合并请求，减少通信损耗；输入已准备完整的普通更新，也可以按执行计划并发办理。

计算和校验通常可以部署在场景所在进程，直接使用已补全的事实，避免为短小计算增加网络调用。压缩较大的传输数据可以减少带宽消耗，但也增加编码与解码成本，应根据实际数据量选择。

优化效果应同时观察完整场景耗时、读取与返回的数据量、缓存命中和来源访问次数。例如，一个订单列表响应变快，却因详情接口反复补查增加了总访问量，就还需要调整数据集合。实验方法见附录 C。

本篇目录 · 全书目录 · 上一章 · 下一章

第25章 加解密指纹与安全服务

© 2026 S++ project contributors. All rights reserved. Book copyright.

业务系统既要执行正确的规则，也要保证请求来源可信、数据受到保护、操作可以追溯。APP 可以组织认证、加解密、完整性验证、输出控制等技术能力，PHY 提供实际协议、密钥设施与资源。业务场景继续围绕可信的参与者事实办理业务。

加解密保护传输与存储

例如，客户地址需要跨域传输，也需要保存到存储中。技术接入可以为通信提供加密保护，为敏感存储字段选择加密方案。业务计算与校验使用具有明确含义的地址，不需要自己处理密文格式、密钥选择或解密调用。

加密策略要明确作用位置。传输加密保护通信过程，存储加密保护落盘数据，二者不能互相替代。密钥由专门设施管理，实际端点与配置由 PHY 接入。密钥更换和协议升级主要改变技术适配，地址的业务含义保持一致。

有些处理必须发生在业务之前，例如认证和解密；另一些发生在业务之后，例如输出加密。这些能力需要自己的执行顺序，不能因为业务原子服务可以并发，就把所有技术处理也任意重排。

指纹识别内容，完整性保护验证内容

这里的内容指纹，是对一组数据按约定规则计算出的摘要，用于识别内容是否发生变化。例如，同一个请求流水号第一次提交转账 100 元，第二次却变成 200 元，技术层可以比较请求事实或其指纹，识别出这不是同一次请求的重试。

指纹需要覆盖业务相关内容，不能只对金额计算而忽略付款方和收款方。数据的规范化方式也要一致，否则字段顺序或格式变化可能被误认为内容变化。请求身份说明是哪一次调用，指纹说明其内容是否符合原调用，两者共同参与协议检查。

普通内容指纹本身不能证明发送者身份，也不能防止攻击者连同内容一起替换摘要。需要防篡改和来源证明时，应由技术接入采用相应的认证与完整性保护机制，例如带认证的消息校验或签名。SPP 使用验证后的业务事实，技术层负责检查报文是否可信。

如果指纹指的是生物特征认证，它用于确认访问者身份，同样由认证接入处理；若某项业务明确要求客户完成身份核验，这项业务资格及核验结果则需要进入业务规则。认证手段与业务办理资格分别有明确位置。

认证权限与业务资格协作

技术认证确认“当前调用者是谁”，访问控制确定“是否允许访问这个入口与这些数据”。业务校验进一步判断“是否符合本次办理条件”。三者共同起作用。

例如，经过登录认证的买家可以进入订单查询接口，但只能读取允许访问的订单；申请退款时，还需要检查订单归属、交付状态与退款期限。APP 提供可信的调用者身份和入口保护，SPP 明确这些业务关系与资格。

客户端提交一个 Buyer 对象，不代表它已经证明自己就是该买家。浏览器的防重标识也不能当作认证身份。技术接入需要验证来源，并把可信身份与业务参与者绑定，业务场景再按相应事实办理。

用 before 接入技术访问控制

例如，某个内部入口只允许已验证的指定调用方访问。Java 技术服务可以读取受信任调用头，在 `before` 检查调用方身份；检查不通过就拒绝调用，业务服务不会开始执行。APP 选择受保护的入口，PHY 配置实现及参数，SPP 不需要增加技术身份校验代码。

`before` 失败会阻止业务调用，并进入 `failure` 记录原技术错误。登录认证与头信息的可信建立仍应由接入设施完成，拦截器不能把用户随意填写的身份当作已认证身份。订单归属、付款资格等业务条件仍由 SPP 校验。

当前接口不开放地址、金额或报文内容，所以字段加解密、内容指纹计算与返回脱敏，需要数据或通信适配。安全审计可以利用拦截事件，但它记录的是调用过程；需要逐笔提交证据时，还应关联事务与存储记录。

输出控制与安全审计

订单详情可能需要展示联系电话，但列表只需显示部分号码。APP 可以结合入口权限、字段选择与脱敏规则控制输出，同样的保护也应覆盖日志和结果回放，不能只过滤正常页面响应。

技术审计记录访问者、入口、验证结果与执行轨迹，业务合约记录成交金额、当事人和办理结果。两类记录可以通过追踪信息关联，分别用于安全核查和业务核对。记录不需要暴露密钥、完整证件或其他不必要的敏感内容。

这些安全能力可以统一接入多个场景。具体算法、密钥管理、认证协议和权限配置需要由相应技术实现提供，并验证其作用范围。本章说明分层职责，不假定当前生成框架已经内置全部安全设施。

本篇目录 · 全书目录 · 上一章 · 下一章

第26章 幂等防重与可靠执行

© 2026 S++ project contributors. All rights reserved. Book copyright.

一笔付款可能因为页面重复点击、通信重试或结果确认丢失而收到多次请求。可靠执行需要识别这些请求与原业务的关系，避免把重复传输变成重复扣款。APP 定义相应技术策略，SPP 继续说明付款资格、金额和业务结果。

幂等识别同一次请求

例如，买家支付 100 元，服务已经成交，但响应未到达页面。调用方使用原请求身份和原内容重试，系统应恢复原来的执行结果，而不是再次扣除 100 元。

当前远程更新协议使用请求方身份、请求方流水号和目标服务识别请求。相同身份、相同内容的重试可以回放原参与者事实与结果；同一个流水号改变金额或收款方，则属于协议冲突。幂等登记不仅保存一个“见过此编号”的标志，还需要区分正在执行、已经完成和需要恢复的请求。

业务订单号与技术请求流水号承担不同责任。一个订单可以有不同办理请求，技术重试则要继续关联原请求；不能每次重试都换一个新流水号，再希望系统自动知道它们属于同一次付款。

防重识别短时间内的重复操作

幂等处理的是同一次请求的重复传输，界面防重还要识别用户短时间内发出的多个相似操作。例如，用户连续点击“加入购物车”，两次点击可能产生不同请求流水号，却属于同一次误操作。

APP 可以为页面入口声明防重范围、内容和时间窗口，如同一买家、同一商品、相同数量，在短时间内合并重复办理。窗口结束后，用户再加一次商品可能是合理的新操作，不能永久拒绝。

订单是否允许再次支付则是业务资格，需要 SPP 校验。界面防重窗口过期不意味着订单可以再次付款，幂等期限结束也不意味着原付款没有发生。技术防重与业务规则共同保障办理结果。

当前实现将远程调用幂等与页面防重分开，本地调用不重复登记请求；量子成交内部去重仍由相应量子实现保障。普通查询不通过更新请求幂等协议回放结果，查询缓存是另一项策略。具体配置与协议见 调用幂等与界面防重。

乐观锁保护不同请求之间的竞争

幂等不能解决所有并发问题。两个不同请求分别修改同一份客户资料，都可能是合法的新请求，却使用了同一份旧事实。

普通更新通过查询基线进行条件提交，发现后续版本时拒绝直接覆盖。量子服务则按增减关系和实际成交范围执行。请求去重、普通版本保护与量子成交分别保护重复调用、竞争覆盖和数量变化，不能只配置一个防重键就替代全部机制。

结果确认决定是否可以重试

更新超时可能意味着尚未提交，也可能意味着已经提交而确认丢失。APP 应先依据原请求查询执行结果，或保留技术恢复状态。结果不明时直接重复执行整个场景，可能造成重复扣款或重复创建申请。

已经确认完整撤销、且没有独立阶段提交的请求，可以按协议允许再次尝试；已经部分提交或无法确认的请求需要继续恢复原办理过程。恢复时，量子服务自动反交易，普通更新可以按其版本保护契约恢复，其他业务处置由明确的业务规则决定。相关区别见第四篇第21章。

技术重试也需要期限、次数与退避策略。重试失败不会自动产生一种新的业务补偿政策；超过技术处理能力时，应保存足够的执行证据并进入明确的技术处置流程。

拦截器观察可靠执行，执行协议负责确认结果

自定义 Java 技术服务可以关联请求身份、服务名称和技术失败，帮助发现重复调用及异常办理。例如，付款场景入口每次调用都可以留下事件，包括原请求的场景幂等重放；重放没有再次执行原子服务，就不会再次出现对应的业务执行事件。

观察到两次入口调用，不能直接认定扣款发生两次；原子服务出现 `after`，也不能直接认定外层事务已经成交。确认是否重复扣款，仍要核对请求登记、量子流水和业务凭证。

当前 `after` 与 `failure` 是观察阶段，技术服务处理失败不会替换原业务返回或原技术错误。接口不提供自行重试、修改业务结果或独立提交回调，不能在 `failure` 中直接重跑付款。请求幂等、事务确认和反交易继续由相应执行协议承担，拦截器为这些协议提供诊断与观察能力。

持久记录让恢复跨越进程重启

如果去重记录只在内存中，服务重启后就可能忘记原请求。对需要重启恢复的更新，应使用满足要求的持久登记与结果存储，并协调登记状态与实际成交事实。

当前框架提供内存与 JDBC 请求存储，二者能力范围不同；结果不明或部分提交的记录不能因普通过期清理被当成全新请求。分库零记录建立也通过持久技术待办恢复，而不是要求业务场景重新实现开户补偿。技术策略集中维护，业务人员才能持续关注办理结果，而不必为每个场景重写一套重试与防重代码。

本篇目录 · 全书目录 · 上一章 · 下一章

第27章 运行治理与技术演进

© 2026 S++ project contributors. All rights reserved. Book copyright.

业务模型正确以后，系统还需要在负载变化、依赖故障和环境迁移中持续提供能力。APP 可以组织追踪、限流、隔离和技术恢复，PHY 提供实际部署、资源与监测设施。这些能力围绕业务服务工作，使问题可以发现、定位和处理。

追踪与指标帮助定位问题

例如，订单场景变慢，可能是产品域查询耗时，也可能是库存成交等待或订单保存发生冲突。只记录“下单失败”，无法判断应该优化哪一项能力。

技术追踪可以关联场景、跨域调用、事务阶段和恢复动作；指标可以统计耗时、错误率、缓存命中率、普通更新冲突与待恢复请求数量。量子阶段已成交、普通阶段失败和反交易尚未确认，也应能在轨迹中区分。

业务合约用于回答买了什么、付了多少、目前如何办理；技术轨迹用于回答调用经过哪里、哪个阶段失败以及恢复进行到哪一步。两类信息通过标识关联，页面业务状态不需要混入数据库连接或线程细节。

用 Java 增加一个慢调用观察服务

假设已有日志之外，还希望单独记录超过 200 毫秒的调用。可以新增 Java 技术服务，而不修改任何业务场景。以下示例使用当前 TechnicalService (maintainer reference: `../../appframe/src/main/java/free/servpp/service/technical/TechnicalService.java`) 与 TechnicalInvocation (maintainer reference: `../../appframe/src/main/java/free/servpp/service/technical/TechnicalInvocation.java`) 接口：

```
package example.technical;

import java.util.Map;
import java.util.logging.Logger;
```

```
import free.servpp.service.technical.TechnicalInvocation;
import free.servpp.service.technical.TechnicalService;

public final class SlowCallObserver implements TechnicalService {
    private long thresholdNanos;
    private Logger logger;

    public SlowCallObserver() {}

    @Override
    public void configure(Map<String, String> config) {
        long millis = Long.parseLong(config.getOrDefault("thresholdMillis", "200"));
        if (millis < 0) throw new IllegalArgumentException("Negative threshold");
        thresholdNanos = Math.multiplyExact(millis, 1_000_000L);
        logger = Logger.getLogger(config.getOrDefault("logger", "spp.slow"));
    }

    @Override
    public void handle(TechnicalInvocation event) {
        if (event.elapsedNanos() > thresholdNanos) {
            logger.warning("service=" + event.serviceName()
                + " phase=" + event.phase()
                + " elapsedMicros=" + event.elapsedNanos() / 1_000);
        }
    }
}
```

实现类需要公共无参构造器。注册时调用 `configure`，执行时调用 `handle`；实例可以被并发使用，因此不要把某一次请求的开始时间或身份放进共享成员。这里阈值与日志器在初始化后不再修改，每次调用的数据来自当前事件。

将类放入应用运行时类路径之后，APP 可以只观察场景：

```
technical services { slowCall; }
intercept scenarios {
    after: slowCall;
    failure: slowCall;
}
```

PHY 指定实现与参数：

```
phy {
    technical service slowCall {
        implementation: "example.technical.SlowCallObserver";
        config thresholdMillis = "200";
        config logger = "spp.slow";
    }
}
```

慢调用观察只是一个起点。相同接入结构可以连接指标库、调用链系统或技术告警设施；若加入基于调用元信息的访问控制或限流，可以选择 `before`。线程池隔离、远程取消、报文转换等能力还需要相应运行时或通信适配，不能只靠观察事件完成。

明确拦截范围与错误含义

全服务匹配只作用于声明所在业务域。商店域消费代理与产品域提供方是两次独立调用，要观察产品域内部服务，就在产品域配置技术绑定。配置按声明与阶段列表顺序执行，同一服务、阶段和技术服务重复匹配会被编译器拒绝。

`before` 可以阻止调用；`after` 失败会留下技术诊断并继续其他观察服务；`failure` 处理失败会附加到原错误并继续其他观察服务，不替换原错误。这样，日志设施故障不会在正常返回之后把原业务结果改写成新失败。

方法返回与事务提交也应分开观察。量子服务方法返回时，外层事务可能还没有结算；一个场景有多笔顺序事务时，一次场景 `after` 更不能替代每笔事务的提交证据。当前接口还没有独立提交回调，成交与恢复的判断需要相应执行记录。

技术服务本身不注册为 SPP 原子服务，因而不会被全服务匹配再次拦截成递归调用。这也保持了职责关系：业务服务完成业务，技术服务围绕调用提供设施。

限流与隔离保护运行资源

某个外部目录服务响应变慢，如果所有请求都长期等待它，连接和工作线程可能被占满，其他业务也受到影响。技术层可以为不同依赖设置容量限制、等待期限和独立资源池，减少故障传播。

限流控制单位时间的受理量，隔离控制不同调用使用的资源，熔断则在持续故障时暂时停止反复访问不可用依赖。这些策略可以提高系统应对故障和负载的能力，但不能把依赖不可用解释为商品不存在，也不能宣称它们自动保证整笔业务成功。

已经开始的更新还要确认真实结果。调用超时后释放页面等待，不意味着后台更新已经停止；技术层必须将执行状态与结果确认纳入协议。响应时间管理与交易恢复要配合，不能分别作出冲突的判断。

降级需要保持业务含义

商品推荐不可用时，页面可以省略推荐；展示图片读取失败时，可以显示占位图。此类调整可以由接口与展示适配承接。

但价格来源不可用时，不能随意改用旧价格完成成交；支付依赖故障时，也不能先把订单标为已支付。哪些内容可以省略、哪些事实允许使用缓存、哪些办理必须暂停，需要有明确的业务与接口约定。技术层选择降级方法，业务层确认允许的结果范围。

消息或异步执行也有同样边界。发送通知可以异步，但“订单成功”的承诺不能在库存与付款尚未确认时悄悄成立。若业务本来就是先受理、后办理，应在 SPP 中明确阶段和合约状态，技术队列负责运输与执行。

数据转换与环境适配促进复用

不同外部系统可能用不同字段名、报文格式和数值表示。技术适配负责把它们转换为约定的业务事实，再交给角色查询、计算或更新。日期格式、序列化、传输压缩等能力也可以集中维护。

例如，产品域使用另一个接口协议，商店仍通过商品角色取得事实；展示文案需要多种语言，常量与语言资源可以由相应机制承接。币种转换、成交舍入等会改变业务金额的规则，则不能仅当作报文格式转换处理。

开发、测试和生产环境的服务地址、数据源、缓存与密钥设施由 PHY 落实。技术能力具备以后，运维可以按环境配置资源；能力尚未实现时，仅增加配置名称并不能使其生效。

让技术演进具有可核对的结果

每项技术改进都需要明确目标与验证范围。增加缓存，应核对读取时效与失效行为；修改防重存储，应核对并发请求和重启恢复；调整事务部署，应核对提交、反交易与结果不明的处理；增加安全适配，应核对身份来源与输出保护。

这些验证围绕技术能力的约定展开，业务验证则继续检查计算、资格和合约结果。分层使技术升级与业务变化各有清楚的评审范围，双方通过共同约定确认系统能力。下一篇以 Petstore 串联角色、服务、场景以及相应技术接入，观察这些原则如何用于完整业务。

本篇目录 · 全书目录 · 上一章 · 下一章

第六篇 完整实践 Petstore

© 2026 S++ project contributors. All rights reserved. Book copyright.

前五篇已经建立业务模型、服务协作、语言表达、一致性与技术分离的原则。本篇沿一位买家的完整办理过程，把这些原则落实到 Petstore 的模型、场景、APP 与 PHY。先建立零事实，充值与入库，再报价、下单、付款、发货，最后观察后续办理和失败处置。

业务线使用一组可逐步核对的假设金额与数量；源码片段链接到实际文件。已有实现与验证范围据实说明，缓存、按需数据和完整安全治理等扩展能力以改造方案呈现。性能实验和详细论证仍放在附录。

先看与 Java 的代码量对照

这套 Petstore 覆盖四个业务域、34 个场景，包括会员、商品、购物车、订单、付款、交付、退货与预约。以本书使用的源码生成 Java，得到以下统计：

内容	文件数	代码行数
SPP 业务模型、服务与场景	63	999
业务枚举	2	18
APP 与 PHY 技术及物理配置	8	331
人工维护的 S++ 源码合计	73	1,348
工具生成的 Java	962	100,151

行数不计空行和注释，包含单独的括号行。这里约一千行的业务表达，加上三百多行配置，生成了约十万行 Java，涵盖模型、请求与响应、接口、服务实现和技术接入。业务人员主要阅读并维护 SPP，模型展开、调用与保存等代码由工具确定性生成。

采用 Spring Boot＋MyBatis＋Lombok 开发同样的商店，可以作一个粗略工程估算：

Java 开发范围	预计需要人工维护的代码与配置
34 个场景的常规单体实现，使用本地数据库事务	约 6,000～10,000 行
再覆盖本案例的多域调用、守恒交易、持久幂等与失败恢复职责	约 10,000～20,000 行

这里把 Java、SQL／Mapper 和必要配置一起计入，与 S++ 的业务及配置合计比较；不计框架依赖、测试、网页和自动展开的访问方法。Lombok、框架和通用组件已经承担一部分重复代码，估算没有假定 Java 开发者需要逐行重写生成结果。

由此可以建立一个大致印象：本案例约 1,350 行 S++ 业务与配置，对应常见 Java 框架方案约一万到两万行需要维护的项目源码。Java 范围是按职责拆分的估算，尚未通过独立实现测量；十万行则是实际生成量，两者不能混用。统计与估算依据见 附录 F7。

本篇目录

- 第28章 从需求识别角色行为与服务
- 第29章 零开户充值与入库
- 第30章 报价下单付款与发货
- 第31章 取消退货预约与失败处置
- 第32章 技术配置部署与综合验证

与前五篇的衔接

|本篇实践|应用的前文原则| |识别角色、合约与服务职责|第一篇的业务完整性与角色多态，第三篇的模型和服务表达| |安排查询、计算、校验与更新|第二篇的依赖与并行，第三篇的场景组织| |资金库存成交与失败处置|第一篇的量子服务，第四篇的事务、守恒与恢复| |存储、请求、安全、性能及部署|第三篇的分层，第四篇的执行保障，第五篇的技术能力|

阅读时依次核对：业务目标是否完整、每项服务的输入是否就绪、成交与保存是否一致、技术策略是否兑现这些要求。

返回全书目录

第28章 从需求识别角色行为与服务

© 2026 S++ project contributors. All rights reserved. Book copyright.

前五篇分别讲过角色、服务、执行顺序、语言、事务和技术能力。本篇把它们放进同一个商店：一位 VIP 买家充值 500 元，购买单价 60 元的商品两件，完成付款与发货，再观察取消、退货和预约失败怎样办理。数字是用于推演的假设输入，不是演示系统固定的初始化数据。

我们先确定业务要完成什么，再查看 SPP 怎样表达，最后在 APP 和 PHY 中安排技术与部署。这样，读者可以同时核对业务意义、源码职责与实际结果。

先写出完整业务活动

“买家购买商品”过于笼统。把它展开，可以得到几项具有明确结果的活动：

|业务活动|主要参与者|完成后应得到什么| |充值|买家、结算来源、充值合约|钱包增加，对侧来源等额减少| |下单|买家、购物车、商品、库存、订单|保存成交约定，预占库存，维护购物车| |付款|买家、商家、订单、付款合约|资金转移，订单记为已付款，生成回执| |发货|买家、订单、库存、发货合约|预留转为已售，保存发货信息| |申请退货|买家、订单、申请合约|记录申请理由与办理状态| |批准退货|买家、商家、订单、库存、退款合约|资金退回，库存回补，保存办理结果|

付款值得成为独立场景，是因为“扣买家钱包”只完成其中一部分。收款方、订单资格、成交金额、订单状态和回执共同决定付款是否完整。只从 Buyer 的方法出发，容易把这些协作散落在不同对象中；从付款活动出发，才能直接看到各个参与者承担的职责。这正是第一篇第1至2章所讲的建模侧重点。

区分实体、角色与合约

客户域的 Person 表达一个人，产品域的 Product 表达一个产品。商店使用 Buyer、Merchant 和 Goods，分别表达购物、经营和销售身份。库存域维护 StockLine 与供应来源，商店不接管产品和客户的所有事实。

Buyer 由 Person 扮演，Goods 由 Product 扮演。CartLine 由买家和商品共同扮演，数量属于这项购物关系。OrderLine 从 Goods 形成，保存成交名称、数量、单价和金额。ShopOrder 则是下单合约，记录买家、明细与成交约定；Receipt 是办理凭证实体。

“商品现在叫什么”和“这笔订单成交时叫什么”需要分别保存。例如商品成交时叫“宠物粮 A”，后来改名为“宠物粮 A 新包装”，历史订单仍显示原成交名称。当前 APP 用 `current(Goods.name)` 与 `snapshot(OrderLine.name, OrderLine.unitPrice)` 区分这两种保存方式，具体配置将在第六篇第32章展开。

会员号和订单号也不是单纯为了建表而添加的字段：顾客用它们识别会员和查询订单，因此源码用 `business` 说明其用途。存储主键、版本和技术防重键由技术层安排。第三篇第13至14章的模型与扮演关系，在这里都有具体用途。

用服务分类拆开场景

以付款为例，先按第一篇第4至7章的分类查看每一步：

|职责|Petstore 中的例子|承担的工作| |查询|findBuyer、findMerchant、findShopOrder|补全已存在的事实| |校验|checkOrderOwner、checkPaymentEligibility|判断归属和付款资格| |计算|preparePayment、preparePaymentAgreementReceipt|准备金额、状态与回执内容| |量子更新|payOrderFunds|在钱包与商家资金之间转移金额| |普通更新|updateShopOrder、insertReceipt|保存已准备好的订单与回执|

普通查询和普通更新由系统提供，业务开发者编写计算、校验与场景。更新内部不再查询金额或计算折扣；这些输入在场景中明确准备。于是，付款的业务规则集中在可阅读的流程中，技术保存机制可以单独实现。

看角色多态能够复用什么

报价规则读取 Buyer 的会员等级、Goods 的价格和 CartLine 的数量。它不需要知道产品最初从哪个数据库、供应商或外部系统取得。另一个参与者如果合法扮演 Goods，并提供报价所需事实，同一套计算和校验就有复用的基础。

复用仍需要满足角色的业务含义：同样有 `price` 字段，不代表可以随意替代销售商品。角色多态复用的是一项已经明确的业务职责。当前 Petstore 展示了具体扮演链，新增扮演者的复用还需要相应模型、映射和验证，不能仅凭字段相同就认定已经完成。

按业务与技术两条线阅读源码

先读 业务需求，再从 付款场景 沿服务名称找到校验和计算。对每一步都问：使用哪些事实，产生什么结果，后一步是否依赖它?

随后读 商店 APP 和 商店 PHY，查看存储、请求防重、事务策略与资源接入。业务目录按会员、商品、购物车、订单、付款、交付、退货和预约组织，目录位置不改变业务域身份，也不决定服务必须运行在哪台机器。

下一章先建立参与者与资源来源，再沿同一组金额和数量观察完整业务线。

本篇目录 · 全书目录 · 上一章 · 下一章

第29章 零开户充值与入库

> © 2026 S++ project contributors. All rights reserved. Book copyright.

购买之前，需要有买家、商店、商品、钱包和库存。本章把“建立参与资格”和“转入资金或货物”分开：开户不赠送余额，开店不凭空增加库存。这样，后续每次非零变化都能找到对应来源。

开户准备身份与零事实

新钱包为零，新商品库存的可售、预留与已售数量为零。结算来源与供应来源也从零开始。建立零事实不产生非零交易流水；开户重试不得把已经充值的钱包重新写成零。

这包含两类工作：业务上建立买家与商店身份，技术上准备普通记录和量子地址。分库零记录的建立通过持久技术待办确认与恢复，不要求业务人员在每个开户场景中编写数据库重试。第四篇第18章说明了这一生命周期，第五篇第26章说明了持久恢复的作用。

如果开户之后还没有充值，买家身份已经存在，钱包仍为零。这是可以直接解释的业务状态。

充值展示一对守恒属性

充值 500 元，Buyer.wallet 增加 500，SettlementAccount.balance 减少 500。假设双方之前都为零，成交后分别为 500 和 −500，合计仍为零。

负来源是演示边界中的对侧记载。实际支付接入还需要确认资金来源、支付结果与结算资格，不能把允许负来源理解成允许无条件赠送资金。

充值源码 中的量子定义直接表达这对属性：

```
quantum update transferWalletFunds(Buyer, SettlementAccount, WalletFundingAgreement) {
    SettlementAccount.balance -= WalletFundingAgreement.amount;
    Buyer.wallet += WalletFundingAgreement.amount;
}
```

服务的最小业务职责是完成这一对守恒变化。把它拆成两个互不相关的远程更新，会失去“一次充值”的完整含义。

查询准备事实，成交检查实际范围

充值场景的关键流程如下，片段取自同一源文件：

```
checkBuyerCredential; checkFundingAmount; prepareWalletFunding;
parallel { findBuyer; findSettlementAccount; }
identify(Buyer); identify(SettlementAccount);
transaction {
    transferWalletFunds bounds {
        SettlementAccount.balance: [null, null];
        Buyer.wallet: [0, null];
    };
}
```

先检查输入，计算准备来源关系，然后补全买家与结算账户。两项查询的输入已经就绪，彼此不依赖，因此可以并发。事务只处理准备好的资金转移，不再在更新内部查找充值金额。

查询得到的余额用于了解参与者，不能代替实际成交约束。查询后仍可能有其他充值或付款发生，量子服务按实际成交值检查范围。本例钱包下限为零、来源不设上下限；如果业务把其他上下限保存在钱包角色中，场景应查询这些属性，并将相应限制传给量子调用。

入库沿用相同的业务结构

某商品入库 10 件，StockSupply.balance 减少 10，StockLine.available 增加 10。库存变化与资金变化使用不同属性，却都需要来源、去向、数量和实际成交约束。

入库场景 先查询并确认商品，再准备库存和供应来源关系，并行查询双方，最后调用 `receiveInventory`。商店中的服务外观对接 库存域量子定义，业务调用不用混入库存数据库的地址和协议。

同一种结构不意味着所有服务都应合并成一个通用动作。充值与入库具有不同业务单位和资格，分别命名后，业务人员可以直接识别它们的含义。

先留下可核对的基线

继续购买之前，记录这一组假设结果：

|事实|开户后|充值及入库后| |买家钱包|0|500| |结算来源|0|−500| |可售／预留／已售|0／0／0|10／0／0| |供应来源|0|−10|

检查失败也要具体：负金额应被校验拒绝，身份不存在不能成交，原请求重放不能再入账。页面防重和远程请求幂等由技术策略承担，不改变充值金额必须为正的业务规则。具体配置留到第六篇第32章统一讨论。

本篇目录 · 全书目录 · 上一章 · 下一章

第30章 报价下单付款与发货

© 2026 S++ project contributors. All rights reserved. Book copyright.

沿用第六篇第29章的基线：买家钱包 500 元，可售库存 10 件。买家是 VIP，选择单价 60 元的商品两件。本章把同一订单拆成报价、下单、付款和发货，观察每项完整活动如何组织不同类别的服务。

报价计算候选约定

小计为 120 元，VIP 折扣为 12 元，折后 108 元达到免运费条件，订单总额为 108 元。计算规则见 订单计算。

报价使用查询得到的商品价格、购物车数量和会员等级。计算形成候选订单，不直接保存成交。明细保留名称、数量、单价与金额，订单保留折扣、运费和总额。商品价格虽可量化，却是报价依据；价格调整与实际资金转移承担不同职责，不能因为字段都是数字就都定义为量子服务。

如果商品后来调价，已经保存的成交明细继续使用原价。业务定义说明需要保留成交约定，APP 的快照映射负责让保存和查询兑现这一要求。

下单把约定与库存预占联系起来

下单场景 的流程可以按依赖分成四段：

1. 准备商品选择，并行查询买家与商品目录。
2. 定位并查询购物车，检查内容，计算订单报价。
3. 准备库存参与者，查询库存，检查预占资格，准备订单和购物车变化。
4. 在事务边界内保存订单明细、订单、购物车和回执，并完成库存预占。

第一段的两项查询互不依赖。库存查询却需要先由订单准备商品关系，不能与最初的所有查询随意放进同一个并行组。这对应第二篇第9章所讲的输入依赖。

预占将两件从可售转为预留，结果是可售 8、预留 2、已售 0。供应来源仍为 −10，该商品合计仍为零。事前“库存够不够”的判断不能阻止其他订单同时下单，因此可售下限还必须在量子成交时检查。

源码中既有量子更新，又有普通保存。编译器根据准备依赖安排量子成交和后续更新。当前下单在事务内还有 `preparePurchaseReceipt`，因此不能把事务中的每一行都当作可以同时执行的普通更新；计算结果必须先准备，依赖它的回执保存才能执行。普通更新内部不会补查或计算这些内容，串行依赖仍在场景中可见。

付款明确金额与参与者

付款不再查询购物车，也不重新计算商品折扣。它读取已经形成的订单，确认买家归属与付款资格，准备 108 元付款约定和回执，然后执行资金转移与普通保存。

付款源码 中的事务片段如下：

```
transaction {
    transaction REQUIRED {
        payOrderFunds bounds {
            Buyer.wallet: [0, null]; Merchant.cash: [0, null];
        };
    }
    updateShopOrder;
    insertReceipt(PaymentAgreement.receipt);
}
```

钱包减少 108，商家资金增加 108，钱包变为 392。量子服务处理守恒变化，普通更新保存订单与回执。订单更新和回执插入使用已经准备的输入，可以由支持该计划的技术配置安排并发执行。

内层 `REQUIRED` 加入外层逻辑边界，不代表内层结束就完成一笔独立付款。实际资源共同提交还是分阶段成交与恢复，需要看 APP 和 PHY。不能从这段 SPP 单独推断两个数据库具有同一物理事务。

发货沿另一对库存属性成交

发货场景 查询买家与订单，再准备库存并查询，检查订单状态与预留数量，准备发货约定和回执。

量子服务把两件从预留转为已售，结果是可售 8、预留 0、已售 2。普通更新保存发货状态、运单信息与回执。付款维护资金，发货维护库存，两项活动使用不同守恒属性，也有不同完成条件。

逐阶段核对同一订单

|阶段|钱包|商家资金变化|可售／预留／已售|需要核对的普通事实| |下单后|500|0|8／2／0|订单明细、108 元总额、未付款状态| |付款后|392|+108|8／2／0|已付款状态、付款回执| |发货后|392|+108|8／0／2|发货状态、运单与回执|

这张表描述假设输入下的业务结果，不能替代运行验证。如果付款超时而结果未知，表中的“付款后”还不能直接显示为失败或成功；必须先由 APP 确认实际成交与保存结果。下一章讨论如何区别失败恢复与正常后续办理。

本篇目录 · 全书目录 · 上一章 · 下一章

第31章 取消退货预约与失败处置

© 2026 S++ project contributors. All rights reserved. Book copyright.

订单已经完成到哪一步，决定后续办理需要哪些服务。取消、退货是新的业务活动；执行失败后的回滚和反交易，是恢复原活动的一致性。这两类过程都可能产生反向变化，却具有不同业务含义。

取消与退货按照已发生事实办理

以下各行是不同分支，不能连续套用到同一订单：

|办理起点|业务活动|资金变化|库存变化| |已经下单，尚未付款|取消未付订单|无|预留两件转回可售| |已经付款，尚未发货|取消已付订单|商家退回 108 元给买家|预留两件转回可售| |已经交付，符合退货政策|申请退货|无|无，记录申请与理由| |退货申请满足批准资格|批准退货|商家退回约定退款金额|对应已售数量转回可售|

申请退货 使用买家、原订单和申请合约即可表达受理范围。批准退货还需要资金与库存参与，源码见 批准场景。如果商店要求先收到退回商品，业务模型还应加入收货事实与相应资格，演示规则不能代替所有商店政策。

业务退款使用已经确认的金额和数量。不能直接恢复旧钱包和旧库存快照，因为付款之后可能已经发生其他业务。

业务退款与自动反交易的区别

一笔成功付款后来被取消，退款是新业务，具有取消资格、退款合约和办理回执。一笔付款在执行过程中失败，需要自动撤销已完成的量子变化，则是原执行的恢复；业务开发者不用再编写一套量子反交易服务。

普通更新的恢复也要保护后续版本。根据第四篇第21章的契约，恢复必须确认当前版本仍符合撤销条件；存在新版本时不能直接覆盖。当前实现中的普通更新恢复仍需相应补偿能力，不能因为保存逻辑简单，就声称所有部署已经自动完成版本条件反交易。

如果付款跨普通库与量子库，量子资金已成交而订单保存失败，APP 应组织已完成阶段的恢复。恢复全部确认后，才可以认定原付款已撤销；部分恢复失败，则保留待恢复状态。SPP 不应接管远程结果查询和数据库技术重试。

预约有意保留已经受理的申请

预约场景 使用两个顺序事务。第一个保存受理合约和预约号，第二个执行库存预占并保存确认状态。它允许受理完成后，预占因为库存竞争而失败。

假设可售库存为 10，申请预约 11 件。申请首先被受理，量子预占因下限要求不能成交。系统确认原预占已回滚或反交易之后，业务流程将申请记为拒绝，预约号继续供顾客查询。

与第三篇第17章的流程组织、第四篇第19章的失败处置对应，第二个边界的真实源码如下：

```
transaction {
    reserveRequestedInventory bounds {
        StockLine[].available: [0, null];
        StockLine[].reserved: [0, null];
    };
    confirmPetReservation; updatePetReservationAgreement;
} fail {
    if (ResultInfo.failure.recoveryStatus == "ROLLED_BACK" ||
        ResultInfo.failure.recoveryStatus == "REVERSED") {
        findStockLine; identify(StockLine[]);
        rejectPetReservation;
        transaction { updatePetReservationAgreement; }
    }
}
```

`fail` 中的重新查询服务于业务判断：当前库存不足，拒绝原因是库存不足；库存仍够但预约未完成，则采用另一种业务原因。原因定义及计算见 预约模型。这不是让业务人员捕获数据库异常，而是在已确认恢复的前提下，明确处理已经受理的申请。

三类失败走不同的路径

|情况|资源与业务状态|处理方式| |输入数量不合法|尚未成交|校验拒绝，不进入库存交易| |预占失败且恢复已确认|受理记录保留，原预占已经撤销|执行 fail 中的业务处置，保存拒绝原因| |预占或恢复结果不明|不能确认资源是否变化|由 APP 确认和恢复，不能直接认定业务拒绝|

技术层可以把已经确定、具有业务含义的失败交给业务层处理。超时不能自动等同库存不足，更不能让 `fail` 再预占一次。当前例子的恢复状态判断正是为了避免这一误判。

如果业务处置本身保存失败，申请也不能假装已经持久化为拒绝。需要保留执行证据，继续确认与恢复。`ResultInfo` 表达本次执行情况，持久预约状态和拒绝原因表达业务办理结果，两者用途不同。

用故障验证完成条件

至少检查库存不足、量子成交后普通保存失败和结果未知三种情况。前两类应分别核对库存、申请状态与拒绝原因；未知情况应核对技术恢复记录，确认没有因为重复请求再次预占。

当前预约验证及其范围见 失败处置说明。Petstore 的 LOCAL 用例不能单独证明真实跨域资源的全部恢复能力，技术资源与测试证据还要在下一章一起核对。

本篇目录 · 全书目录 · 上一章 · 下一章

第32章 技术配置部署与综合验证

© 2026 S++ project contributors. All rights reserved. Book copyright.

前四章已经表达了业务参与者、报价、成交与失败处置。本章沿同一组场景查看技术配置：哪些能力已在 Petstore 中提供，哪些能力可以继续加入，部署改变后如何验证。第五篇讨论的业务与技术分离，在这里应当能落实到文件和结果。

先把三个层次放在一起

|层次|付款业务中的内容|主要文件| |SPP|付款资格、金额、资金转移、订单和回执|付款场景及模型、计算、校验| |APP|存储映射、请求策略、逻辑绑定、事务及环境选择|petshop.app| |PHY|数据库、量子存储、协议路由、超时和适配器|petshop.phy|

业务人员检查“谁付款、付多少、何时允许、失败后怎样办理”，技术人员检查保存和调用策略，运维人员检查实际资源与环境。AI 可以帮助修改各层，但业务人员不需要通过审查生成的全部 Java、数据库驱动和部署代码，才能理解付款规则。仍需用运行证据确认工具和技术适配兑现了这些规则。

以下片段分别来自 商店 APP 与 商店 PHY。

用存储映射保留成交事实

APP 中的明细表包含：

```
table SHOP_ORDER_LINES(OrderLine) {
    primarykey(OrderLine.lineNumber);
    auto OrderLine.lineNumber prefix "ORDER-LINE";
    foreignkey(OrderLine.goods) references Goods;
    snapshot(OrderLine.name, OrderLine.unitPrice);
}
```

商品名称另有 `current(Goods.name)` 配置。验证时，先保存一笔订单，再修改产品名称和商品价格，分别查询当前商品与旧订单。前者反映当前事实，后者保留成交名称和单价。这一实验同时检查第三篇的扮演映射与第四篇的存储归属。

用请求策略保障重复办理

当前 APP 有以下全局策略和付款入口配置，分别位于 `requests` 与 `scope local` 中：

```
requests { idempotency retention 3 days; duplicateWindow 5 seconds; }
```

```
duplicate key(Buyer.membershipNumber, ShopOrder.orderNumber)
    facts(ShopOrder.orderNumber) payShopOrder;
```

PHY 使用 `config requestIdempotencyAdapter = "jdbc";`。页面防重处理短时间重复操作，远程更新幂等处理同一调用的重复传输。付款资格仍由 SPP 检查，不能把防重窗口当成付款业务状态。

验证至少分为三项：短时间重复点击不产生第二笔付款；远程更新以相同身份与内容重放，返回原结果而不重复成交；相同请求身份改变金额或参与者，应被判定为协议冲突。本地调用不额外登记远程请求，不能用一次 LOCAL 调用测试代替远程幂等验证。协议说明见 调用幂等与界面防重。

持久适配器还需要核对实际数据源及重启后的记录。声明保留三天并不意味着结果未知的请求到期后可以直接当成新付款，未完成恢复的记录需要相应处理。

比较集中部署与拆分部署

develop 的 APP 将客户、产品、库存和商店四个域部署在同一进程，并声明：

```
transactions {
    mode: LOCAL;
    resource: shopRecordsResource;
    ordinaryUpdates: PARALLEL;
}
```

PHY 将普通记录和量子存储接到对应本地资源。已经准备好、互不依赖的普通更新由该配置安排并发；输入尚未准备好的步骤仍按场景依赖执行。

testing 将客户、产品拆到独立进程，库存与商店同进程，普通库与量子库也分别配置。PHY 提供客户、产品的远程路由与超时，APP 为量子服务选择统一存储，普通更新资源使用相应恢复策略。

业务域划分仍相同，数据库和调用路径发生改变。LOCAL 的共同回滚不能直接推定为跨资源共同回滚；量子先成交而普通更新后失败时，需要依照第四篇第21章确认阶段结果并恢复。配置改变没有免除业务一致性要求。

仓库已有网页流程、重启守恒核对及失败处置记录，具体版本与证据见附录 F。它们证明所记录路径，不能自动证明所有故障组合，也不是性能基准。

将性能优化应用到具体查询

在商品浏览中，可以缓存产品名称、说明和展示数据，避免每次沿扮演链重复读取同一事实。查询范围也可以按场景实际使用的属性收集：报价需要价格和资格，商品列表需要展示字段，不应每次递归取回全部参与者属性。

在订单列表中，页面通常只需要订单号、状态与总额，点击详情时才需要明细。按需查询减少后端读取，按需返回减少传输，两项优化要分别测量。部分加载的对象不能被当成完整事实覆盖保存。

这些是结合第五篇第24章提出的改造方案，当前案例尚未证明已经具有完整的多级缓存、编译期属性裁剪和通用返回数据集配置。实施时应补充缓存键、更新失效、权限范围和未加载属性的保存约束，而不是只在场景里删掉查询。

钱包余额和可售库存即使用于页面展示缓存，实际付款与预占也必须按真实成交范围检查。技术优化不能把“缓存显示有两件”变成“这次必然能买两件”。

从现有日志样例扩展技术服务

Petstore 已增加 技术服务 APP 和 技术服务 PHY。APP 用 `intercept services` 为商店域服务统一安排 `invocationTrace`，PHY 绑定通用 Java 日志实现，记录调用名称、阶段与耗时，业务 SPP 没有增加日志调用。

业务人员仍检查付款金额、订单状态和预约拒绝原因，技术人员通过日志检查实际调用。预约预占失败经过 `fail` 处置后，场景正常返回会记录 `AFTER`；这表示调用正常结束，并不表示预约成功。日志也不打印客户和订单内容。

沿第五篇第27章的接口，可以新增 Java 技术服务并通过 APP／PHY 接入指标、告警或技术访问控制。查询缓存和输出脱敏则使用相应数据适配。样例及验证范围见 Petstore 技术服务。

将安全与运行治理应用到办理入口

当前 testing PHY 已声明域间调用令牌配置，但单个令牌不等于完整的用户认证、资源授权和数据保护。生产改造还应围绕付款和订单入口安排以下能力：

|能力|Petstore 中的应用位置|需要验证的结果| |认证与访问控制|页面与域间入口|调用方身份可信，不能代其他买家操作| |加解密与脱敏|通信、地址等敏感事实、日志与返回数据|未经授权不能读取敏感内容| |请求内容指纹|技术请求登记|同一请求身份更换内容能被识别| |追踪与指标|查询、成交、保存和恢复阶段|能定位耗时及确认各阶段结果| |限流、隔离与熔断|浏览和更新入口、远程资源|故障不会无限扩散，已受理更新仍可确认|

表中是技术设计与验证任务，不代表这些能力都已经实现。请求内容指纹不能替代身份认证，业务归属校验也不能替代调用方认证；第五篇第25章解释了两者的区别。

浏览页面的推荐或图片可以按策略降级，付款不能在超时后返回虚假的成功或拒绝。追踪信息应关联原请求、量子成交、普通保存与恢复证据，让技术人员能确认结果，而不要求业务人员在 SPP 中编写网络异常捕获。

最后形成一份综合验证记录

先用第六篇第29至30章的数值表核对完整业务，再选择取消、退货或预约分支。检查订单状态、回执、资金和库存守恒，以及重复调用、版本竞争、库存不足和结果未知的处理。

性能只保留必要对照：在独立实验副本中，对互不依赖的查询分别使用串行和并行，保持业务规则与部署资源一致；记录端到端延迟、分支重叠、吞吐、错误比例和资源占用。缓存或按需数据若实际加入，则另设对照，避免把几种优化的效果混在一起。实验方法放在附录 C，不在正文重复推导。

报告同时注明源码版本、APP 策略、PHY 资源和覆盖范围。业务正确、技术配置符合预期、部署后结果可确认，三项一起成立，才完成本篇的实践闭环。

本篇目录 · 全书目录 · 上一章 · 下一章

第七篇 进阶案例与架构方法

© 2026 S++ project contributors. All rights reserved. Book copyright.

用银行存款与冲正展示复杂规则，随后讨论系统迁移、设计评审与复杂需求的表达能力。区分面向角色的建模方法、商业服务架构和当前语言实现，并用哈希表说明简单概念怎样构成复杂算法。把案例中的参与身份、行为、事实归属与执行证据提炼为可复用的方法。

本篇目录

- 第33章 银行存款偿还分配与冲正
- 第34章 从现有面向对象系统迁移
- 第35章 架构设计评审与能力边界

返回全书目录

第33章 银行存款偿还分配与冲正

© 2026 S++ project contributors. All rights reserved. Book copyright.

某银行的存款业务包含受理资格、授权、偿还分配、计息、账务与冲正。它适合检验角色与服务架构能否表达复杂规则，同时保持业务与技术边界清晰。

本章采用仓库中的某银行存款案例。业务由 SPP、APP、PHY 表达，生产 Java 由工具生成；测试只准备输入、调用生成服务、断言及注入技术故障。案例规则与演示配置有各自的来源，不能直接视为原银行系统的全部政策。

不同参与事实

账户、债务、产品政策、业务日期、授权事实、资金来源、处理计划和原交易共同进入业务。账户余额、计息金额、欠息、欠款本金与透支敞口需要分别解释，不能因技术上都表现为金额便混为同一个字段。

前台现金受理与核心计算还可能有不同金额条件。前台要求正金额，并不意味着核心零金额偿还路径应被删除。服务契约需要保留调用层次的业务差别。

当前案例按账户、资格、计算、手续费、积数、分录、存入与冲正划分能力目录。每个场景独立保存，场景直接组织查询、校验、计算与更新，避免用一个总场景隐藏全部步骤。源文件导航提供各项能力的查阅入口。

偿还分配与真实资金去向

普通债务与不良债务可以具有不同的本金、利息偿还顺序。业务计算先形成明确分量，再由场景组织账户变化和分录。资格检查与授权判断在场景中可见，查询与存储由 APP、PHY 接入。

例如，现金存入 100 元，其中 10 元偿还利息，90 元增加客户余额。真实资金变化为：

```
现金结算账户        −100
客户账户              +90
银行利息收入账户     +10
资金变化合计           0

客户欠息另减少 10，这是债务清偿事实。
```

只把“客户欠息减少 10”当作资金去向是不完整的：债务减少说明客户少欠了多少钱，却没有说明银行实际收到的钱在哪里。因此，案例明确引入利息收入、催收利息收入和本金回收三个内部账户。偿还计算决定各分量，量子成交按约定更新真实收款方，合约记录实际账户及金额，分录提供对应账务证据。

欠款与敞口仍按案例的量子属性约定处理，但不能再重复计入上述资金守恒组。资金守恒与债务变化各自表达自己的含义。当前 APP 指定资金守恒组，运行时仍记录其他量子属性的实际变化、范围与反交易信息。

分量需要保留足够精度。三位利息可以按确认的业务数学规则计算，物理列不能静默截断；需要舍入时，由业务明确舍入政策。演示参数与从来源追踪到的规则也需要分别标明。

查询与计算先准备，事务内完成成交

现金存入场景先并行查询客户账户、资格、资金对方和积数事实，再校验归属、资格与授权，计算费用、偿还分量、分录及积数。需要共同成立的更新放在明确的事务边界中。

开户只建立零余额，存入才形成真实资金变化。银行内部收款账户也要先建立，不能在第一次还息时凭空生成非零资金。

事务内先执行量子成交，其后积数与办理信息的计算形成普通保存所需事实，再保存账户信息、存入合约和分录。这些计算有前后依赖，由串行流程保留；可并行的普通更新仍受其准备条件约束。场景声明完整业务边界，APP、PHY 决定怎样落实到实际资源。

授权校验与授权续办

待授权是一种业务结果，不应统一当成技术异常。当前案例支持资格和授权评估；待授权申请的持久化、授权域归属及跨进程续办仍有需要明确的范围。

完整续办应在授权事实到达后重新查询和校验可能变化的资格，再根据当前事实执行。已有授权判断不能代替完整续办流程。这项区别记录在案例的已知问题中。

原交易驱动冲正

现金冲正场景查询原合约、实际分录与当前账户事实，校验原交易归属、收款账户、冲正资格及分录证据，再按原成交分量形成反向变化。当前政策或贷款状态变化，不应导致重新分配原交易金额。

例如，先存入 100 元，后来又存入 5 元，撤销第一笔应保留后来的 5 元。冲正不能把整个账户恢复成第一笔之前的快照。同样，原交易的手续费及收款内部户已经记录在成交约定中，不能按当前费率或当前账户路由重新选择。

合约保存成交分录的快照，冲正前将实际分录与该证据比较。科目、金额或记录被破坏时，应明确拒绝，不能只看总额平衡就继续办理。跨日、Mode2、积数和当前资格也有自己的检查与计算。

案例另外提供只读冲正试算：执行相同的事前查询、校验和候选计算，但不成交、不保存。展示八条候选分录，不意味着账务已经变化；正式冲正仍需重新核对当前事实。

同一失败在不同部署下怎样恢复

develop 环境让相关更新共用本地事务资源。冲正分录保存失败时，量子与普通变化共同回滚，原交易仍未冲正，可以在确认结果后重新办理。

testing 环境把量子库与普通记录分开。量子成交后，普通 Saga 阶段可能已经部分提交；随后分录保存失败，系统可以反交易量子资金，已提交的普通事实却未必已经恢复。当前案例明确保留 `RECOVERY_REQUIRED` 和 `handled=false`，不能把资金恢复当作整笔业务已经撤销。

这是第四篇第21章所述普通恢复能力的实际边界。按版本保护实现普通自动冲正的架构方向，并不意味着当前所有仓储与部署都已经具备该能力。案例的 `fail` 保存失败处置记录，只有系统确认 `ROLLED_BACK` 或 `REVERSED` 时才记录本次资金变化未完成；待核对记录不能替代真正恢复。

处置记录保留当时的合约与系统失败信息。后来查询这条记录，应仍能看到原失败原因、阶段和恢复状态，不能被当前账户余额或当前关联事实覆盖。即使保存处置记录再次失败，也要保留原交易失败和真实恢复状态。

从来源规模看业务表达

以 2026 年 10 月 14 日的源码统计为例，案例文档中有具体文件或行号定位的来源包括 9 个 COBOL 程序。它们直接引用 135 个 COPYBOOK，继续追踪嵌套引用后，共有 161 个去重的 COPYBOOK。

内容	文件数	物理行数，含空行与注释	去除空行与注释后的行数
直接定位分析的 COBOL 程序	9	14,156	11,410
对应 COPYBOOK，含嵌套引用	161	23,488	11,179
原始来源合计	170	37,644	22,589
当前 SPP 业务代码及枚举	35	1,043	979
当前 APP、PHY 配置	4	86	86
当前 S++ 源码合计	39	1,129	1,065

如果扩大到文档提及的全部相关程序，来源范围为 22 个 COBOL 程序、192 个去重 COPYBOOK，共 45,587 个物理行，去除空行与注释后为 28,358 行。这一范围还包括调度、数据访问、账务配置及未迁移分支。

这些数字说明业务表达与原始实现的维护规模，但两边的覆盖范围不同：原始统计包含来源文件的全部内容，案例迁移的是其中选定的业务路径；当前 S++ 统计不含测试、网页、生成 Java 和公共框架。因此，不能把它表述为三万多行 COBOL 已由一千多行 S++ 完整等价替代，也不能只据行数推导开发效率倍数。

S++ 将已选规则集中在业务模型、计算、校验和场景中，将重复的数据访问与执行设施交给工具。需要阅读和维护的业务表达因此更集中，工具和框架仍承担实际实现责任。逐项统计、来源清单与复现方法见附录 F 的 F9 节。

从网页办理到验证证据

中文存款工作台可以完成开户、现金存入、冲正试算、正式冲正与账务查询。页面负责提交参与者、展示返回结果，不计算费用或偿还分量。通信中断时显示结果未确认，不自动重复发起更新，也不把输入中的余额与候选分录当作已成交事实。

案例已有真实 JDBC 存储、规则向量、两环境失败路径、并发竞争、请求幂等、独立进程重启及浏览器流程的验证。各批次结果、配置和限制见最新验证记录与附录 F。普通 Saga 恢复、完整授权续办及未知成交自动查证等未完成范围，仍按完整性审计分别说明。

复杂业务的关键，是把规则、参与事实、处理计划、成交与恢复分别表达，并用场景连接起来。已有验证支持明确的业务路径；原 COBOL 实测和银行确认结果仍属于不同证据层级，不能只用测试数量替代业务等价判断。

本篇目录 · 全书目录 · 上一章 · 下一章

第34章 从现有面向对象系统迁移

© 2026 S++ project contributors. All rights reserved. Book copyright.

迁移可以从一条完整业务开始。先理解现有系统实际承诺的结果，再识别参与者、角色和行为，使新模型能够与旧系统进行明确对照。

选择可验证的业务线

选择具有清楚输入、持久结果和失败规则的活动，例如钱包充值或订单付款。既保留正常路径，也保留拒绝、重复请求与竞争情况。已有数据库和外部接口可以继续作为技术承载。

迁移的单位应当包含完成条件。如果只迁移一个公式，却没有接入实际成交，就应把结果表述为计算迁移，而不能宣布整个交易已经替换。

从类识别参与身份

读取需求与现有调用，把 User、Product 等类型在不同场景中的责任拆开。寻找买家、付款方、账户所有者和履约参与者，再明确谁扮演角色、哪些事实借用、哪些事实自有。

现有 DTO 可以提供字段线索，但不能直接换成 Role。字段的业务用途、维护者和生命周期需要重新判断，技术请求键与平台版本移到应用配置。

把隐藏协作移到场景

将数据库查询、资格、业务计算和持久化职责识别出来，建立清晰原子契约。把完整活动的顺序和边界放到场景，让原来分散的隐含调用可以被审查。

纯计算可优先迁移并做同输入对照，随后接入查询、身份与成交。对照实现要有独立预期或历史业务证据，不能仅把同一公式复制一遍就称为正确性证明。

保持部署可控

最初可以把新服务与既有适配器放在同一个进程中运行，减少同时改变模型、协议和资源带来的变量。业务边界清楚之后，再根据容量与团队责任决定远程部署。

角色表、明细和快照结构可能改变，因此数据迁移需要独立设计。旧记录能解码不代表身份与事实归属仍正确。切换期间也要明确谁是事实维护入口，避免双重写入造成冲突。

用证据推进边界

每一步保留源码版本、业务样例、持久结果与失败记录。生成成功、局部计算通过、真实成交通过和跨进程恢复通过分别代表不同进度。

迁移完成的判断应回到选定业务的验收条件。模型清晰、行为正确、资源保障和运维恢复都达到约定之后，新的角色与服务边界才成为可持续维护的系统结构。

本篇目录 · 全书目录 · 上一章 · 下一章

第35章 架构设计评审与能力边界

> © 2026 S++ project contributors. All rights reserved. Book copyright.

角色与服务架构的评审应围绕业务意义、组合关系和执行保障展开。语言声明帮助发现问题，设计者仍需要判断概念是否真实、规则是否完整，以及实现是否兑现了约定。

建模方法与架构思想是两个层次

“面向角色的服务架构”包含两个层次：面向角色是一种建模方法，服务架构是一种组织系统的架构思想。为了对照，可以把以对象为建模基础的服务架构称为“面向对象的服务架构”。这两个名称帮助我们分别讨论建模的表达能力和架构的适用领域；面向对象本身也可以用于其他架构。

本书讨论的服务架构面向商业服务领域。购买、付款、预约、发货都有明确的参与身份、办理规则和完成结果，适合通过角色、服务和场景表达。S++ 将这类活动共有的查询、更新、守恒成交与技术接入交给工具，使业务开发者可以用很少的声明表达一项业务。代码简短来自领域中的共同结构和工具提供的能力，复杂规则本身仍需说明。

自然科学中的流体模拟、天体运动或数值求解，关注的主要问题可能是方程、离散方法和误差。将这些问题全部组织为商业服务，通常不方便，也不能期待几句业务声明就替代数值算法。科学计算系统的任务提交、资源安排和结果交付可以采用服务架构，内部求解算法则可以采用适合自身领域的表达。

因此，判断“能否满足任意复杂度需求”，首先要问：是在讨论一种建模方法、一套领域架构，还是当前语言与工具已经支持的功能？这三个问题需要分别回答。

面向角色也可以由简单概念构成复杂模型

面向对象从实体出发，把世界看作许多具有状态的对象，再赋予对象动作，通过对象之间的交互描述完整过程。面向角色从行为出发，把世界看作许多活动，再识别参与这些活动的角色，通过行为的组合描述完整过程。场景在这里可以看作一种广义行为：它组织若干行为，共同完成更大的目标。

两种方法都需要表达状态、变化和组合，区别在于建模的起点。面向角色没有取消实体或数据结构；它把这些实体与结构放到所参与的行为中理解。一个复杂过程可以逐层展开为子行为，参与者也可以由更简单的结构组成。只要每一层的状态、规则与组合关系都有明确表达，就可以继续构造复杂模型。

例如，描述一次棋局，可以先建立棋盘和棋子的对象，再提供走子方法；也可以先定义走子行为，识别执棋方、待移动棋子和棋盘这些角色，再组织轮流走子、判定局面与结束棋局。棋盘仍然保存局面，棋子仍然有位置，只是行为成为组织模型的主线。

这说明面向角色的建模方法并不天然局限于商业活动。对于可以由状态与可计算规则描述的过程，两种方法都可以通过组合继续扩展。模型能表达一个过程，与能简短表达、能高效执行是不同的问题。表达能力的构造论证及其条件放在附录 A 的 A8、A9 节。

用哈希表比较两种建模起点

哈希表可以直观说明：底层概念简单，并不妨碍它们组合出复杂结构和算法。

面向对象通常先定义一个哈希表对象。对象内部保存一张稀疏的存储表，提供 `put(key, value)` 与 `get(key)` 方法。方法负责计算存储位置、处理冲突、保存或查找记录。我们首先看到的是“哈希表是什么，以及它能做什么”。

面向角色可以先定义“哈希保存”行为，识别三个参与角色：作为查找依据的键、作为待保存内容的值，以及作为存储载体的稀疏表。具体的字符串或编号可以扮演键，某条记录可以扮演值，具体存储结构可以扮演稀疏表。行为说明它们怎样共同完成保存：

```
哈希保存（键、值、稀疏表）
1．根据键及表容量，计算候选存储位置。
2．在该位置检查是否已有相同的键。
3．已有相同键，替换对应值。
4．存在其他键，按约定的冲突处理规则继续寻找位置。
5．找到可用位置，保存键和值。
```

这是一段建模示意，不是当前 SPP 的可执行语法。冲突处理规则可以选择开放寻址；需要扩容时，再定义分配新表、迁移记录和切换存储的行为。复杂性没有消失，而是由几个明确行为共同承担。

“哈希查找”以键和稀疏表为参与角色，按照相同的哈希与冲突处理规则寻找记录，返回找到的值或未找到的结果。保存和查找必须共享这些规则，否则保存成功的值也可能无法找到。

例如，容量为 8，整数键按除以 8 的余数定位。保存键 10 时，先检查位置 2；保存键 18 时，也会检查位置 2。如果那里已经保存键 10，采用向后寻找空位的规则，就把键 18 保存在位置 3。查找键 18 时同样从位置 2 开始，比较键后继续到位置 3，才能取得正确的值。对象方法和角色行为都能表达这套过程。

这里的键和值是算法中的参与角色，不要求把它们声明为 SPP 的业务 Role，也不要求把每次探测都部署为远程服务。这个例子讨论的是面向角色作为底层建模方法的表达能力。第三篇的业务角色、原子服务和场景，是这套方法在商业服务领域中的具体约束与用法。

利用现有对象实现承接复杂算法

如果需要，可以围绕面向角色继续构造一门通用语言，为通用数据结构、算法组合和执行方式提供更多表达。当前 S++ 主要服务于商业服务领域，没有必要为了证明建模能力，就把全部通用计算设施重新实现一遍。

面向对象已经积累了丰富的数据结构与算法实现。角色建模可以继续使用这些成果：业务层说明所需能力与结果，APP 声明相应绑定或适配关系，PHY 落实具体 Java 实现和运行资源。一个 Java 复杂对象可以在内部使用哈希表、图结构或数值算法，向业务侧提供明确的能力契约。

例如，配送报价需要根据已准备好的地点和路线网络计算距离。业务层应说明距离如何参与计价，技术适配可以复用 Java 中的路线搜索实现。换用另一种搜索算法，只要仍满足约定的输入、距离含义和精度，就不必改变配送报价的业务规则。会员折扣、能否配送与费用承担等业务决定，仍留在 SPP 中。

这种接入应当通过相应的计算或数据适配接口完成。第五篇介绍的 `before`、`after`、`failure` 拦截器只接收调用元信息，不能用它来替代路线计算或改写业务参与者。这里说明 APP 绑定复杂实现的架构位置，具体绑定形式要以实现提供的接口为准。

这样，复杂需求可以分别落在明确的业务模型和可复用的技术实现中。业务复杂并不要求把算法、数据结构和部署细节全部写进业务语言；需要评审的是分工是否清楚、组合是否完整，以及每一层是否兑现自己的契约。

检查模型意义

每个角色应当能说明扮演者、参与行为与自身事实。每份合约应对应明确活动、当事人与约定。每个字段应当说明维护者与用途，业务凭证和技术身份各有归属。

若新角色只是为了包装列表，或合约只是通用返回容器，应重新寻找真实业务关系。若同一属性被多个域独立维护，应检查所有权是否明确。

检查服务组合

每项服务需要清楚的职责，场景需要可理解的依赖与完成条件。parallel 之前检查输入、读写冲突和失败汇合，事务之前确认范围和资源，分阶段之后确认恢复责任。

普通 check 失败不自动回滚，REQUIRED 不额外提交，放在同一个进程中运行不改变域归属，远程调用失败也不等于交易未发生。这些判断直接影响业务结果，应在评审中得到具体说明。

检查收益与代价

清晰角色和声明式组合可以降低业务与技术混杂，也会把映射、元数据和执行计划的责任集中到生成器与运行时。团队需要能够维护这些能力，并理解生成结果。

如果业务十分简单，新增分层和工具链可能带来额外成本。如果业务存在多参与方协作、复杂规则、资源守恒和多环境部署，显式契约更容易形成可复查的设计依据。是否采用需要结合实际团队与系统条件判断。

检查证据层级

语法解析证明表达被接受，编译证明产物在相应层次成立，运行测试证明所测路径，故障与重启测试进一步覆盖特定恢复条件。性能报告还需要明确工作负载和资源。

把这些层级分开，可以避免用一次构建成功支撑所有可靠性结论。已知限制也应随实现版本维护，使设计承诺与实际能力一致。

保持架构可解释

一笔业务应当能够沿参与身份、行为、合约、事实归属与执行结果被解释。故障时能定位未完成阶段，变化时能找到责任边界，优化时能识别关键路径。

本书从参与者与行为出发，经由角色和服务形成场景，再通过应用与物理契约走向运行系统。这条联系也是持续评审和演进架构的依据。

本篇目录 · 全书目录 · 上一章 · 下一章

附录

> © 2026 S++ project contributors. All rights reserved. Book copyright.

附录 A、B、C、D 按定义、假设、推导与反例展开原理，供需要深入理解的读者阅读。附录 E 提供语言与运行参考，附录 F 提供资料和验证入口。所有公式使用普通文本呈现。

附录目录

- 附录 A 服务基本模型与形式化推导
- 附录 B 交换重排与并行的成立条件
- 附录 C 性能模型与实验方法
- 附录 D 量子服务与守恒机制
- 附录 E 语言与工程参考
- 附录 F 资料来源与验证证据

返回全书目录

附录 A 服务基本模型与形式化推导

> © 2026 S++ project contributors. All rights reserved. Book copyright.

“服务＝参与者×行为”给出了分析业务的起点。本附录先用一笔转账解释符号，再说明怎样分解、怎样组合，以及每个等式究竟在什么意义上成立。符号采用普通文本，所有公式均可在 Markdown 预览中直接阅读。

A1 参与者与行为共同确定活动

考虑甲向乙转移 100 元。参与者配置 p 包含甲、乙以及本次转账约定；行为 b 说明来源减少、去向增加、范围检查和结果要求。活动 B 由二者共同确定：

```
B = p × b

p = {来源：甲，去向：乙，转账约定：100 元}
```

```
b = 按约定完成资金转移
B = 甲向乙转移 100 元这一业务活动
```

p 不是金额向量，b 也不是一个可参与数值乘法的数字。这里的乘积表示构成关系。把乙改成丙，得到另一项活动；把行为改成余额查询，目标和结果也改变。

“缺少任一项则结果为零”的意思是：缺少必要参与者，或没有实际行为，就不能形成所定义的活动。例如转账契约要求收款方，未指定收款方不能当作一笔金额为零的成功转账。业务拒绝与数值零需要区分。

A2 先定义我们比较什么

设状态 x 包含参与者的相关事实。对已经确认身份的甲、乙，可以写为：

```
x = (甲余额，乙余额)
x₀ = (1000，200)
```

一个活动的执行结果还包括业务结果与外部观察：

```
Exec(B, x) = (执行后状态，结果分类，可观察记录)
```

成功执行转账，状态变为 (900，300)，形成与本次交易身份相连的成功记录。拒绝执行则应保持 (1000，200)，给出相应拒绝结果。通信故障可能暂时无法取得确定答复，其实际状态需要另行查询。

两种实现“业务等价”，必须说明哪些记录需要相同。若只要求资金相同而允许凭证内容不同，是一种等价；若还要求相同原交易关联和重复处理结果，就是更强的契约。业务观察范围由服务定义决定，不能在证明中悄悄缩小。

A3 完整服务的组合

这里的完整服务采用传统意义上的宽泛概念，例如通常所说的订单付款服务。它可以组合订单查询、归属检查、金额准备、资金转移与凭证记录。用加号表示活动组合：

```
S = B₁ + B₂ + … + Bₙ = Σ Bᵢ
```

此处 S 表示整项业务，Σ 表示活动组合。映射到 SPP 时，各项明确职责由服务承担，整体组织由场景表达；公式不要求再定义一个普通服务来包含所有调用。组合还包含依赖、提交边界与失败规则，所以不能只给出服务名称之和就认为执行语义已经完整。

例如“资金转移＋订单保存”若共同提交，与资金先提交、订单后提交且需要恢复，是两种执行契约。它们可以以相同成功目标为目的，失败过程和保证范围却需要分别说明。

最小服务与组合边界

上述组合以明确的最小服务为组成单位。量子服务由一对量子属性形成守恒变化；其他服务以针对一个角色的操作为最小单位。属性读取、表达式求值和局部赋值属于服务内部表达，不进一步定义为更小的业务服务。

服务所需的辅助事实与服务的作用对象也要区分。例如明细计价可以使用已准备的价格规则，但作用是形成明细角色的金额；转账约定提供金额，量子服务仍以一对余额的守恒变化为最小单位。集合处理、计算复用和量子请求合并可以覆盖多个这样的单位，不改变基本颗粒度。

A4 按行为分解的分配关系

把复合行为 $b_1 + b_2$ 定义为：在同一组参与者上，依照明确依赖与边界组合两项行为，则有：

```
p × (b₁ + b₂) = (p × b₁) + (p × b₂)
```

等式的左边把整体称为一次活动，右边把同一活动展开为两项职责。它成立需要左右保持相同参与者绑定、输入、顺序要求、提交边界和观察规则。

以两条明细计价及汇总为例，单价和数量分别是 (60，2) 与 (20，3)：

```
明细一金额 = 60 × 2 = 120
明细二金额 = 20 × 3 = 60
订单小计   = 120 + 60 = 180
```

把“逐项计价并汇总”展开为两项计价与一项汇总，仍得到 180。证明依据是汇总读取的正是两项计算产生的金额。若把汇总提前，或改变单项舍入规则，等式需要重新检查。

因此，分配关系说明如何展开定义，并不单独证明子行为可交换。汇总必须等待计价就是一个明确反例。

A5 按参与方分解的是影响贡献

对转账，甲的贡献是余额减少 100，乙的贡献是余额增加 100。将影响向量展开：

```
甲的贡献 δ甲 = (−100，0)
乙的贡献 δ乙 = (0，+100)
整体变化 δ   = δ甲 + δ乙 = (−100，+100)

执行后状态 = (1000，200) + (−100，+100) = (900，300)
```

这给出参与方分配式的一种精确解释：

```
(p甲 + p乙) × b转账 = p甲 × b转账 + p乙 × b转账
```

右边每一项表示同一行为在该参与方上的影响，不是各自再执行一遍完整转账。否则右边会转账两次，显然不等价。

贡献能够分开分析，提交仍要满足整体契约。若仅甲的变化成功，状态 (900，200) 就不符合守恒。相加得到整体变化的计算证明，不能代替实际共同成交的协议。

联合资格也是限制。例如两位申请人共同满足资格，不能直接推断每人各自都满足。参与者分解前要明确哪些部分是局部贡献，哪些规则只能针对整体判断。

A6 量子服务的交换关系

设量子服务 A、B 的变化分别为 δ_A、δ_B，输入中的参与身份与增减量已经准备。两项变化均成交时：

```
先 A 后 B：(x + δ_A) + δ_B
先 B 后 A：(x + δ_B) + δ_A

两者相等，执行顺序不影响共同结果。
```

这里允许两项服务作用于同一个量子属性。例如余额1000，分别扣100与200，两种顺序均得到700。它不依赖两个服务写入不同对象，而依赖确定增减的加法交换性。详细证明见附录 B 的 B0 节。

场景中的一般查询、校验和计算还要检查数据依赖，其重排条件在附录 B 中另行论证。范围拒绝与故障恢复如何保持业务要求，也需要相应执行契约。

A7 业务与技术分离怎样检验

设逻辑业务变换为 F，具体实现为 G，π 将物理记录映射为业务事实。对成功执行，保持业务语义要求：

```
π(G(物理输入)) = F(π(物理输入))
```

例如数据库金额列用分表示，业务用元表示，π 将 10000 分解释为 100 元。一次 10 元转移，物理实现应转移 1000 分，映射后与业务计算一致。若物理实现转移了 10 分，就不满足等式。

完整契约还要求物理实现保持身份、范围、失败与重复执行语义。技术活动不改变业务目标，并不表示其耗时为零或实现责任为零。

A8 从对象执行构造角色行为模型

本节论证建模方法的表达能力，不论证当前 SPP 编译器已经支持任意通用程序，也不论证一种表示比另一种更省代码或更快。“复杂”首先指结构、规则与执行步骤可以继续组合，而不是把一个无限过程在有限时间内全部执行完。

考虑一个已经定义清楚的对象模型。它由对象状态、操作及操作组合构成。把对象身份、相关字段和执行位置共同记为状态 x，把一次操作及其输入记为 a，则一步执行可以写成：

```
对象模型：x --a--> x′
```

这里的状态需要包含会影响后续执行的事实。例如哈希表不仅包含键值记录，还包含容量和冲突处理规则；若扩容会影响后续位置，扩容后的容量也属于状态。若原模型允许多种下一步，例如并发调度，可以把上述关系理解为允许的状态转移集合，而不是单一函数。

为每项对象操作构造一个角色行为：操作目标成为参与该行为的角色，输入值成为必要参与者或行为输入，状态变化遵循原操作规则。一次调用涉及多个对象，就让相关对象分别扮演相应角色。方法调用、条件选择和重复执行，则转为行为间的组合、分支和重复。

设 E 将对象模型状态编码为角色模型状态，构造后的行为记为 b_a。要求每一步满足：

```
原步骤：x --a--> x′
对应步骤：E(x) --b_a--> E(x′)

业务或算法观察：Obs_role(E(x)) = Obs_object(x)
```

可以据此对任意有限执行序列作归纳：

1. 初始状态按 E 编码，相关事实与观察相同。
2. 假设执行第 k 步后两种表示对应，第 k+1 项操作已有按原规则构造的行为，所以状态变化及相应观察继续对应。
3. 因而执行任意有限步后，角色模型仍能表达对象模型已经描述的过程。

例如对象模型中一次哈希保存经历“位置 2 冲突、检查位置 3、写入记录”，角色模型也按相同规则经历这些步骤。若对象模型先扩容后重排，角色模型必须包含同样的迁移与切换，不能只编码最后的键值集合而丢失对后续行为有影响的事实。

这一构造依赖几个能力：能够保存原模型所需的状态，表达原操作的规则，并支持必要的选择、重复与组合。算法中的这些能力不必具有远程服务、业务事务或商业合约的含义。把 SPP 的领域约束硬套到通用建模上，会改变论证对象。

反方向也可以构造：用对象保存角色模型的参与状态，用方法实现行为，用协调对象组织行为组合。它说明两种建模起点可以承载同一可计算过程；设计是否清楚、实现代价多大，还需要独立判断。

A9 表达能力、领域简洁性与实现能力

上节给出的是有条件的构造论证。若没有说明状态、规则和组合如何表达，仅以“尚未找到不能模拟的例子”作为依据，还不能完成普遍性证明。一个反例也需要指明究竟缺少哪项表达能力，而不是仅说明一种写法不方便。

讨论现实世界的模拟时，应先确定可观察范围。例如棋局模型记录规则、位置和胜负，不需要同时记录棋子的材质与每个分子的运动。两种模型对指定观察保持一致，才是这里所说的模拟。无限精度的连续事实或不可计算的规则，不能因为换一种建模起点就变成有限计算可以完整处理的对象。

三个不同问题可以据此分开：

问题	需要说明什么	不能据此直接推出什么
建模方法能否表达复杂过程	状态、变化与组合的构造，及观察对应关系	当前编译器已实现全部所需功能
领域架构能否简洁表达商业服务	角色、服务和场景如何复用领域中的共同结构	所有科学算法都能用几句业务声明实现
当前工具能否运行某项需求	语法、绑定接口、数据表示、执行与资源能力	仅凭建模成立即可保证性能与可靠性

S++ 的简洁性主要来自商业服务领域的共同能力。例如声明付款涉及的参与者、金额和办理关系之后，工具承担标准查询、更新、身份维护和技术接入。若一项需求包含全新的路径搜索算法，算法本身的复杂度仍然存在；可以由已有 Java 实现承担，而不必把它展开为大量业务声明。

复用外部计算时，可以把业务所需结果记为 C，把适配及实现记为 J，输入、输出映射分别记为 encode 与 decode。保持约定需要：

```
decode(J(encode(业务输入))) = C(业务输入)
```

等式还需覆盖约定的精度、缺失输入和失败语义。路线搜索若约定返回道路距离，实现不能返回直线距离却仍称为等价；输入缺少道路资料，也不能静默返回零距离。若采用近似算法，应把允许的误差写入契约，此时比较的是结果是否满足契约，而不是要求精确值相等。

对纯计算，实现不应隐藏数据查询、持久化或交易副作用，否则场景的依赖和事务边界就不再完整。需要外部数据时，先由相应查询准备，再交给计算；需要保存结果时，仍由场景组织更新。通用算法可以复用对象实现，业务责任不能因此变成技术适配中不可见的规则。

因此，一个复杂需求可以由角色模型表达业务，由 APP 的绑定与适配衔接现有能力，再由 PHY 选择实现及资源。对某个绑定接口尚未支持的能力，应按实现缺口处理，不能把它当作建模方法不能表达该需求的证明。

资料来源：S++ 概念与应用 (maintainer reference: `../../../spp/docs/cn/concept.md`)、数值语义与参与者执行。跨文档仍需讨论的问题记录在书稿外的 known issues。

本篇目录 · 全书目录 · 上一章 · 下一章

附录 B 交换重排与并行的成立条件

© 2026 S++ project contributors. All rights reserved. Book copyright.

本附录先证明量子服务增减的交换性：即使作用于同一属性，多个已经确定的变化全部成交后，结果也与执行顺序无关。随后另行证明一般活动的独立读写条件，并讨论并行实现、范围裁决和失败处理。

B0 量子服务的交换性

设 x 为同一业务维度的属性状态向量，量子服务 A、B 分别表达已经准备好的变化 δ_A、δ_B。参与身份、金额与单位已经确定；服务执行不会用中间余额重新定义这两个变化。对两项均成交的情形：

```
F_A(x) = x + δ_A
F_B(x) = x + δ_B

先 A 后 B: F_B(F_A(x)) = (x + δ_A) + δ_B
先 B 后 A: F_A(F_B(x)) = (x + δ_B) + δ_A

由加法的结合律和交换律:
(x + δ_A) + δ_B = (x + δ_B) + δ_A
```

因此，两种顺序产生相同状态。这个证明允许 δ_A 与 δ_B 改变同一个坐标，不要求它们的写集合互不相交。例如账户1000元，两项变化分别为−100与−200，两种顺序都得到700元。

若两项服务分别是甲向乙转100、甲向丙转200，可写为：

```
x = (1000, 0, 0)
δ_A = (−100, +100, 0)
δ_B = (−200, 0, +200)
```

```
两种顺序的结果均为：(700，100，200)
```

每项变化自身守恒，两项共同结果也守恒。顺序改变可能使中间状态不同，但不改变全部变化成交后的结果；不能把中间读数用于重新计算原本已经确定的转移金额，否则讨论的便不再是同一组变化。

对 n 项变化，任意排列 π 都满足：

```
x_final = x + Σ δᵢ = x + δ_π(1) + … + δ_π(n)
```

任意排列都可以通过有限次相邻交换得到，每次交换保持结果，所以整体结果与顺序无关。该结论比较的是同一组成交变化。范围竞争怎样决定哪些请求成交、故障时怎样保证变化不丢失不重复，是另需说明的执行契约，见 B7—B9；不能用交换律代替这些保障。

B1 事实的粒度

把状态看成“事实位置到事实值”的映射。例如：

```
(买家 A，会员等级) → VIP
(商品 P，销售价格) → 60
(订单 O，折扣金额) → 12
```

同一类型的不同对象是不同位置；同一对象的不同属性也可能是不同位置。分析必须包含实际回填范围。例如 mapper 复制整个对象时，不能只把服务明面上使用的一个字段算作写入范围。

对活动 A，R_A 为它可能读取的全部位置，W_A 为它可能改变的全部位置。条件分支会读取的事实也要计入 R_A。若存在别名，要先判断两个名称是否指向同一事实。

B2 证明的假设

以下证明限于满足这些假设的活动：

1. A、B 在讨论的输入范围内确定、终止且成功；拒绝和故障另行讨论。
2. R、W 覆盖全部影响，包括条件判断、对象映射和隐藏访问。
3. 活动只根据自身读取事实计算，只改变声明的写入位置；没有额外外部效果。
4. 比较期间没有第三项活动改变相关输入。
5. 无冲突的逻辑事实写入由实现正确承接，最终结果在两项活动完成后观察。

第五项意味着不能用破坏相邻字段、读取半份对象或未定义内存行为的实现替代逻辑模型。

B3 独立读写的充分条件

设活动 A、B 满足：

```
W_A ∩ R_B = ∅    A 写的事实，B 不读
W_B ∩ R_A = ∅    B 写的事实，A 不读
W_A ∩ W_B = ∅    双方不写同一事实
```

∩ 表示交集，∅ 表示空集。读集合可以重叠：双方读取相同、稳定的商品价格不会造成覆盖。

B4 成功状态交换性的逐步证明

从任意满足假设的状态 x 开始，记 A 单独执行后的状态为 a，B 单独执行后的状态为 b。

第一步，A 只修改 W_A，而 W_A 与 R_B 没有交集。因此，B 在状态 a 中读取的事实，与它直接在 x 中读取的事实相同。

第二步，B 的计算确定，所以先执行 A 不会改变 B 计算出的写入值。反向同理：先执行 B 不会改变 A 计算出的写入值。

第三步，逐个比较最终状态位置：

事实位置	先 A 后 B	先 B 后 A
位于 W_A	A 算出的值，B 不覆盖	A 算出的同一个值
位于 W_B	B 算出的同一个值	B 算出的值，A 不覆盖
两个写集合之外	保持 x 中的值	保持 x 中的值

写集合不相交，每个位置都属于表中唯一一类，所以两种顺序的最终状态相同。若业务输出只是这些最终事实的确定函数，输出也相同。这完成了假设范围内的证明。

B5 为什么可以考虑并行

在上述条件下，A 执行期间无法改变 B 的读取事实，B 也不会影响 A。不同位置的写入不会互相覆盖，因此符合物理实现保障的交错执行可以得到同一最终状态。

但业务不能要求同时观察尚未完成的整份状态。例如一条记录的两个字段分别写完之前，对外读取可能看到半份结果。若只允许完整结果可见，就要加入汇合后的发布、合适的隔离或共同提交。

这也说明，交换性证明没有自动获得线程安全或事务原子性。它提供的是允许并行的逻辑条件，运行实现负责正确调度、写入和发布。

B6 有依赖时的具体反例

设 A 将明细金额设为 120，B 将订单小计设为当前明细金额。初始二者都为零：

```
先 A 后 B：明细 = 120，小计 = 120
先 B 后 A：明细 = 120，小计 = 0
```

A 的写入与 B 的读取相交，结果不同。并行时 B 若先读取旧明细，也会得到小计零。可以并行的是多条互不干扰的明细计价，汇总仍要等待。

B7 数学可交换但实现丢失更新

设 A 给余额增加 10，B 增加 20，初始余额为 100。数学上两种顺序都得到 130。但读后写实现可能出现：

时刻	A	B	余额
1	读到 100		100
2		读到 100	100
3	写入 110		110
4		写入 120	120

增加 10 被覆盖。原子增量、互斥保护或合适隔离可以避免这一问题，但必须由实现提供。共同写入不符合 B3 的充分条件，其他可并发方式需要另外证明。

B8 范围约束改变交换结果

钱包为 100，A 扣 80，B 扣 30，余额不许为负：

```
先 A：A 成功，余额 20；B 被拒绝，最终余额 20。
先 B：B 成功，余额 70；A 被拒绝，最终余额 70。
```

两项增量的代数和与次序无关，但每笔请求是否成功、最终余额均与成交顺序有关。业务若允许竞争者由实际成交顺序决定，可以采用对应执行规则；不能宣称每个请求的结果顺序无关。

B9 失败与外部观察

A 修改甲，B 修改乙，写集合不相交。A 已独立提交后 B 失败，会留下 A；若 B 先失败且场景停止，则没有 A。于是成功路径交换性不能推广为整个失败过程等价。

若要求二者共同成功，需要共同提交或明确的恢复契约。若允许部分结果，需要把部分结果作为业务状态说明。并行分支报错时，还要知道其他分支是否已完成，而不是只终止等待线程。

通知也是观察。扣款后就通知成功、再等待入账，可能把不完整结果暴露给客户。若通知属于完成后的承诺，就要在成交确认之后发布，并安排通知重复与失败处理。

B10 校验并行如何汇合

两个校验读取稳定事实，不修改参与者时，可以分别得到通过或拒绝结果。若要求全部通过，汇合结果为：

```
允许继续 = 校验 A 通过 AND 校验 B 通过
```

AND 的结果与检查次序无关。错误展示若选择“第一个完成的拒绝”，消息会受调度影响；若要求固定业务优先级，就应收集结果后按规定选择。任何“校验失败立即停止”的语言控制还要明确作用范围和对其他分支的影响。

这种论证说明了并行校验需要的契约，不代表任意裸 check 已有对应调度语义。具体表达能力在附录 E 和书稿外的问题记录中说明。

B11 从两项活动扩展到一组活动

为每项活动建立读写范围，根据数据生产、冲突、观察与提交要求画依赖边。对没有依赖的节点应用上述充分条件。合法执行顺序应满足全部前置依赖。

对两个只在独立节点次序上不同的合法顺序，可以通过相邻独立节点交换逐步转换。每次交换保持上述条件内的最终状态，因此整体成功结果保持。存在共同写入、动态别名或外部状态时，不能直接使用这项推论。

B12 完整输入下普通更新的交换性

本节讨论同一个事务边界中的普通更新阶段。普通更新由系统内置提供，开发者不能修改其实现，更新内部不做业务查询、计算或校验。查询与计算已经准备事实，量子服务已经完成该阶段所需的增减并补全结果；若成交之后仍需计算或校验，场景的串行组先完成这些步骤。设此时完整待保存事实为 P，普通更新 U_i 只负责把自己承担的部分 P_i 写入存储。

由于 P 已经准备完整，U_i 的写入内容不取决于其他普通更新的执行结果。证明中使用的保存关系为：不同服务保存不同事实；若它们覆盖同一事实位置，则保存的是 P 中同一个已经确定的值。

对任意事实位置 k，比较 U_A、U_B 的两种次序：

位置 k	两种次序的最终值
只有 A 保存	P_A(k)
只有 B 保存	P_B(k)
A、B 都保存	P_A(k) = P_B(k) = P(k)

位置 k	两种次序的最终值
二者都不保存	保留原值

每个位置的最终值相同，因此：

```
U_B(U_A(x; P); P) = U_A(U_B(x; P); P)
```

任意数量的普通更新可以通过相邻交换改变执行次序，每次交换都保持结果。因此，同一边界内的这些保存动作可以在量子服务之后并发执行，并在全部完成后汇合。这个论证针对完整业务事实的保存；实际并发执行还需保持事务上下文、保护物理写入并落实共同提交或约定的恢复机制。

例如订单状态、付款金额和凭证内容已经准备好，保存订单与保存凭证分别落库。并发不会改变其输入，也不会改变完整保存的业务结果。若凭证内容需要依据成交结果继续计算，场景必须用串行组先执行计算和校验，再启动保存。交换性证明比较的是输入准备完成后的保存动作，不把前序计算与保存互换。

系统内置更新保证保存内部不混入业务计算、查询或服务互调。场景的串行组保证需要先完成的计算与校验按依赖执行，完整输入与一致的保存关系则用于上述交换性证明。相同身份的插入删除冲突、物理外键顺序和运行层并发能力如何处理，记录在书稿外的 KI-011 中。

不同事务可能携带不同的 P，它们之间仍需版本检查、冲突裁决或其他并发保护。本节的结论不将一个事务内部的交换关系推广成任意事务之间的覆盖写入都可交换。

B13 为什么量子交易可以自动提前

同一个事务边界内，设量子交易集合为 Q，普通更新集合为 U。按照本书的服务规范：

1. Q 的参与身份、增减量和调用范围由场景准备，U 不再补全或改变这些业务输入。
2. U 保存完整事实，不产生 Q 所需的业务计算结果，也不修改量子属性。
3. Q 内部各项确定增减符合 B0 的交换关系。
4. Q 完成后，依赖成交结果的事实得到补全；若仍需计算与校验，由场景串行组完成，随后 U 按 B12 的保存关系执行。

由前两项，Q 不存在必须等待 U 的业务依赖。将普通保存动作推迟，不会改变 Q 的成交输入或变化关系。普通更新可能需要量子成交结果，因此依赖方向是“量子结果到普通保存”。编译器可以把边界内的量子交易集中到前面，再安排普通更新。

```
场景准备输入
        ↓
Q：量子交易集中执行（内部可乱序或合并请求）
        ↓
补全成交结果
        ↓
场景串行组：补充计算、校验（按业务需要）
        ↓
U：普通更新并发保存
        ↓
汇合并按事务契约完成
```

Q 内部由交换律保证共同结果不受执行次序影响，U 内部由完整输入下的保存关系保证次序可交换。量子优先的阶段安排则来自二者之间的单向依赖。这三个理由分别说明内部乱序、普通更新并发和跨类别重排。编译器将量子交易提前到普通更新之前，不取消场景串行组明确表达的“成交结果、补充计算、校验、保存”依赖。

重排的是同一事务边界内的执行动作。它不把独立事务并入当前边界，也不自动把多个物理提交阶段变成单一原子提交。LOCAL 可以在共同资源上最终提交；分阶段后端仍需落实自己的提交与恢复协议。规范与具体执行计划实现范围的核对，记录在 KI-012。

依据：参与者执行规范、事务与存储规范与场景组合设计 (maintainer reference: `../../scenario/docs/Composit.docx`)。

本篇目录 · 全书目录 · 上一章 · 下一章

附录 C 性能模型与实验方法

© 2026 S++ project contributors. All rights reserved. Book copyright.

本附录从一项调用的成本开始，推导串行、并行与合并调用的时间，再解释关键路径、容量、完整业务成功率与实验方法。所有数字例子都是假设计算；它们展示模型怎样使用，不代表 Petstore 的实测性能。

C1 定义一次调用的时间

将第 i 项调用沿真实执行路径的耗时分解为：

$$l_i = c_i + e_i + q_i + t_i + d_i$$

符号	含义
c_i	传输及通信往返成本
e_i	编解码与适配成本
q_i	在服务、连接和其他资源上的排队
t_i	业务处理及其资源访问的净执行时间
d_i	未计入其他项的事务建立、协调和提交成本

这些项必须按测量口径互不重复。若服务端计时包含数据库提交，就应进一步分解，或直接使用整体 l_i，不能重复加一次 d_i。时间重叠时也不能简单相加，应按后面的路径模型处理。

C2 串行调用为何累积

若第 i+1 项必须在第 i 项完整返回后开始，第一个开始至最后一个结束包含各项不重叠区间，因此：

$$T_{串行} = h + \Sigma l_i + u$$

h 是共同输入准备，u 是所有分支完成后的公共处理及提交。分支自己的事务若已经计入 d_i，不能在 u 中重复计入。

若 n 项调用都承担相同通信 c 和事务 d，省略其他成本：

$$\begin{aligned} T_{细粒度串行} &= \Sigma(c + d + t_i) \\ &= n(c + d) + \Sigma t_i \\ \\ T_{一次调用} &= c + d + \Sigma t_i \\ 差值 &= (n - 1)(c + d) \end{aligned}$$

这项差值来自重复调用边界。成立的假设是两种实现具有可比业务工作、相同结果要求，并且一次调用确实能够合并通信和提交。若独立事务表示不同业务阶段，就不能为了公式较小而合并其语义。

C3 独立分支并行的推导

设分支同时起跑，各自耗时 l_i；它们不争用导致额外排队，输入已准备，也不需要彼此等待。所有分支完成的时刻等于最晚结束时刻，因此：

```
T并行 = h + max(l₁, l₂, …, lₙ) + j + u
```

j 为未包含在分支内的额外调度和汇合成本。由两式相减：

```
节省时间 = Σ lᵢ − max(lᵢ) − j
并行有净收益的条件：j < Σ lᵢ − max(lᵢ)
```

这个条件说明短任务不一定值得并行。若两个分支各为 1 毫秒，新增调度成本为 3 毫秒，顺序 2 毫秒，并行 4 毫秒。

C4 一次通信与一次提交的并行模型

若整个组合通过一次入口请求，内部业务活动独立、资源足够，并在一次实际共同提交中完成，可以进一步写为：

```
T组合并行 = c + d + max(tᵢ) + j
```

它同时减少重复边界并重叠独立工作。但若每个分支仍远程调用，就应保留分支自己的通信；若量子和普通变化分阶段提交，就应保留各阶段成本。不能仅因为使用 SPP 就删掉这些项。

C5 用完整算例检查公式

设准备 h 为 5 毫秒，查询 l_1 为 40 毫秒，查询 l_2 为 70 毫秒，汇合 j 为 3 毫秒，后续 u 为 10 毫秒：

```
T串行 = 5 + 40 + 70 + 10 = 125 毫秒
T并行 = 5 + max(40, 70) + 3 + 10 = 88 毫秒
节省  = 125 − 88 = 37 毫秒
加速比 = 125 / 88 ≈ 1.42
```

若并行争用让 l_2 变成 110 毫秒：

```
T并行争用 = 5 + 110 + 3 + 10 = 128 毫秒
```

此时结果比原串行更慢。不能继续沿用“分支耗时不变”的预测；需要测量争用后的实际 l_i。

C6 一般依赖图与关键路径

对于任意没有循环的依赖图，设节点 v 的执行耗时为 t(v)。在资源无限、无额外调度成本的理想条件下，节点最早完成时间为：

```
E(v) = t(v)                          v 没有前置节点时
E(v) = t(v) + max{E(p)}               p 是 v 的前置节点时
T理想 = max{E(v)}                    v 是终点时
```

理由是节点只有等所有前置节点完成才能开始，最晚前置完成时刻决定开始时刻。按依赖顺序逐个计算 E，就得到最长依赖路径。

例如查询为 40、70，报价依赖两查询并耗时 10，成交依赖报价并耗时 20：

```
E查买家 = 40
E查商品 = 70
E报价   = max(40, 70) + 10 = 80
E成交   = 80 + 20 = 100
```

实际调度、资源排队和重试只能在这一模型之外增加或改变相应耗时。优化 40 毫秒分支不会缩短 70 毫秒分支所在关键路径。

C7 有限资源的下界

若讨论的是同类处理资源，所有节点占用资源的工作量为 W，最多 k 个等效执行槽，则至少需要 W/k 时间。总完成时间也不能短于最长依赖路径 CP：

```
T ≥ max(CP, W/k)
```

证明第一项：关键路径上的前后活动不能重叠，必须经历其总耗时。证明第二项：长度 T 的区间内，k 个执行槽最多完成 k×T 工作量，所以 k×T 必须至少为 W。

网络等待不应全部算作 CPU 工作，数据库连接也不是 CPU 执行槽。异构系统应对各类瓶颈分别建立资源模型，不能把线程数直接代入上述 k。

C8 单次延迟与整体吞吐

并行可能降低单次请求时间，同时让每个请求占用多个连接。若数据库每秒只能处理固定工作量，工作没有减少时，吞吐未必增加。

在稳定负载和固定统计边界内，平均在途请求数 L、吞吐 λ、平均停留时间 T 具有计数关系：$L = \lambda \times T$。可以通过统计窗口内所有请求占据系统的时间，再分别按请求数和时间求平均理解这个关系。它要求系统统计稳定，不能直接用于不断增长的队列，也不能用高分位代替平均值。

因此应同时测量响应时间、吞吐、在途请求与资源利用。只报告单请求改善不足以说明容量。

C9 实验如何保持可比

选择相同业务输入、数据规模、事实关系、资格、提交保障、协议和资源额度。只改变所研究的调度或部署因素。预热后交替进行多轮对照，避免数据库缓存或运行时编译使固定顺序的一方受益。

串并行实验先保持部署相同；本地与远程对比另作一组。不要一次同时改线程数、存储、提交方式和业务规则，再把全部差异归因于并行。

低负载观察关键路径，高负载逐步增加请求并发，记录何时出现排队、拒绝或资源饱和。通过分支起止事件确认真实重叠，人工延迟只能验证调度，不代替真实负载结果。

C10 指标与报告

保存各请求的开始、结束、结果类别、重试次数和关联分支。延迟排序后报告中位数和高分位，例如采用最近秩法时，p95 取排序后第 ceil(0.95×N) 个值，并说明计算口径。

成功、业务拒绝、技术失败和超时分别报告。不能删除慢失败后只呈现成功平均值。吞吐明确按完成请求或成功业务计数，重放请求与新成交也要区分。

报告同时保留源码版本、构建环境、资源配置、工作负载、预热和轮次、业务正确性检查、原始记录与分析。Petstore 的功能回归不能代替这里定义的性能基准。

C11 远程依赖数量与完整业务成功率

先明确本节的口径：讨论一项合法业务能否在约定期限内完整完成。业务资格拒绝不算技术不可用。单次请求的完成概率也不能直接代替按时间统计的服务可用率。

设一项业务必须调用 n 个远程服务，事件 S_i 表示第 i 项调用在期限内成功。若整项业务要求全部成功，则：

```
P(业务完成) = P(S₁ ∩ S₂ ∩ … ∩ Sₙ)
```

若进一步假设各项成功事件独立，没有重试、替代路径或允许部分完成的规则，且每项成功概率均为 p，则独立事件的乘法规则给出：

```
P(业务完成) = pⁿ
```

```
p = 0.999, n = 20:
pⁿ ≈ 0.980189, 即约98.02%
```

只要 0 < p < 1，增加一项必经调用，就把原完成概率再乘以小于1的 p，因而降低完成概率。这证明的是上述依赖模型中增加必经服务的成本，不能推出所有微服务系统的可扩展性与可用性都不可能兼得。

实际调用还可能共用数据库、网络和宿主，事件未必独立。这时需要联合概率或实测结果；不能继续用 p^n 预测。重试、冗余、隔离、替代路径和恢复会改变事件定义及依赖结构，需按相应协议重新建模。

C12 并发、合并与本地部署分别改变什么

对同一组成功事件，串行与并行都要求所有 S_i 成立。若调度不改变各项成功条件，单纯改变启动次序不会改变交集事件。因此，并发主要改善完成时间；在实际期限固定时，它可以减少由串行累积造成的超时，但不能自动消除分支故障。

合并请求减少重复通信与协议边界。若多项量子变化原本分别访问同一执行端，合并后可以由一次请求交付，降低请求级通信失败与结果确认成本。执行端仍要正确完成全部变化，不能把“入口只有一次请求”当作内部资源已不可能失败。

本地部署使细小的计算、校验不再依赖独立的远程进程与网络请求。逻辑服务数量可以保持不变，远程必经依赖数量却可以减少。宿主、存储和本地执行仍有故障概率，需要另外统计；不能直接把原来的 p^n 换成 p 并宣称获得相应实测收益。

因此，本书讨论的响应时间收益有三条具体路径：服务规范暴露可并发的依赖关系，量子交换性支持请求合并，本地计算与校验减少细小职责的远程调用开销。这些推导不用于宣称 SPP 改善了微服务远程依赖带来的可用性问题。

验证响应时间收益时，按 C9—C10 在相同业务保障和资源额度下比较，并记录实际远程调用次数、超时与失败。失败请求仍需纳入统计，不能删去慢失败后只呈现成功请求的速度改善。

C13 属性使用分析与多级缓存

查询集合怎样确定

设 R 是一个角色，Uses(R) 是在规定编译范围内，场景、计算、校验、量子调用及其范围表达等实际读取的角色属性集合。应沿计算调用图、属性映射和参与者关系收集依赖，不能只扫描一项服务的直接表达式。

角色查询还需要定位与确认参与者，并为后续存储提供相应技术基线。设 NeededIdentity(R) 为必要身份与定位信息，NeededBaseline(R) 为后续保存需要的版本或其他基线信息，则候选查询集合为：

```
Fields(R) = Uses(R) ∪ NeededIdentity(R) ∪ NeededBaseline(R)
```

输出页面或外部响应需要的属性也属于使用范围。业务模型没有直接读取、但 APP 实现或输出协议会使用的属性，必须通过可分析代码或明确的依赖声明纳入集合。缺少这些证据时，应采用保守查询范围，不能把未识别的访问当作不存在。

对于从扮演者映射而来的属性，需要进一步求来源依赖。例如 Buyer 的会员等级来自一个角色，收货地址来自根实体，可以分别生成所需来源的字段集合。映射身份、关联定位和身份确认仍要保持，按需查询不等于任意跳过必要的扮演关系。

正确性目标是：在业务需要读取属性之前，该属性已经由对应事实来源补全。未查询的属性应当与实际空值或零值区分，不能因为查询被裁剪就把默认值当成业务事实。查询优化也不改变入口参与者保持稳定的规则。

部分查询怎样与保存协作

若查询只读取了部分字段，后续内置保存不能把未读取的字段以空值或默认值覆盖。可采用查询范围与加载状态元数据、明确的写入范围或其他保持未涉及事实的机制；版本比较仍须使用有效查询基线。

因此，按需查询是一项从属性分析到读取、映射和保存共同承接的优化。减少 SELECT 列或远程响应字段只是其中一步，不能独自证明优化后的完整业务与原实现等价。

缓存命中需要哪些条件

请求内缓存可以减少同一场景的重复读取，进程内共享缓存可以复用不同请求取得的相对稳定事实。对远程实体，也可以缓存本地角色实际需要的来源事实，再执行角色映射。

缓存身份需要覆盖来源域、参与者身份及必要的业务隔离条件，不能只用对象类型或不完整编号作为键。缓存记录还应标明已加载属性范围；若本次查询要求的集合是 F，缓存已有集合是 C，只有 F 包含在 C 中时，字段范围才满足命中条件。否则应补读缺失部分或重新查询。

刷新与失效可以按事实性质选择版本校验、变更通知或有效期。它们对应不同的新鲜度保证，应由业务要求决定。身份未找到、技术读取失败与缓存过期，也需要分别处理，不能把旧的确认状态无条件沿用。

缓存通常改善读取效率，但不能替代量子成交时的实际范围与守恒检查。缓存余额可以供业务展示或准备阶段参考，成交仍使用执行机制认可的实际事实。事务内受保护的普通读取是否允许使用缓存，同样取决于其版本、隔离和基线要求。

性能收益与验证范围

字段裁剪减少读取、传输和映射的数据量，不保证数据库扫描时间按字段数等比例下降。缓存命中减少实际来源访问，收益还取决于命中率、刷新成本、对象大小和并发竞争。

比较时应保持角色、业务规则和保存结果相同，记录字段数量、响应数据量、实际来源调用次数、缓存命中率、响应时间和刷新成本。验证还需覆盖属性遗漏、部分缓存命中、来源数据变化、版本冲突和未加载字段保存，才能确认性能收益没有改变业务含义。

本节给出优化方法与正确性条件，不作为现有编译器已完成属性裁剪或通用多级缓存的验收证据。实现范围记录在书稿外的 KI-014 中。

来源：S++ 概念与应用 (maintainer reference: `../../../spp/docs/cn/concept.md`)与环境部署配置。

本篇目录 · 全书目录 · 上一章 · 下一章

附录 D 量子服务与守恒机制

© 2026 S++ project contributors. All rights reserved. Book copyright.

本附录用余额与库存说明量子变化、守恒、范围、原子成交和反交易之间的关系。每个结论分别列出数学依据与执行条件，避免用守恒计算代替提交保证。

D1 将变化写成向量

最小量子服务由一对量子属性组成，表达同一业务维度上的等量反向变化。向量可以放在更大的状态空间中表示这一对属性，也可以表示多个量子服务合并后的变化组；状态空间的维数与最小服务的颗粒度需要区分。

设同一资金维度包含甲、乙、丙三个账户，状态为 x。甲向乙转 100 元：

```
x = (1000, 200, 50)
δ = (−100, +100, 0)
x' = x + δ = (900, 300, 50)
```

坐标对应完整参与身份及属性，不能只用“余额”二字定位。金额需要采用相同单位和精度约定。库存则按商品分别建立坐标。

D2 守恒证明

设总量 Q(x) 为同维度各坐标之和：

```
Q(x) = Σ xᵢ
Q(x + δ) = Σ(xᵢ + δᵢ) = Σ xᵢ + Σ δᵢ
```

所以，当且仅当增量之和为零时，前后总量相同：

```
Σ δᵢ = 0  ⇒  Q(x + δ) = Q(x)
```

上例增量合计 −100＋100＋0＝0。若甲减少 100，乙只增加 99，差额为 −1，便不满足所定义的守恒关系。费用如果转入另一个收费账户，应把它作为真实去向纳入模型。

此结论只证明完整向量被应用后的总量。甲先减钱、系统崩溃、乙未加钱，实际状态仍可能破坏守恒，因此还需整体提交。

D3 独立维度分别证明

商品 P 的可售、预留、已售和供应来源可以表示为：

```
入库后:          (10, 0, 0, −10)
预占 2 件后:     (8, 2, 0, −10)
发货 2 件后:     (8, 0, 2, −10)
```

每行合计都为零。另一个商品 Q 应有独立等式。P 少一件、Q 多一件，虽然全部件数相抵，仍没有证明 P 的业务数量正确。

金额与件数也不能相加。涉及货币兑换或计量换算时，需要明确汇率、单位、差额来源与舍入政策，不能机械使用直接求和。

D4 准备输入与实际范围

调用方准备固定金额 a，并约定成交后范围。成交实现读取执行时的余额 x，而不是直接覆盖事前快照：

```
候选值 xᵢ' = xᵢ + δᵢ
要求: Lᵢ ≤ xᵢ' ≤ Uᵢ
```

null 表示关闭相应方向。范围可以由参与者与合约事实在调用前确定，实际动态余额则由成交机制处理。

初始余额 100，两笔各扣 80 的请求都可能通过事前查询。若第一笔成交后余额 20，第二笔实际候选为 −60，应被拒绝。范围计算必须与受保护更新处于同一成交机制内，不能先检查再无保护写入。

D5 什么叫整体成交

对一个量子变化组，业务要求成功应用全部 δ，或拒绝时不应用该组变化。执行还要保证未完成的部分状态不被当作成功发布。

一种本地实现可以在同一数据库事务内读取受保护事实、计算、检查并写入，最后共同提交。其保证依赖实际连接、隔离或原子条件写入，以及数据库事务能力。

跨资源实现要提供相应协调与恢复契约。声明“同一事务”不能使多个互不协调的连接自动拥有共同原子提交。量子与普通更新分阶段成交时，也不能把普通阶段失败当作已提交量子组的本地回滚。

D6 合并提交与增量汇总

设多个量子服务的变化已经确定。由附录 B 的 B0 节，它们全部成交后的共同结果与执行顺序无关。因此，提交方可以将多项服务放进一个量子请求，不需要以逐项发送、等待响应的方式固定次序。执行端接收完整请求后，统一安排变化并处理共同提交。

合并请求可以保留各项变化、请求身份、范围与结果对应关系。若执行契约允许进一步汇总，则按完整参与身份与业务维度求净增量。例如：

```
甲→乙 100: δ₁ = (−100, +100, 0)
乙→丙 30:  δ₂ = (0, −30, +30)
净变化:    δ = δ₁ + δ₂ = (−100, +70, +30)
净变化合计 = 0
```

守恒在代数合并下保持，因为各 δ 的和均为零。合并通信与净值汇总是两个步骤：一个大请求可以携带原始变化明细，不必只保留最后的净值。范围检查在逐项与净值两种语义下未必相同。

例如初始为 (0，0)，先甲→乙 100，再乙→甲 100，净变化为零。只检查最终净值会通过；若每个步骤都要求来源不透支，第一步就不允许。逐步检查和净值检查表达不同成交要求。系统必须明确采用哪种边界，不能由最终守恒自动推断业务允许中间透支。

通信收益可以用一个简化模型说明。设有 n 项变化，每次请求的固定通信与协议开销为 h，单项变化数据的传输开销为 p：

```
逐项请求的通信开销: n × h + n × p
合并请求的通信开销: h + n × p
减少的固定开销: (n − 1) × h
```

这个模型说明合并减少的是重复的请求开销，变化数据本身仍需传输。它不是具体系统的耗时预测；请求大小、响应明细、分片路由与实际协议会影响收益。即使提交端只发出一个请求，执行端也可能需要跨资源通信，因此“一次提交”不能直接理解为后端没有协调成本。

合并后仍需保留识别重复、解释原交易和恢复的证据。净变化为零也不等于业务从未发生。合并的目标是利用交换性降低通信损耗，同时保持原有业务含义与执行保障。

D7 请求身份与幂等

请求身份应当绑定金额、参与端点、变化组和范围等不可漂移事实。相同身份重试时，若已成功，应返回原结果而不再次应用 δ；若事实改变，则不能当成原请求重放。

这需要持久化的身份与结果证据，并使证据与实际写入具有可靠联系。“先写余额再写去重记录”中间崩溃，可能重复入账；反过来先写成功记录也可能形成记录成功而余额未变。具体后端需要解决这一提交关系。

D8 反交易为何保留后续变化

设原状态为 x，原成交变化为 δ，此后其他业务产生净变化 e。执行反交易 −δ：

```
反交易前 = x + δ + e
反交易后 = x + δ + e − δ = x + e
```

因此，增量的反向关系保留 e。余额 100，原增加 50，后续增加 20，反向减 50 得到 120；恢复旧快照 100 则会抹去后续 20。

量子服务由系统依据原成交记录全自动执行反交易，无需业务开发者另外实现反向服务。上述结论要求系统准确保存原成交分量和参与身份，相关变化仍能以加法表示，并保证反向执行不会重复。这些是自动反交易的执行责任；状态赋值或不可撤回的外部效果不能直接套用这一证明。

D9 自动反交易与后续业务的区别

失败恢复中的量子反交易依据原成交事实恢复该笔变化，不需要业务开发者另外声明反向交易及其补偿逻辑。原成交定位、反向约束的执行机制、持久结果、防重和技术恢复由系统承接，技术处理策略在 APP 层定义。

业务完成后另行提出的退款、冲正或其他变更，需要先明确其业务含义。例如，退货只退部分款项或扣除费用，金额已经不同于原交易的相反增量，应按新业务规则办理，不能仅凭自动反交易推导。原成交值为 δ 的自动反交易是 −δ；新的业务变化若为 δ′，其金额与资格应由该业务确定，不能混为同一操作。

D10 未知结果与重试

提交完成、响应返回之前断线，调用方看到超时，余额却可能已变化。稳定身份查询应区分已确认成功、已确认拒绝、协议确认未执行和未知。

未知不等于未执行。使用新身份重发可能造成第二次转移；直接补偿也可能撤销一笔并未发生的交易。应当先按后端协议确认原结果或等待恢复证据收敛。

D11 业务边界与实现范围

关系	需要确认的实际保障
LOCAL	同 JVM、同实际数据库与 schema、共同连接和提交资源
非 LOCAL 混合阶段	量子组与普通阶段的明确成交顺序、持久结果与恢复规则
REQUIRED	复用外层边界，不增加独立提交点
REQUIRES_NEW	独立资源与传播能力，以及外层失败后的处理约定
不支持的传播或协议	构建或部署明确拒绝，不能悄悄降低保证

具体示例后端能力见附录 E、F。部署名、协议名与测试通过数量都不能独自证明任意跨资源组合已实现。

D12 成交后的数值不确定与观测

设请求准备时查询到余额 b，本次已经成交的变化为 δ。从该次查询到成交后观测时刻，其他请求中实际生效的变化集合为 C。若这些变化都可以按同一余额维度相加，则观测余额为：

```
b_observed = b + δ + Σ e⌋     (j ∈ C)
```

即使已经确认 δ 成交，若尚不知道 C，就不能只用 b + δ 确定成交后观察到的余额。设 b = 1000，δ = −100，另一请求的变化 e = −200：

```
另一请求尚未生效：C = ∅，结果为900
另一请求已经生效：C = {e}，结果为700
```

900与700分别对应不同的生效事实，两者都以本次扣除100已经成交为前提。这里的叠加表示成交后、观测前的数值不确定性，不表示数据库同时存放两个余额，也不要求为它们赋予物理量子系统的概率或振幅。已确认的是本次变化成功；仅凭这一确认，还不能知道观测时包含哪些并发变化。

对于同一个已经确定的成交集合，加法给出相同最终值：

```
(1000 − 100) − 200 = (1000 − 200) − 100 = 700
```

但加法等式不证明所有请求都能成交。例如余额100，两笔分别扣80和30，若要求余额不低于零，就不能同时成功。执行次序与裁决规则可能影响哪笔请求被拒绝。候选状态必须经过范围与成交规则筛选，不能把数学上可写出的每个值都当作合法结果。

成交确认与数值观测回答不同问题。前者回答本次增减是否成功，后者回答观测时属性值是多少。一次准备阶段的查询不能冻结后续变化；一次成交确认也不把余额固定为事前余额加本次增减。观测得到的值需要有明确的读取语义，此后仍可能继续变化。

传统的锁定读、计算新值、覆盖写入，在锁保护期间确定本次结果。量子服务则以增减表达变化，将动态数值与约束交由成交机制处理。这一语义区别并不单独证明后端没有锁：后端仍可能在实际应用增量、检查范围或提交时使用短时锁、原子条件更新或其他协调机制。具体执行能力应由实现证据说明。

依据：量子调用范围、事务与存储、环境与物理配置。需要继续讨论的概念契约见 known issues。

本篇目录 · 全书目录 · 上一章 · 下一章

附录 E 语言与工程参考

© 2026 S++ project contributors. All rights reserved. Book copyright.

本附录提供初稿采用的语言约定和实验命令入口。它是阅读辅助，完整配置仍以对应模块源码和运行说明为准。命令在仓库根目录执行。

概念速查

概念	职责
Entity	当前域拥有的自然实体
Reference	明确来源的外域实体、角色或合约引用
Role	业务参与身份及角色事实
Contract	场景的当事人、约定与结果
take ... as	声明单方、联合或再次扮演关系
mapsame	显式启用满足条件的同名属性映射，默认不启用
takeall	明确继承扮演者属性，并生成相应映射
business	声明疑似技术 ID 的真实业务含义，消除习惯性 ID 警告
query	补全参与者并承接身份确认
check	判断业务条件，不修改参与者
calculate	形成候选事实，可组合纯计算
update	实施明确变化，由应用与资源兑现
quantum update	定义量子增减与成交关系
Scenario	组织完整业务活动
transaction ... fail	事务执行与技术恢复后，办理明确的失败业务处置
ResultInfo	场景内置系统参与者，提供执行结果与失败恢复事实，不作为原子服务参数

服务入口采用参与者或参与者集合，实例名可以省略。无参数调用需要唯一类型绑定。没有业务返回值，标量入口实例保持稳定；集合内容可以更新，生成 Java 采用可变容器。

普通 check 失败结束词法范围，缺省不触发回滚。显式撤销采用仓库约定的 `breakTransction` 拼写，支持条件和暂定控制流见参与者执行规范。正文没有将这种控制方式当成所有事务模式的通用能力。

示例后端的能力范围

下面列的是案例源码基线中的具体能力，不用于重新定义服务、角色或事务概念：

项目	使用条件
LOCAL	相关域同 JVM，并使用可核对的共同数据库、schema 和连接资源

项目	使用条件
REQUIRED	复用已有执行与提交上下文
REQUIRES_NEW	仅受支持的同质外层及本地资源组合；独立结果另行处理
NESTED	示例后端不支持，构建拒绝
parallel 内裸 check	停止范围有限制，使用受支持串行或 guard 表达
增强 for	按集合顺序执行，不自动变为并行集合计算
calculate 调用	可以组合纯计算，禁止循环调用图和技术副作用
fail 块	当前支持串行场景的外层事务；多个合约用 fail(目标合约) 选择；嵌套及并行分支内拒绝
远程事务与量子协议	需要实际协调能力及恢复证据，缺失时拒绝配置

这些能力的具体判定见参与者执行、事务存储和环境规范。书中的原理推导说明某种组合何时合理，后端还需真正提供对应的运行能力。

各类业务服务的实现依据

第一篇第7章的调用、流程与执行特点，可沿以下入口核对：

特点	实现或示例入口
独立查询并发，完成后再使用事实	下单场景、付款场景
查询原地补全与身份确认	参与者查询绑定、参与者执行
普通查询与普通更新均为内置服务，开发者不修改实现；场景组织调用、计算与校验	内置 CRUD 规范
普通原子服务不允许调用其他服务，计算调用图禁止循环	SppDomain 校验 (maintainer reference: `../../lang/src/main/java/free/servpp/lang/models/SppDomain.java`)中的 `validateBusinessCalculations` 、 `validateCalculationCycles`
计算只调用计算，校验不修改参与者，计算不能使用 require	业务计算校验器 (maintainer reference: `../../lang/src/main/java/free/servpp/lang/models/app/expr/BusinessCalculationValidator.java`)
订单报价复用明细计算	Petstore 定价服务中的 `quoteOrder` 与 `priceLine`
校验控制词法范围、局部 guard 与显式撤销	参与者执行规范、原子服务规范
事务恢复后记录业务拒绝	预约场景、失败处理与系统结果
计算与校验可以生成本地实现	SppService (maintainer reference: `../../lang/src/main/java/free/servpp/lang/models/SppService.java`)中的 `isLocalService` 、生成器 (maintainer reference: `../../generator/src/main/java/free/servpp/generator/openapi/MustacheClassWriter.java`)、Petstore APP

查询并发与本地顺序执行是根据职责和依赖选择的安排，不是由服务类别自动获得的调度效果。本地执行也不等于不能处理并发请求。第一篇说明常见业务的安排，第二篇进一步讨论实际成本与依赖。

构建与测试

```
mvn -pl :petstore-demo -am package -DskipTests -Dspp.environment=develop
mvn -pl :petstore-demo -am test-compile -DskipTests -Dspp.environment=testing
mvn -pl :petstore-demo -am test -Dtest=PetstoreScenarioTest -Dsurefire.failIfNoSpecifiedTests=false
```

前两项分别构建、编译，均跳过测试执行；第三项实际执行所选测试。模块回归不能仅从 package 成功推断。公共联合回归可沿最新 Petstore 验收记录中的命令执行。

本地网页

```
SPP_PETSTORE_JDBC_URL='jdbc:h2:file:/tmp/spp-book-develop' \
  ./demos/petstore/run-demo.sh web
```

默认端口为 8086。已有构建可设置 `SPP_DEMO_SKIP_BUILD=1`，但必须确保产物对应当前环境。固定文件路径会保留前次实验数据，需要根据实验轮次选择路径。

三进程 testing

先构建 testing 产物，在三个终端设置同一调用凭据和实验资源。以下变量块应在三个终端分别执行，凭据占位值替换为本次实验值：

```
export SPP_ENVIRONMENT=testing
export SPP_DEMO_SKIP_BUILD=1
export SPP_DOMAIN_RPC_TOKEN='replace-with-session-token'
export SPP_TESTING_CUSTOMER_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-customer'
export SPP_TESTING_PRODUCT_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-product'
export SPP_TESTING_SHOP_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-shop'
export SPP_TESTING_LEDGER_JDBC_URL='jdbc:h2:file:/tmp/spp-book-testing-ledger'
```

随后先启动提供方，确认就绪后启动商店应用：

```
# 终端一，默认端口 8087
./demos/petstore/run-demo.sh web customerService
```

```
# 终端二，默认端口 8088
./demos/petstore/run-demo.sh web productService
```

```
# 终端三，默认端口 8086
./demos/petstore/run-demo.sh web petstore
```

跨主机需要修改端点和量子资源，确保所有参与方使用同一个真实量子系统。文件型 H2 示例不能仅通过复制路径字符串变为跨主机共享数据库。

数值与输出

SPP 的数字规格提供默认提示，PHY 全局替换与字段覆盖选择实际表示，字段覆盖优先。容量检查不能代替业务舍入。生成产物在 target，业务源码在 `src/main/spp`。

阅读入口：运行说明、启动脚本、参与者执行、数值语义与环境配置。

本篇目录 · 全书目录 · 上一章 · 下一章

附录 F 资料来源与验证证据

© 2026 S++ project contributors. All rights reserved. Book copyright.

本附录提供概念、源码、执行与性能资料的查阅入口。书中概念作为同一个体系阐述，跨文档仍需统一的问题单独维护在 编辑工作记录，不进入正文的叙述线索。

F1 理论与方法资料

资料	可查阅内容
S++ 概念与应用 (maintainer reference: `../../../spp/docs/cn/concept.md`)	服务模型、分解组合、量子服务、多态、业务技术分离与性能模型
S++ 建模案例 (maintainer reference: `../../../spp/docs/cn/modeling.md`)	商店参与者、身份、业务约定与存储组织
场景组合设计 (maintainer reference: `../../scenario/docs/Composit.docx`)	职责分类、依赖分组与组合调度
应用框架设计 (maintainer reference: `../../lang/doc/ApplicationFrame.docx`)	调度环境、技术处理与事务组织

相邻 spp 仓库资料需要单独取得。独立克隆本仓库时，指向该仓库的相对链接需在同级目录备齐资料后使用。

F2 语言与工程规范

内容	资料
模型意义及参与关系	业务模型边界
查询、身份与控制范围	参与者执行
业务计算与规则	原子计算
数值表示与业务舍入	数值语义
量子成交范围	量子调用范围
事实归属、提交及恢复	事务与存储
多环境资源与部署	环境与物理配置

Petstore 代码统计保留 F7 所列的独立基线；本次银行案例与新增能力核对以主线 `0fafa21d` 为基线。规范说明目标与约束，源码说明表达与实现，运行记录提供特定路径的证据。单项资料不能独自代替全部层次。

F3 Petstore 功能验证

2026 年 10 月 13 日 APP 技术服务验证记录完整生成与回归 1,123 项通过：语言 362、运行框架 617、生成器 105、Petstore 39，失败、错误、跳过均为零。覆盖匹配和编译检查、调用阶段、技术访问拒绝、观察失败隔离、并发和幂等重放；远程代理使用可控传输验证，未进行真实跨 JVM 验收。Petstore 样例展示商店全服务日志拦截。该能力来自主线 `65c766fc`；本书此次引用已有测试记录。

2026 年 10 月 13 日失败处理验证记录 develop 完整生成链路回归 1,104 项通过：语言 356、运行框架 607、生成器 105、Petstore 36，失败、错误和跳过均为零。预约专项说明说明真实 fail 样例的库存不足、普通保存失败与幂等重放结果。框架专项另验证真实 JDBC 量子成交后的自动反交易先于 fail 执行；本轮 Petstore 使用 LOCAL，不代表已经验收真实分布式 TCC 或外部量子服务。本书此次引用已有记录，没有重新运行测试。

2026 年 10 月 12 日最终回归记录报告：develop 联合回归共 1080 项，分别为 lang 340、appframe 581、generator 105、appframe-seata 20、Petstore 34，失败、错误和跳过均为零。

同一记录报告 develop 完整 Chrome 流程，以及 testing 三 JVM、四个隔离文件数据库的完整网页与商店重启验证。停机后核对资金来源与去向合计、按商品核对库存与供应来源、按交易核对流水增减合计。

这些是仓库已有验证记录，编写书稿时没有重新运行对应业务测试。记录中的临时日志路径用于追踪当次证据，独立环境需按记录重新执行才能取得新的运行结果。

已知问题记录初始化重试的实现边界，以及分库零记录技术恢复的已实现范围；具体机制与验证见 分库零记录技术恢复。功能回归没有提供附录 C 所定义的串并行性能基准。

F4 某银行存款案例验证

当前某银行存款案例已完成对齐新语法与运行协议的重构。生产源位于案例目录，只使用 SPP、APP、PHY，没有手写生产 Java。SPP 不使用 `try/catch/throw`；生成的 Java 与运行时内部仍可用异常处理承接技术故障。历史源仅保留作规则对照，不能作为当前可执行示例。

本节以主线 `0fafa21d` 为当前案例检查基线，引用仓库中已有记录，本次书稿修改没有重新运行银行业务测试。最新验证记录按批次保存命令、结果及适用范围：

验证批次	结果	涵盖范围
2026 年 10 月 13 日 clean develop 联合回归	1,226 项，零失败、零错误、零跳过	语言 366、运行框架 628、生成器 107、某银行案例 86、Petstore 39；包含内部收款户、历史快照及已加入的故障与并发测试
随后 testing 银行全量回归	86 项，零失败、零错误、2 项跳过	本 JVM 内独立量子与普通 H2 数据源；跳过两个 LOCAL 专属提交故障测试
独立现金冲正试算专项	develop、testing 各 2 项通过	试算不写入，正式冲正保留后来存入；缺原成交拒绝无副作用
中文工作台专项	两环境后端专项及浏览器流程通过	精确小数显示、负数拒绝、试算、冲正、回执查询与移动布局；具体范围见工作台验收

testing 的分库证据不能直接扩大为真实网络分布式部署、XA、Seata AT 或 TCC 的完整验收。独立 JVM 重启与成交确认后强制退出，覆盖各自规定的持久化路径，不能代替提交中断或未知成交的自动查证。后续专项也不能直接累计成一次未执行过的全量联合回归。

完整性审计说明已落实的场景、存储、事务与技术能力。86 个历史入口核对台账逐项列出证据、差异及未迁移部分；86 个历史入口与上述 86 项 JUnit 测试是不同统计，不能因数字相同而认为已经逐项等价。

已知问题仍保留普通 Saga 已提交阶段恢复、待授权持久化与跨进程续办、未知成交自动确认、全调用持久化审计等边界。当前分库失败可以自动恢复量子资金，并可能留下普通已提交事实；系统明确报告 `RECOVERY_REQUIRED`、`handled=false`，不声称全回滚。

来源映射提供当前能力目录和源程序规则定位。源码推导、业务数学预期、演示输入、生成执行对照、原 COBOL 实测与银行确认结果代表不同证据。案例成功不能扩大为全部银行政策或原系统输出等价。

F5 QService 资料

相邻 qservice 仓库的 SPP 边界契约 (maintainer reference: `../../../qservice/docs/spp-qservice-boundary-contract.md`)与量子服务设计 (maintainer reference: `../../../qservice/docs/qservice-early-design.md`)描述逻辑命令、持久身份、状态查询和恢复。

该项目的集成产物和测试路径有自己的基线。使用其中一种后端时，需要核对真正接入的协议与执行能力，不能把独立项目的全部验证结论直接扩展到任意部署。

F6 证据怎样使用

解析说明表达被接受，编译说明产物通过相应静态检查，运行说明所测路径，故障与重启验证进一步覆盖特定失败条件。性能证据还需要明确资源和工作负载。

阅读结论时，可先找到它对应的参与者、行为和配置，再查看输入、预期、实际结果与失败范围。保持这些信息关联，概念论证才能与可复查的工程实践连接。

F7 Petstore 代码量统计

2026 年 10 月 13 日，以本书工作分支 `5b9b182c` 的 Petstore 源码进行统计。该次书稿修改未改变案例源码。从本工作区原先不存在的生成 Java 目录完成 develop 生成，避免将其他版本遗留文件计入。命令在仓库根目录执行：

```
mvn -pl :petstore-demo -am package -DskipTests -Dspp.environment=develop
```

统计输入为 `demos/petstore/src/main/spp/` 下的 `.spp`、`.enum`、`.app`、`.phy`，输出为 `demos/petstore/target/generated-sources/openapi/src/main/java/` 下的 `.java`。不计测试、网页、资源文件、构建脚本、生成器和公共运行框架依赖的代码；Java 数字包含 S++ 生成器及 OpenAPI 生成器的输出。复现时使用干净生成目录，源码或工具版本变化后应重新统计。

文件类别	文件数	物理行数，含空行与注释	去除空行与注释后的行数
SPP	63	1,104	999
枚举	2	25	18
APP	4	126	117
PHY	4	222	214
输入合计	73	1,477	1,348
生成 Java	962	131,107	100,151

生成 Java 中，模型目录占 64,431 行，服务目录占 21,031 行，接口与处理目录占 14,535 行，映射及配置占 154 行。这说明模型和请求响应展开占较大比例，不能将每一行都解释为需要人工设计的新业务逻辑。

以下脚本按物理行计数，去除行注释、块注释与空行，保留字符串中的注释符号及独立括号行。它适用于本次源码，不是所有语言结构的通用代码分析器：

```
from pathlib import Path
import re

pattern = re.compile(
    r'"(?:\\.|[^"\\])*"|\'(?:\\.|[^\'\\])*\'|//[^\n]*|/\*[\s\S]*?\*/'
)

def code_lines(source):
    def remove_comment(match):
        token = match.group()
        if token.startswith(("//", "/*")):
            return "\n" * token.count("\n")
        return token
    source = pattern.sub(remove_comment, source)
    return sum(bool(line.strip()) for line in source.splitlines())

source_root = Path("demos/petstore/src/main/spp")
java_root = Path("demos/petstore/target/generated-sources/openapi/src/main/java")
for suffix in (".spp", ".enum", ".app", ".phy", ".java"):
    root = java_root if suffix == ".java" else source_root
    files = sorted(root.rglob("*" + suffix))
    sources = [path.read_text(encoding="utf-8") for path in files]
    print(suffix, len(files),
          sum(len(source.splitlines()) for source in sources),
          sum(code_lines(source) for source in sources))
```

SPP 中一行可以包含多个声明或调用，Java 的模板格式、访问方法和请求响应类型又会扩展物理行数。因此这是一组明确版本下的源码规模统计，不是语言表达能力、生产率或性能的统一比例。准确测量手写 Java 的规模，还需要具有同等业务范围、技术能力和验收条件的独立实现。以下只给出按职责拆分的工程估算。

使用常见 Java 框架的估算

假设采用 Spring Boot＋MyBatis＋Lombok，复用框架与已有通用组件，不为每个内部服务额外建立一套独立远程接口，也不手写访问方法。Lombok 可以生成访问方法等模型重复代码，MyBatis 提供 SQL 映射，Spring 提供声明式事务支持；这些已有能力在估算中视为依赖，不计入项目手写代码。参考 Lombok Data、MyBatis Mapper 和 Spring 声明式事务。这些文档说明框架能力，下面的行数是本书的工程判断，并非文档提供的基准。

|需要维护的内容|估算行数|估算理由| |业务模型、DTO、枚举|1,000～2,000|四个域的参与者与请求结果，复用模型，使用 Lombok| |入口、参数转换与对象映射|1,000～2,000|34 个场景的入口及边界适配| |场景编排、计算与资格校验|3,000～5,000|每个场景约 90～150 行，包含共享规则及依赖组织| |存储接口、SQL、映射与表结构|2,000～4,000|普通事实、成交快照、版本条件和查询关联| |守恒交易、幂等与失败恢复接入|2,000～5,000|双边变化、范围控制、原请求确认与阶段恢复| |域调用、环境配置与技术错误处理|1,000～2,000|本地与远程调用适配、资源与环境选择| |合计|10,000～20,000|项目 Java、SQL／Mapper 和必要配置的合计|

各行按主要职责分配，避免重复计入。例如业务拒绝规则计入场景与校验，远程超时确认计入恢复与技术处理。估算没有加入第五篇提出、但案例尚未完整实现的通用缓存裁剪、全套安全治理等扩展需求，也不包含公共框架本身的开发。

若只做常规单体商店，采用本地事务、合并部分模型、减少跨域适配，并把恢复缩减为本地回滚，项目可能约为 6,000～10,000 行。但这一较小范围没有覆盖多域部署及其结果确认与恢复职责，应单独列出，不能直接作为相同能力的比较。

已有账务、幂等与协调中间件越完善，需要手写的代码越少；反之，自行开发通用恢复设施会超出上述范围。行数还受格式影响，SPP 当前有多条语句写在一行的情况。因此“一万到两万行”只用于解释项目维护规模，不能推导精确的生产率倍数或开发工时。

F9 某银行案例的 COBOL 与 S++ 代码量

2026 年 10 月 14 日，以本仓库 `92ed491b` 的案例源码和来源文档为统计基线。原 COBOL 与 COPYBOOK 从本地银行来源目录读取，没有改动，也没有复制原源码到本书。该目录需单独取得；统计文件清单与数字保存在本附录的统计结果中。

来源范围与去重

“直接定位分析”口径选取来源文档中有程序文件名或具体行号定位的 9 个 COBOL 程序。这里的“定位分析”表示已有源码追踪依据，不表示整份程序的所有分支都已迁移：

程序	来源职责	物理行数	去除空行与注释后的行数
GSA01102	一般存入前台受理	2,325	1,820
KSADEP0	存款核心、偿还分配及冲正	4,197	3,508
CSAQRY1	账户资格与受理检查	2,654	1,867
KSAFEE0	收费入口	384	310
CSAFEE0	收费规则	415	331
CCMDAT1	日历与天数	2,856	2,563
CSAPRD2	存款及协议积数	483	315
CSAPRD4	透支积数	237	154
GCCBAAR	账务规则接入与配置来源分析	605	542
合计	9 个程序	14,156	11,410

从上述程序的有效 `COPY` 语句追踪，直接引用 135 个 COPYBOOK，再加入嵌套引用新增的 26 个，共 161 个。共享 COPYBOOK 按实际文件去重，同一文件被多个程序或多次引用只计一次；不会按调用次数或展开次数重复累计。

更宽的“全部相关”口径选取来源文档明确提及、且在原源码目录中实际存在的程序，共 22 个。它包含调度、查询、配置等相关程序，尚未递归扩展为全部 `CALL` 依赖图，不能称为整套银行系统的代码量。

来源范围	COBOL 程序数	去重 COPYBOOK 数	程序物理行数	COPYBOOK 物理行数	合计物理行数	合计去除空行与注释后的行数
直接定位分析	9	161	14,156	23,488	37,644	22,589
文档提及的全部相关程序	22	192	20,626	24,961	45,587	28,358

逐文件清单分别见直接来源清单与全部相关来源清单。清单只保存文件位置与行数，不包含银行源码内容。

当前 S++ 源码

仅统计案例生产源目录中的 SPP、枚举、APP、PHY，不统计历史资料、Java 测试、测试输入、网页、构建脚本、生成 Java 或公共编译器与运行框架。

源码类别	文件数	物理行数，含空行与注释	去除空行与注释后的行数
SPP	34	1,037	973
业务枚举	1	6	6
APP	2	48	48
PHY	2	38	38
合计	39	1,129	1,065

SPP 与枚举合计 979 行，表示当前需要维护的业务模型、服务、场景和有限取值；APP、PHY 再增加 86 行。当前有多条 S++ 语句写在同一物理行，因此不能将行数比例直接当成算法复杂度、工作量或生产率比例。

计数与复现

物理行数按文件原始换行计数。COBOL 去除空行与注释后的行数按固定格式处理：排除第 7 列为 `*` 或 `/` 的注释行，在第 8～72 列取有效内容，移除 `*>` 后的行内注释，再计非空行；不把序号列及右侧标识区当作代码。该计数仍包含数据定义、`COPY` 指令、过程与控制声明，不是业务语句数。

COPYBOOK 使用同一规则。SPP、枚举、APP、PHY 去除 `//` 与 `/* … */` 注释，保留字符串中的注释符号，计剩余非空物理行。统计没有把 COPYBOOK 展开到每个程序中重复累计。

复现脚本接受原 COBOL 根目录作为参数，可在仓库根目录执行：

```
python3 book/附录/统计资料/count-bank-code.py /path/to/original/cobol-root \
  --output-dir /tmp/bank-demo-code-count
```

根目录中应具有原程序与 COPYBOOK 的分类目录。脚本输出两份来源清单和汇总 JSON；源码或来源文档变化后，选取范围与结果也可能变化，应重新统计并标明版本。当前两种来源范围均没有未解析或歧义的 COPYBOOK 引用。

比较结论的范围

以上来源规模与当前实现规模可用于解释表达的集中程度，不能证明所有来源代码已经完成等价转换。9 个或 22 个来源程序的全部内容包含更多交易方式、共享配置及未迁移分支；S++ 案例只覆盖已明确实现和验证的业务路径。

角色与服务模型集中表达这些路径中的规则、参与关系与执行边界，工具提供大量重复实现，APP、PHY 接入存储与运行能力。案例仍依赖公共框架，科目及外部事实仍有演示配置，授权续办和普通 Saga 恢复等边界见 F4。不能把来源全部行数与案例行数的商，写成无条件的压缩倍数或完整替代结论。

本篇目录 · 全书目录 · 上一章